

Combined Confluence Export

Global Table of Contents

API Governance: Approved Artifacts

- [API Governance: Approved Artifacts](#)
- [API Governance Metadata Model](#)
- [Mastering API Governance: The New Automated Prompt for OAS Review](#)
- [Lumen API Governance — Comprehensive Review Prompt](#)

Architectural Principles & Patterns

- [Architectural Principles & Patterns](#)
- [A Strategic Four-Layer Approach to API Architecture](#)
- [The Canonical Data Model \(CDM\): The Foundation for API Standardization](#)
- [Canonical Model Creation Workflow](#)
- [Principles of Cloud-First API Design](#)

Reference Architectures (The "Playbooks")

- [Reference Architectures \(The "Playbooks"\)](#)
- [Proposed Sell Reference Architecture: : A Strategic Interface Layer Architecture](#)
- [Proposed "Buy" Reference Architecture for Partner Integration at Lumen: The PIL Model](#)

Lumen API Style Guide

- [Lumen API Style Guide](#)
- [API Standard: Rest & Resource Design](#)
- [API Versioning Strategy](#)
- [API Standard: POST Method](#)
- [API Standard: JSON Payload](#)
- [API Standard: GET Method](#)
- [API Standard: Batch Operations](#)
- [API Standard: PATCH Method](#)
- [API Standard: Caching & Concurrency with ETag](#)
- [API Standard: Long-Runing Operations \(LRO\)](#)
- [API Standard: Bulk Data Transfer \(Handling Massive Payloads\)](#)

- [API Standard: HTTP Headers](#)
- [API Standard: Delete Method](#)
- [API Style Guide Governance Tracker](#)
- [API Standard: Rate Limits & Error Handling Standard](#)
- [OAS Specification Best Practices](#)
- [API URI Standard Proposal](#)
- [API Pagination Standards](#)
- [API Standard: Lumen Problem Details — Minimal Profile \(LPDP-Mini v1.0\)](#)
- [API Standard: Idempotency](#)
- [API Standard: Partial Resource Retrieval](#)
- [API Standard: Standardized Data Types and Formats](#)
- [API Standard: Date & Time Naming](#)
- [API Standard: PUT Method](#)

API Security Standards

- [API Security Standards](#)
- [API Security Standard: Identity Derivation, Multi-Tenancy & Zero Trust](#)

API Governance Process (Integrated with SDLC)

- [API Governance Process \(Integrated with SDLC\)](#)
 - [Enterprise API Governance & Publishing Architecture](#)
 - [API Governance: Operating Model](#)
 - [API SDLC Stage 0: Discovery & Strategy](#)
 - [API SDLC Stage 1: Planning & Design](#)
 - [API SDLC Stage 2: Implementation & Testing](#)
 - [API SDLC Stage 3: Deployment & Release](#)
 - [API SDLC Stage 4: Publication & Enablement](#)
 - [API Governance: Operations & Continuous Improvement](#)
 - [API Governance: Reporting & Exception Management](#)
 - [API Governance: Repository Onboarding - Bootstrap Structure and Check-In Paths](#)
-

API Governance: Approved Artifacts

API Governance: Approved Artifacts

API Governance Metadata Model

1. Introduction

This document defines the metadata model for API governance. Its purpose is to capture the essential "delta" information—such as ownership, business context, and governance status—that is not already present in the OpenAPI Specification (OAS) file.

This model is a two-part system designed to work with an API's source of truth (the OAS) to enable discovery, automated governance, and lifecycle management.

- **API-Level Metadata (`api-metadata.yaml`):** A separate metadata file that lives alongside the OAS file in the API's repository. This file describes the API as a whole.
- **Endpoint-Level Metadata (OAS `x-` tags):** Custom extension properties placed inside the OAS file on specific operations to enable fine-grained, automated policy enforcement.

This structure makes the metadata discoverable by a cataloging tool (like Backstage) while keeping the machine-readable, automated governance rules co-located with the endpoint contracts they apply to.

2. API-Level Metadata

This metadata must be stored in a dedicated YAML file (`api-metadata.yaml`) in the API's root directory.

- **`schemaVersion` (String, Required):** The version of this metadata schema, which enables CI validation and controlled evolution.
Value: `v1.0`
-

2.1. Ownership & Team

Identifies who is responsible for the API's development, maintenance, and business alignment. This is critical for accountability and for consumers to know who to contact.

- **`apiName` (String, Required):** The human-readable, canonical name of the API (e.g., "User Profile API", "Payment Orchestration Service").
- **`ownershipModel` (String, Required):** For the current governance scope, this value is always Lumen-Owned. This ensures a consistent ownership declaration and establishes a foundation for future expansion (e.g., Externally-Owned, Joint-Owned) when the internal API repository is introduced.
- **`assetId` (String, Required):** Lumen Asset ID (SYSGEN) that maps the API to its registered owner application in the Asset Management system (e.g., SYSGEN-45123). Used for authority, audit, and MAL PoC linkage.
- **`businessOwner` (String, Required):** The primary stakeholder or Product Manager responsible for the API's business function and roadmap (e.g., an email, user ID, or

group alias).

- **technicalOwner (String, Required):** The Engineering Lead or Architect responsible for the API's technical design, implementation, and operational health.
 - **developmentTeam (String, Required):** The name or alias of the team that builds and maintains the API (e.g., "identity-platform-team").
 - **supportContact (String, Required):** The official support channel for consumers (e.g., a group email `api-support@example.com`, a Slack channel `#slack-team-identity`).
-

2.2. Business Context

Provides the "why" and "where" for the API within the larger enterprise architecture.

- **consumerAudience (String, Required):** The intended consumer channel, which dictates security and exposure.
Values: `internal-ui` (for internal UIs/frontends), `internal-service` (for other internal systems/automation), `external-partner`, `external-public`
 - **apiLayer (String, Required):** The architectural classification of the API, based on its primary role.
Values: `Experience`, `Canonical`, `Process`, `System`
 - **system (String, Optional):** A key/name that links this API to a larger "System" or "Domain" in the software catalog (e.g., `billing-platform`, `crm`, `customer-identity-domain`).
-

2.3. Governance & Lifecycle

Tracks the API's maturity, provenance, and the level of governance applied.

- **lifecycleStage (String, Required):** The current stage of the API's life.
Values: `Design`, `In-Development`, `Active`, `Deprecated`, `Retired`
- **apiOrigin (String, Required):** Declares whether the API is a new (greenfield) contract created under the governance program, or an existing API being onboarded. This field is required because governance policy and allowed profiles differ for greenfield vs existing APIs (especially for public exposure).
Values: `greenfield`, `existing`
- **governanceLevel (String, Required):** The level of review and adherence to standards applied to this API. This informs consumer trust and risk.
Values: `Fully Governed`, `Partially Governed`, `Lift and Shift`, `Exception`
Note: This field is pipeline-owned and is computed on successful pipeline execution.
- **governanceProfile (String, Required):** The specific ruleset or "linter profile" that automation should apply to this API. This allows teams to select the strictness of review based on policy and API origin.
Values: `basic` (WARN-only profile, no blocking rules), `full` (strict adherence to standards with blocking rules)

- **deprecationDate (String, Optional):** (If lifecycleStage is "Deprecated") The date (ISO 8601) when the API will no longer be supported.
 - **sunsetDate (String, Optional):** (If lifecycleStage is "Deprecated") The date (ISO 8601) when the API will be shut down.
-

2.4. Technical & Design Details

Captures high-level technical attributes not defined in the OAS.

- **apiStyle (String, Required):** The architectural style of the API.
Values: REST, RPC-style, GraphQL, gRPC, Async
 - **protocol (String, Required):** The primary protocol used.
Values: HTTPS, WSS (WebSockets), AMQP (Messaging), etc.
 - **swaggerHubName (String, Optional):** The unique, URL-friendly identifier (slug) for this API in SwaggerHub. This is used by CI/CD pipelines to sync the repository OAS with the SwaggerHub catalog.
Example: customer-order-api (Must match the SwaggerHub URL, not the display name).
-

2.5. Security & Compliance

Defines the overall risk and compliance posture for the API.

- **dataClassification (String, Required):** The highest sensitivity level of data handled by any endpoint in the API. This acts as a high-water mark for compliance.
Examples: Public, Internal, Confidential, Restricted (PII, PCI, PHI)
 - **complianceRequirements (List, Optional):** A list of any compliance regulations that apply to the data handled by this API.
Examples: ["GDPR", "HIPAA", "PCI-DSS"]
-

2.6. Review Audit Trail (Updated)

An immutable log of the **latest terminal governance review**.

This section is written automatically by the governance pipeline or tooling **only after a successful review**.

reviewAuditTrail (Object, Optional)

A container for all review-related audit metadata. This object represents a **terminal record** and must not be modified manually.

Fields

- **reviewId (String, Required)**
A unique identifier for the latest governance review.
- **reviewDate (String, Required)**
The timestamp (ISO 8601) when the review was completed.
 - For an approved review, this is the date the specification was approved.
 - For an exception, this is the date the exception was granted.
- **reviewStatus (String, Required)**
The terminal outcome of the review.
Allowed values:
 - Approved
 - Exception
- **reviewers (Array of Objects, Required)**
The list of reviewers who approved the specification or exception.

Each reviewer entry includes:

- **name (String, Required):** Name of the reviewer or approving body
 - **role (String, Optional):** Reviewer role (e.g., Product Owner, API Steering Committee)
 - **securityReviewId (String, Optional)**
A unique identifier linking this governance review to an internal risk, compliance, or security review record.
-

Notes and Constraints

- This section is written **only after the pipeline reaches a terminal state**.
 - Intermediate states such as *Pending*, *Rejected*, or *Approved with Conditions* are **not recorded** here.
 - If the pipeline fails, **no** reviewAuditTrail **is written**.
 - Any subsequent successful review **replaces the previous audit trail**.
-

Example

```
reviewAuditTrail:
  reviewId: GOV-2026-00192
  reviewDate: 2026-01-05T14:22:31Z
  reviewStatus: Exception
  reviewers:
    - name: Jane Doe
      role: Product Owner
```

This revision makes the audit trail **explicitly terminal, auditable, and aligned** with your stateless pipeline and exception model, without introducing new concepts or metadata.

2.7. Risk & Routing (Consultative Governance)

Defines the risk profile and determines the required reviewers for Pull Requests. This model supports Risk-Based Routing with Voluntary Up-Leveling.

- **riskBasedRouting (Object, Required):**
 - **level (String, Required):** The declared risk level of the API. This sets the Minimum Reviewer Requirement (The Floor) based on the standard governance table.
Values:
 - Low (Internal, simple CRUD → Auto-Approve possible)
 - Medium (Partner-facing, standard patterns → Architect Review)
 - High (Public, PII, complex workflows → Architect + Security (optional) + Domain(optional))
 - Critical (Core Banking/Payment → Board Review)
 - **justification (String, Required):** A brief explanation for the chosen risk level (e.g., "Public API handling PII data").
 - **additionalReviewers (List, Optional):** A list of extra reviewer groups to make mandatory for this API.
Function: Additive (The Ceiling).
Use Case: Voluntarily request a Domain Architect for complex logic, or enforce a specific Security Lead review for sensitive projects.
-

2.8. Schema Validation

To enable automated CI validation, editor autocompletion, and ensure conformity, all `api-metadata.yaml` files must validate against the official JSON Schema.

Schema Location: `catalog-schema.json`

3. Endpoint-Level Metadata (Inside the OAS File)

For fine-grained, automated governance (e.g., security policies, linting, gateway configuration), we must apply metadata at the Operation (Endpoint) level. This metadata

must be stored inside the OAS file using OpenAPI Specification Extensions (fields prefixed with x-).

3.1. How to Store Endpoint Metadata

This metadata lives directly on the operation object within the OAS file, making the spec a self-contained source of truth for automation.

Example within the OAS (api.oas.yaml):

```
paths:
  /users:
    post:
      summary: Create a new user
      operationId: createUser
      # --- Automated Governance Metadata Starts Here ---
      x-data-classification: "Restricted (PII)"
      x-operation-type: "lro" # (Long-Running Operation)
      x-financial-operation: false
      x-idempotent: false
      # --- Automated Governance Metadata Ends Here ---
      tags:
        - Users
      requestBody:
        description: User object to be created
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'
      responses:
        '202':
          description: Accepted. The user creation is in progress.
        '400':
          description: Invalid input
```

3.2. Key Endpoint-Level Metadata Fields

- **x-data-classification (String, Required):** The sensitivity of the data handled by this specific endpoint. This is the most critical field for automated security and can be more granular than the API-level high-water mark.
Examples: Restricted (PII), Confidential, Internal, Public
- **x-operation-type (String, Optional):** Defines the behavior of the endpoint, allowing the API gateway to apply different policies (e.g., timeouts, retry logic).
Examples: lro (Long-Running Operation), bulk-data, standard-sync
- **x-financial-operation (Boolean, Optional):** Flags if this operation initiates a financial transaction, which can trigger stricter audit logging policies. Defaults to false.
- **x-idempotent (Boolean, Optional):** Explicitly declares if a POST or PATCH operation is safe to retry. This is crucial for resilient consumer design.

3.3. Addressing the "Messy Presentation" Concern

A common concern is that x- tags will clutter the rendered documentation (e.g., Swagger UI). This is a solvable presentation problem.

- **Priority:** The primary consumer of x- tags is automation (linters, security scanners, API gateways). It is essential for these tags to be machine-readable.
 - **Solution:** Documentation tools should interpret these tags rather than just displaying them. Your developer portal can be customized to:
 - Hide governance tags from the default view.
 - Render them as badges: x-data-classification: "Restricted (PII)" → red "PII Data" badge.
 - Render them as icons: x-operation-type: "lro" → hourglass icon.
-

4.0 API-Metadata JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "API Governance Catalog Metadata",
  "description": "Schema for the api-metadata.yaml file, which provides governance and d",
  "type": "object",
  "properties": {
    "schemaVersion": {
      "description": "The version of this metadata schema.",
      "type": "string",
      "enum": ["v1.0"]
    },
    "apiName": {
      "description": "The human-readable, canonical name of the API.",
      "type": "string",
      "minLength": 1
    },
    "ownershipModel": {
      "description": "Defines the ownership classification of the API. For governance sc",
      "type": "string",
      "enum": ["Lumen-Owned"]
    },
    "assetId": {
      "type": "string",
      "minLength": 1,
      "description": "Lumen Asset ID (SYSGEN) linking to Asset Management",
      "examples": ["SYSGEN-45123"]
    },
    "businessOwner": {
      "description": "The primary stakeholder or Product Manager (email, ID, or alias).",
      "type": "string",

```

```
    "minLength": 1
  },
  "technicalOwner": {
    "description": "The Engineering Lead or Architect (email, ID, or alias).",
    "type": "string",
    "minLength": 1
  },
  "developmentTeam": {
    "description": "The name or alias of the team that builds and maintains the API.",
    "type": "string",
    "minLength": 1
  },
  "supportContact": {
    "description": "The official support channel for consumers (email or Slack channel)",
    "type": "string",
    "minLength": 1
  },
  "consumerAudience": {
    "description": "The intended consumer channel, which dictates security and exposure",
    "type": "string",
    "enum": [
      "internal-ui",
      "internal-service",
      "external-partner",
      "external-public"
    ]
  },
  "apiLayer": {
    "description": "The architectural classification of the API, based on its primary",
    "type": "string",
    "enum": [
      "Experience",
      "Canonical",
      "Process",
      "System"
    ]
  },
  "system": {
    "description": "A key/name that links this API to a larger System or Domain in the",
    "type": "string"
  },
  "lifecycleStage": {
    "description": "The current stage of the API's life.",
    "type": "string",
    "enum": [
      "Design",
      "In-Development",
      "Active",
      "Deprecated",
      "Retired"
    ]
  },
  "apiOrigin": {
    "description": "Declares whether the API is greenfield (created under the governan
```

```
    "type": "string",
    "enum": [
      "greenfield",
      "existing"
    ]
  },
  "governanceProfile": {
    "description": "The specific ruleset or 'linter profile' that automation should ap",
    "type": "string",
    "enum": [
      "basic",
      "full"
    ]
  },
  "governanceLevel": {
    "description": "The level of review and adherence to standards applied to this API",
    "type": "string",
    "enum": [
      "Fully Governed",
      "Partially Governed",
      "Lift and Shift",
      "Exception"
    ]
  },
  "deprecationDate": {
    "description": "The date (ISO 8601) when the API will no longer be supported.",
    "type": ["string", "null"],
    "format": "date-time"
  },
  "sunsetDate": {
    "description": "The date (ISO 8601) when the API will be shut down.",
    "type": ["string", "null"],
    "format": "date-time"
  },
  "swaggerHubName": {
    "description": "The unique, URL-friendly identifier (slug) for this API in Swagger",
    "type": "string"
  },
  "apiStyle": {
    "description": "The architectural style of the API.",
    "type": "string",
    "enum": [
      "REST",
      "RPC-style",
      "GraphQL",
      "gRPC",
      "Async"
    ]
  },
  "protocol": {
    "description": "The primary protocol used.",
    "type": "string",
    "enum": [
      "HTTPS",
```

```

        "WSS",
        "AMQP",
        "MQTT"
    ]
},
"dataClassification": {
    "description": "The highest sensitivity level of data handled by any endpoint in t",
    "type": "string",
    "enum": [
        "Public",
        "Internal",
        "Confidential",
        "Restricted (PII, PCI, PHI)"
    ]
},
"complianceRequirements": {
    "description": "A list of any compliance regulations that apply to this API.",
    "type": "array",
    "items": {
        "type": "string"
    },
    "uniqueItems": true
},
"riskBasedRouting": {
    "type": "object",
    "description": "Defines the risk profile and reviewer routing (Consultative Govern",
    "properties": {
        "level": {
            "type": "string",
            "enum": ["Low", "Medium", "High", "Critical"],
            "description": "The declared risk level of the API. Sets the minimum required",
        },
        "justification": {
            "type": "string",
            "minLength": 10,
            "description": "Explanation for the chosen risk level."
        },
        "additionalReviewers": {
            "type": "array",
            "description": "Optional list of EXTRA reviewer groups to make mandatory (Ceil",
            "items": { "type": "string" },
            "uniqueItems": true
        }
    },
    "required": ["level", "justification"]
},
"reviewAuditTrail": {
    "description": "An immutable log of the latest governance review.",
    "type": "object",
    "properties": {
        "reviewId": {
            "description": "A unique identifier for the latest governance review.",
            "type": "string",
            "minLength": 1
        }
    }
}

```



```

    },
    "reviewDate": {
      "description": "The timestamp (ISO 8601) when the latest review was completed.",
      "type": "string",
      "format": "date-time"
    },
    "reviewStatus": {
      "description": "The outcome of the review.",
      "type": "string",
      "enum": [
        "Pending",
        "Approved",
        "Approved with Conditions",
        "Rejected"
      ]
    },
    "securityReviewId": {
      "description": "A unique identifier that links to an internal risk or security",
      "type": "string"
    }
  ],
  "required": [
    "reviewId",
    "reviewDate",
    "reviewStatus"
  ],
  "additionalProperties": false
}
},
"required": [
  "schemaVersion",
  "apiName",
  "ownershipModel",
  "assetId",
  "businessOwner",
  "technicalOwner",
  "developmentTeam",
  "supportContact",
  "consumerAudience",
  "apiLayer",
  "lifecycleStage",
  "apiOrigin",
  "governanceProfile",
  "governanceLevel",
  "apiStyle",
  "protocol",
  "dataClassification",
  "riskBasedRouting"
],
"additionalProperties": false
}

```

Mastering API Governance: The New Automated Prompt for OAS Review

The integrity of our API ecosystem relies on strict adherence to the **Lumen API Style Guide**. Manual reviews—slow, inconsistent, and prone to interpretation—cannot keep pace with modern development. As part of our governance modernization, we are adopting a fully **automated, deterministic OAS review standard**.

At the core of this system is a **single, authoritative review prompt** that all automated tools must follow. This ensures **zero-tolerance compliance**, complete traceability, and a consistent developer experience.

Below is the **official, copy-paste-ready prompt** that all tooling must use.

OAS Governance Review Prompt

Please replace the file name in the below context.

```
Review the file [INSERT_OAS_FILENAME] exhaustively and systematically against the OAS 3.
```

```
=====
```

```
1. SCOPE OF REVIEW – YOU MUST CHECK ALL OF THE FOLLOWING
```

```
=====
```

```
You MUST perform a complete review of the OAS, covering:
```

- Global metadata
- EVERY path, method, and parameter
- EVERY request body and response body
- EVERY response code and header
- EVERY schema property and reusable component
- ALL examples, \$refs, and security definitions

```
If the file is too large to review in one response, specify exactly where you stopped and wait for confirmation before continuing.
```

```
=====
```

```
2. REFERENCE MATERIAL ENFORCEMENT (MANDATORY)
```

```
=====
```

```
You MUST use the **OAS 3.0.3 Specification**, **Lumen API Style Guide**, and **Canonical
```

1. ****Rule Lookup:**** For every element in the OAS, cross-reference it against:
 - The official ****OpenAPI Specification (version 3.0.3)**** for structural validity.
 - The specific rules found in the ****Lumen API Style Guide**** (Naming, Error formats
2. ****Canonical Types:**** Verify that the OAS reuses the canonical schemas defined in the
3. ****Precedence:**** If the OAS contradicts the Style Guide, it is a violation. If it con

3. OUTPUT FORMAT – MANDATORY STRUCTURED JSON

For EVERY violation found, output a JSON object in this exact format:

```
{
  "severity": "CRITICAL|HIGH|MEDIUM|LOW",
  "location": "Path: /endpoint, Method: GET, Area: specific.field.path",
  "jsonpath": "$.paths['/endpoint'].get.parameters[0].schema",
  "issue": "Clear explanation of the exact issue",
  "violated_rule": "Quote the specific text from the Style Guide or OAS 3.0.3 spec that",
  "recommendation": "Specific, actionable fix"
}
```

4. SEVERITY ASSIGNMENT LOGIC

Assign severity based on the strictness defined in the Lumen API Style Guide and OAS 3.0

- * **CRITICAL:** Violations of **OAS 3.0.3 structural requirements** (invalid syntax), vi
- * **HIGH:** Violations of "SHOULD" rules found in the Lumen API Style Guide, major consi
- * **MEDIUM:** Violations of "MAY" rules, missing descriptions, or minor naming inconsist
- * **LOW:** Documentation polish or optional improvements.

5. CROSS-CUTTING VALIDATION

In addition to the specific rules in the Lumen API Style Guide, validate:

- **Strict adherence to OpenAPI 3.0.3 structure** (e.g., correct placement of servers, c
- No unused schemas (DRY violation)
- No circular \$refs
- Logical consistency (e.g., GET requests should not have request bodies)

6. FINAL SUMMARY

After listing all JSON findings, provide:

- Total violations by severity
- Top 3-5 critical issues found in the OAS

Why This Prompt Matters

If you plan to use GitHub Copilot Space for review, please use the above prompt. It incorporates not just OAS validation rules, but also **canonical type enforcement through**

repository lookup, ensuring that every API adheres to the standards defined in the **Lumen API Style Guide** and the shared schema library.

This means the reviewer:

Ensures Consistency Across All APIs

The automated reviewer follows a deterministic, step-by-step evaluation.

There is **zero room for interpretation, skipping, or subjective judgment**.

Every path, parameter, schema, example, and reusable type is reviewed the same way every time.

Enforces All Lumen Hard Standards

This prompt strictly enforces every mandatory governance rule, including:

- **LPDP-Mini** (RFC 9457–aligned error structure)
- **Canonical data types** detected through repository lookup
- **Opaque, server-generated IDs**
- **lower_snake_case** for all fields and parameters
- **Pagination standards** (limit, offset, and canonical metadata)
- **Security for all mutations**
- **Correct HTTP semantics**
- **PATCH + If-Match concurrency control**
- **Reusable schema enforcement** (no redefinition of standard types)

If a developer reinvents a type (e.g., custom address, custom ID format), the prompt catches it instantly.

Produces Machine-Readable, Developer-Actionable Output

Every issue is reported in a strict JSON format that downstream systems can consume automatically.

Each finding includes:

- **Severity (CRITICAL → LOW)**
- **Human-readable location**
- **JSONPath** for precise navigation
- **Exact description of the issue**
- **A specific, actionable recommendation**

This transforms feedback from “review comments” into **direct, fix-ready instructions**.

Leverages the Style Guide Repository (GitHub Copilot Integration)

The prompt requires the reviewer to:

- **Search the repository for canonical schemas**
- **Compare API schemas against standard definitions**
- **Detect deviations from shared types**
- **Enforce reuse via `$ref` instead of reinvention**

This creates a **single semantic contract** across all APIs.

Final Outcome

By standardizing this prompt, Lumen now has:

- ◆ **A reproducible, auditable governance process**

Every review is performed identically, with full traceability.

- ◆ **A universal validator for all API teams**

Every API is evaluated against the same standards and canonical definitions.

- ◆ **A deterministic enforcement mechanism for the Lumen API Style Guide**

Repairs ambiguity, eliminates drift, and ensures strong contract discipline.

- ◆ **A massive reduction in manual review effort**

Days of manual checks shrink to seconds, with higher accuracy.

- ◆ **Guaranteed adherence to canonical schemas across the portfolio**

No more duplicated addresses, IDs, timestamps, error formats, pagination blocks, etc.

Lumen API Governance — Comprehensive Review Prompt

Standardized LLM-Assisted Review Framework for Confluence API Designs

Overview

As part of Lumen's move toward **automated, consistent, and standards-driven API governance**, the Architecture Office is introducing the **Comprehensive Governance Review Prompt**.

This prompt is designed to provide a **repeatable review workflow** when architects upload a **Confluence API analysis page**.

It ensures that every API proposal is evaluated using the **entire Lumen API Governance Framework**, including:

- Canonical Data Model alignment
- URI & resource taxonomy rules
- REST method semantics
- Identifier & data-type standards
- JSON payload conventions
- LPDP-Mini (Problem Details)
- Pagination & partial retrieval
- Idempotency & concurrency
- LRO/async workflow
- Batch vs bulk
- Security & identity
- Multi-tenancy
- Versioning rules
- Consistency & reuse
- Publishing workflow
- Governance metadata

When Architects Should Use This Prompt

Use this prompt when:

- **Performing peer review inside domain teams**
- **Preparing materials for White Glove Review**
- **Validating vendor API specifications before acceptance**
- **Ensuring multi-team alignment (Rubicon, BerkeleyNID, AO, MCGW, etc.)**

How to Use

1. Upload your **Confluence page export** (PDF, HTML, DOCX, or pasted text).
2. Copy/paste the full prompt below.
3. The LLM will run a **structured, section-by-section review** aligned with Lumen standards.

Comprehensive Governance Review Prompt

```
# LUMEN API GOVERNANCE REVIEW – FILE-BASED MASTER PROMPT
# (Use this when uploading a Confluence page file)

You are an API Governance Reviewer operating under all Lumen API Standards.
I have uploaded a file that contains the full API design analysis from Confluence.

Use ONLY the uploaded file + the Lumen standards indexed in this workspace.

Do not use external assumptions and do not infer details not present in the file

---

# INPUT
Uploaded File: The file I attached contains the entire Confluence analysis.
Treat it as the authoritative source for all design, URLs, schemas, and decisions.

No other inputs will be provided.

---

# YOUR TASK
Perform a complete Lumen API Governance Review following all required standards.

Your output MUST follow the structured format below.

---

# 1. Extract Key API Concepts

## 1A. Resource Inventory
From the uploaded file, list all resources referenced or implied:
- Resource name
- Resource type (CRUD / workflow / configuration / operational / composite)
- Canonical or non-canonical
- Notes

## 1B. Verb / Action Audit
Identify all verbs and operational intents:
- Detected verb/action
```

- Business intent
- Modality (Command / Query / Notification)
- Correct REST representation
- RPC risk

1C. Modality Summary

Summaries of:

- Commands
- Queries
- Notifications

Highlight modality mismatches.

2. Full Governance Review (Apply All Lumen Standards)

Apply the following standards fully and strictly:

2A. Canonical & Domain Modeling

- Alignment to canonical model
- Use of canonical fragments
- Canonical field names
- Identifier correctness: opaque, fixed-length, stable, URL-safe, ``<resource>_id``
- Domain/capability mapping

2B. URI & Resource Design

- Pure noun-based URIs
- No verbs in paths
- Proper resource hierarchy
- Correct pluralization
- No RPC or action endpoints
- Canonical taxonomy

2C. HTTP Method Semantics

- GET is safe/idempotent
- POST only for create or commands
- PUT = full replace
- PATCH = partial update
- DELETE = **no request or response body**

Identify any misuse in the uploaded file.

2D. Payload Standards

- JSON only (unless exception noted)
- lower_snake_case names
- Standard types (UUIDv4, ISO 8601, money type, etc.)
- Required vs optional semantics
- No system IDs exposed

2E. Error Handling – LPDP-Mini

- Correct Problem Details structure
- String-based error codes
- Correct status mapping
- No vendor-specific envelopes

2F. Pagination & Partial Retrieval

- Approved pagination pattern
- Partial retrieval / field selectors if present
- No custom pagination fields unless approved

2G. Idempotency, Caching, Concurrency, Async (LRO)

- Idempotency keys for POST
- ETag/If-Match for updates
- Async uses 202 + Location header
- Operation resource defined for LRO
- Batch vs bulk distinction correct

2H. Security, Identity, Multi-Tenancy

- Authentication scheme alignment
- Identity derivation
- Tenant isolation
- No sensitive data leaks
- Correct header usage

2I. Versioning

- Correct version placement (URI or header, per standard)
- No breaking changes in same version
- Deprecation rules

2J. Consistency & Reuse

- Reuse of canonical schemas, fragments, patterns
- No snowflake or one-off structures
- Naming consistency

3. Findings & Remediation

3A. Findings Table

For each issue found in the uploaded file:

- Severity (Critical / Major / Minor)
- Location (quote relevant section)
- Standard violated (cite standard name)
- Why it violates
- Exact recommended fix

3B. Summary Judgment

Choose one:

- ****Pass****
- ****Conditional Approval****
- ****Reject / Blocker****

3C. Remediation Examples

Provide corrected:

- URLs
- Resource names
- Payload fields
- Problem Details examples

```
- Async workflow structure
- Pagination samples
- Idempotency usage
- Identifier formats

---

# OUTPUT RULES
- Follow the structure EXACTLY as above.
- Base the review ONLY on the uploaded file.
- Do NOT hallucinate content not inside the file.
- Do NOT fill sections with generic advice – everything must reference file content.
- Cite each issue precisely and make fixes concrete.

# END OF PROMPT
```

Why This Matters

- Ensures Architectural Consistency Across All Domains

Removes ambiguity and subjective interpretation of standards.

- Reduces Governance Review Time

Architects pre-clean designs before submitting, reducing back-and-forth.

- Enforces Canonical Models & Style Guide Rules Automatically

LLMs act as deterministic validators using your standards.

- Produces Actionable, Machine-Readable Findings

Output can be attached to Jira, PRs, and governance logs.

- Serves as The Standard Input for White Glove API Reviews

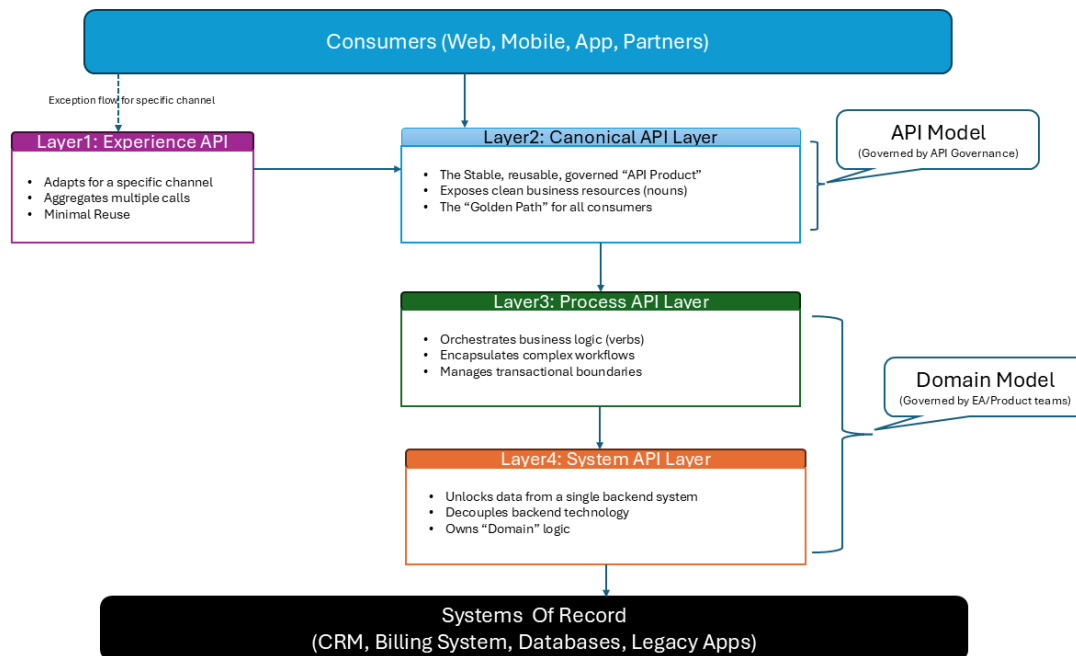
Teams arrive prepared with structured analysis already completed.

=====

Architectural Principles & Patterns

Architectural Principles & Patterns

A Strategic Four-Layer Approach to API Architecture



A Layered Approach to API Design

In a large-scale enterprise like Lumen, a well-defined API architecture is the foundation for achieving digital agility, scalability, and reusability. While the traditional three-layer model (Experience, Process, System) provides a good starting point, a more mature **four-layer architecture** offers greater clarity by explicitly separating the stable, public-facing API contract from consumer-specific adaptations.

This model serves three distinct but related functions: as a classification system, a design pattern, and most critically, as an architectural pattern.

The Three Facets of the Four-Layer Model

It's important to understand that this model provides value in three distinct ways.

1. **As a Classification System:** At its simplest, the model is a taxonomy. It gives us a shared vocabulary to categorize our APIs based on their function (e.g., "Is this a Process API or a System API?"). This helps us inventory our assets and understand their intended purpose at a glance.
2. **As a Design Pattern:** The model provides guidance for separating concerns within a single application or service. It encourages developers to write cleaner, more maintainable code by isolating orchestration logic from backend integration logic.
3. **As an Architectural Pattern:** This is its most strategic role. The model defines the **non-negotiable rules of interaction between the layers**. It mandates, for example, that an Experience API must call the Canonical Layer and is forbidden from calling a

System Layer directly. This is the rule that enforces enterprise-wide decoupling, security, and governance.

The Four Layers Explained

This refined model consists of four logical layers: Experience, Canonical, Process, and System.

1. The Experience API Layer (The Exception)

This is a thin, non-reusable adaptation layer whose sole purpose is to handle the unique needs of a specific consumer that cannot be efficiently served by the standard canonical API. It is an **exception, not the norm**.

- **The "Why":** To optimize for a specific channel's user experience (e.g., a mobile app, a partner portal) or to handle the technical vagaries of a particular consumer.
- **Advantages:**
- **Flexibility & Usability:** Tailored to specific user experiences, making them easier for front-end developers to consume.
- **Consumer Agility:** Allows teams to rapidly build or change APIs to meet new channel needs without impacting the core API products.
- **Performance Optimization:** Can aggregate multiple calls to the Canonical Layer into a single, efficient response for "chatty" clients.
- **What to Watch For (Architectural Concerns):**
- **Minimal Reuse:** These APIs are purpose-built and should not be reused, leading to potential maintenance overhead if not governed properly.
- **No Business Logic:** This layer must never contain core business rules. It only transforms, aggregates, and can obfuscate data for presentation.
- **User-Centric Design:** Low latency is crucial. Performance must be monitored and optimized for speed and responsiveness.
- **Scalability & Deployment:** Must be designed for elastic scalability to handle fluctuating user traffic. Regional deployment closer to end-users may be required to reduce latency.
- **Edge Security:** As the public-facing entry point, this layer requires robust security measures like DDoS protection, Web Application Firewalls (WAF), and client certificate management.
- **Governance:** The creation of an Experience API must be a deliberate, justified decision to prevent API sprawl.

2. The Canonical API Layer (The Norm)

This is the **default, reusable, and governed API "product"** for the enterprise. It is the stable, well-documented storefront that should be used by all internal and external channels whenever possible.

- **The "Why":** To provide a single, consistent, and stable contract for all consumers, abstracting away the complexity of the internal systems. This is the API that realizes our strategic business goals.
- **Advantages:**
- **High Reusability:** Designed to be the "golden path" used by 90% of consumers.
- **Stability & Decoupling:** Provides a long-term, stable contract. The internal implementation can change, but this public-facing contract remains the same.
- **Clear Ownership:** Managed as a formal product with a clear roadmap and governance.
- **What to Watch For:**
- **Purity:** This layer must be protected from being polluted with channel-specific logic or internal implementation details.
- **Data-Centric:** Focuses on exposing clean, consistent representations of business resources (the "nouns" like Order, Service, Customer).

3. The Process API Layer (The Engine)

This internal-facing layer contains the business logic that orchestrates multiple systems to complete a business process. It is the engine that powers the layers above it.

- **The "Why":** To encapsulate and reuse complex business processes and workflows, ensuring that business rules are applied consistently regardless of which channel initiated the request.
- **Advantages:**
- **Consistency:** Standardizes business processes across the organization.
- **Agility:** The business process can be changed or optimized in one place, and that improvement is instantly available to all consuming APIs.
- **Business Logic Encapsulation:** Centralizes complex orchestration, preventing business rules from being duplicated.
- **What to Watch For (Architectural Concerns):**
- **Owns Process Logic:** This is the correct place for logic that spans multiple systems or domains (e.g., "update billing, then update provisioning").
- **Transactional Boundaries:** This is where business transactions often begin and end, including logic for rollbacks, error handling, and concurrency management.
- **Dependency & Version Management:** Requires careful management of dependent APIs and their versions. A visual dependency map is critical for documentation.

- **Observability:** Requires robust monitoring for performance (response time, throughput), reliability (uptime), and business process analytics (workflow states, processing time). Alerts on dependent API performance are crucial.
- **Resilience:** The design must include strategies for graceful degradation, defining fallback behaviors or compensation steps if a dependent API fails.
- **Auditing:** Must ensure detailed logging of critical events per user for auditing and debugging purposes.

4. The System API Layer (The Foundation)

These are the foundational APIs that provide direct, unlocked access to core systems of record (e.g., a legacy CRM, a billing system, a database).

- **The "Why":** To expose the data and capabilities of a specific backend system in a clean, reusable way, while insulating the rest of the architecture from the specifics of that system's technology.
- **Advantages:**
 - **Reusability & Stability:** Highly reusable across different processes and provides a stable foundation for other APIs to build upon.
 - **Backend Decoupling:** If a legacy system is replaced, only its System API needs to be updated.
 - **Access Control:** Provides a secure access point to core data.
- **What to Watch For (Architectural Concerns):**
 - **Owns Domain Logic:** This layer **must** contain and enforce the business logic specific to the entity it manages (e.g., a CRM-System-API must enforce the rules for a valid "Customer").
 - **No Process Logic:** This layer should **never** contain orchestration logic that calls other systems. A System API should not know about the existence of other systems.
 - **Transactional Integrity & Idempotency**
The System API layer is responsible for enforcing **transactional correctness and idempotency** for all state-modifying operations. Because this layer owns the system of record and its domain semantics, it is the only layer that can reliably determine whether an operation has already been applied and ensure that retries do not result in duplicate side effects.
System APIs must support idempotent behavior (e.g., via idempotency keys or natural business identifiers) and return consistent results for repeated requests. Higher layers may propagate or coordinate idempotency keys but must not redefine idempotency semantics.
 - **Robustness:** Redundancy and high availability are critical for these foundational APIs.
 - **Security:** Requires strong access controls (e.g., MFA for state-modifying APIs) and may reside in a highly restricted network segment.

- **Data Governance:** Must ensure data integrity, and enforce data retention and encryption policies.

Governance and Ownership: API as a Product

A successful API strategy requires a clear ownership model where APIs are treated as products. While governance is a shared responsibility, the roles for each group are distinct and complementary.

Role	Responsibility
Product Teams	Act as the "API Product Owners." They own the end-to-end lifecycle of the APIs within their business domain, across all four layers. This includes defining the roadmap for the Canonical API (Layer 2), prioritizing features, and ensuring the underlying Process and System APIs (Layers 3 & 4) meet the business needs.
API Governance	Act as the "Portfolio Managers" or "City Planners." They do not own individual APIs but own the overall API landscape. They define the enterprise-wide standards (the "building codes"), manage the API catalog, and ensure consistency and quality across all API products.
Enterprise Architecture (EA)	Act as the "Chief Architects" or "Building Inspectors." They own the strategic patterns, like this four-layer model, and ensure that the technical implementations are sound, scalable, and align with the broader technology vision of the company.

Why These Should Be Separate APIs, Not a Single Monolith

The core reason for separating these layers is to prevent the creation of a **monolithic API** that becomes brittle, difficult to maintain, and a bottleneck for business innovation. A single API that attempts to do all three would suffer from several critical flaws:

a. Tight Coupling and Lack of Reusability

- **Single API Problem:** If a single API handles a request from a mobile app, calls a legacy system, and orchestrates a process, it is tightly coupled to all three. If the mobile app's data needs change, or the legacy system is replaced, or the business process is altered, you have to modify and re-deploy the entire monolithic API.
- **Layered Solution:** In the layered model, an **Experience API** for a new web portal can simply call the same **Process API** that the mobile app uses. This promotes code reuse and prevents redundant development. If a legacy billing system is replaced, only the

corresponding **System API** needs to be updated. The higher-level Process and Experience APIs remain unaffected, as their contracts with the System API remain stable.

b. Inefficient Scalability and Performance

- **Single API Problem:** A monolithic API becomes a single point of failure and a performance bottleneck. A request for a simple data retrieval from the mobile app is forced through the same code path as a complex, multi-system orchestration, leading to inefficient resource utilization.
- **Layered Solution:** Each layer can be scaled independently. If your mobile app experiences a surge in traffic, you can scale the **Experience API** without having to scale the expensive and resource-intensive **System APIs** that interact with core databases. Similarly, you can optimize each layer for its specific function (e.g., a high-volume caching layer for the Experience API).

c. Hindered Agility and Innovation

- **Single API Problem:** Changes become slow and risky. Any small change to the API requires a full regression test of the entire application, as a modification for one client could inadvertently break another. This slows down your ability to respond to market demands.
- **Layered Solution:** The layered model allows for parallel development and faster innovation.
- **Experience APIs** can be built quickly to experiment with new user experiences, as they are simply compositing existing functionality.
- **Process APIs** provide a stable foundation for business workflows.
- **System APIs** can be developed and maintained by specialized teams who are experts on the core systems of record, without worrying about how those systems are presented to the end user.

Api Classification Example

Consider the scenario of allowing a customer to upgrade their service plan.

The Right Way (Four-Layer Model):

1. **System APIs:**
2. **CRM-System-API:** Exposes raw customer data and enforces all business rules for a valid customer.
3. **Billing-System-API:** Exposes billing data and enforces all rules for valid billing plans.
4. **Provisioning-System-API:** Exposes low-level provisioning actions and enforces rules for valid service configurations.

5. **Process API:**

6. Update-Service-Process-API: Contains the cross-system orchestration workflow:

1. Calls Billing-System-API to change the plan.
2. Calls Provisioning-System-API to apply the new service configuration.
3. Manages the transaction, with logic to roll back if a step fails.

4. **Canonical API:**

7. Customer-API: Exposes a clean, governed Customer resource, including their ServicePlan. Provides a standard PATCH /customers/{id}/servicePlan endpoint. The mobile and web teams both build their UIs against this stable, reusable contract.

8. **Experience API (An Exception):**

9. A new, high-value partner requires a legacy SOAP XML interface for service upgrades. Instead of polluting the modern Customer-API, we create a short-lived Partner-Upgrade-Experience-API.
10. This API receives a SOAP request, transforms it into a standard JSON request, and calls the reusable Customer-API's PATCH endpoint. This meets the partner's unique need without disrupting our core architecture.

By adopting this four-layer approach, Lumen can create a clear separation of concerns that drives standardization and reuse through the **Canonical API Layer**, while providing managed flexibility for specific consumer needs through the **Experience API Layer**.

The Canonical Data Model (CDM): The Foundation for API Standardization

In our pursuit of a robust and agile enterprise architecture, we face a fundamental challenge: managing the immense complexity of interrelated services and configurations. The common pitfall is to chase a single, perfect, all-encompassing data model, which is unrealistic and ultimately stifles innovation.

This document outlines our strategy for the Canonical Data Model (CDM). It is not a proposal for a rigid, monolithic system, but rather a **principle-based framework for creating a system of models**. This approach allows us to build a stable foundation while embracing flexibility and isolating complexity. Our strategy is founded on three core architectural principles:

1. **Bounded Contexts:** We will manage complexity by breaking down our enterprise into logical domains, each with its own specific and consistent model.
2. **The Immutable Core + Optional Extensions:** We will define a stable, non-negotiable core for each data entity and provide flexibility through well-defined,

optional extensions.

3. **Standardized & Extensible by Design:** All models will be defined using a common, machine-readable format that is inherently designed to be extended over time without breaking its foundation.

Adopting this approach is mandatory for achieving the goals of our Strategic Interface Layer (SIL) and Partner Interface Layer (PIL). It is the key to delivering standardization, agility, and true decoupling.

Example Bounded Contexts

Defining clear bounded contexts is crucial for managing complexity across Lumen's vast product and network portfolio. Here are some logical domains:

1. Product Catalog & Quoting Context

- **Responsibility:** Manages the commercial definitions of Lumen's products. This context is concerned with what can be sold, where it can be sold, and at what price.
 - **Example Data:** Product SKUs, pricing rules, service level agreements (SLAs), and serviceability data (which addresses are on-net). It defines a "service" from a sales perspective.
-

2. Order Management & Orchestration Context

- **Responsibility:** Governs the end-to-end lifecycle of a customer order from submission to completion. It tracks the status of an order as it moves through various fulfillment systems.
 - **Example Data:** Order ID, customer details, list of services on the order, and high-level order statuses (e.g., In Progress, Completed, Cancelled).
-

3. Customer & Site Management Context

- **Responsibility:** Acts as the master source for all customer account information and their physical locations (sites).
 - **Example Data:** Customer legal name, account numbers, contact information, and site addresses (CLLI codes, latitude/longitude, site contacts).
-

4. Service Provisioning & Activation Context

- **Responsibility:** This is a large context often broken down further. It's concerned with the technical configuration of a service on the network.
- **Access/UNI Sub-Context:** Manages the customer-facing interface. Data includes port speed, encapsulation type and VLAN IDs. This is the physical or virtual handoff to the customer.

- **Metro & Transport Sub-Context:** Manages the underlying network paths within and between cities. Data includes MPLS LSPs, segment routing paths, and optical circuit details.
 - **Edge & Peering Sub-Context:** Manages how customer traffic gets to the internet or other VPN sites. Data includes BGP sessions, route-maps, and VRF/GRT definitions.
 - **Service Layer Sub-Context:** Defines the end-to-end service construct itself. For an L3VPN, this context would orchestrate elements from the Access and Edge contexts to build the complete virtual private network.
-

5. Network & Service Assurance Context

- **Responsibility:** Monitors the health of the network and services post-activation. It manages fault detection, performance monitoring, and trouble ticketing.
 - **Example Data:** Circuit IDs, alarm statuses, performance metrics (latency, jitter), and trouble ticket numbers. A "service" in this context is something to be monitored.
-

6. Billing & Invoicing Context

- **Responsibility:** Manages the financial aspects of a customer's service. It's concerned with monthly recurring charges (MRCs), usage-based charges, and generating invoices.
- **Example Data:** Billing account numbers, charge codes, and invoice details.

What is the Canonical Data Model (CDM)?

The Canonical Data Model (CDM) is the single, unified, and consistent data structure for our key business entities (e.g., Customer, Order, Service, Port) across the enterprise. It acts as the universal language for all our APIs.

In the context of our new architecture—the Strategic Interface Layer (SIL) and the Partner Interface Layer (PIL)—the CDM is not just beneficial, it is **absolutely mandatory** for achieving our goals of standardization, agility, and decoupling.

Why the CDM is Critical in the PIL and SIL Context

The CDM's primary role is to enforce the principle of **decoupling**—shielding consuming applications from the technical complexity and volatility of backend systems.

Strategic Interface Layer (SIL)	Partner Interface Layer (PIL)
Mission: Provide clients (internal and external) with a stable, predictable, and consistent view of our business services.	Mission: Protect our internal services from the complexity and constant change of integrating with external third-party vendors.
How the CDM Helps: The SIL uses the CDM as its public contract. All data, regardless of its source system, is translated into the standard CDM format before being exposed through the SIL.	How the CDM Helps: The PIL uses the CDM as its internal standard. It translates vendor-specific data models into our internal CDM, ensuring our core

Strategic Interface Layer (SIL)	Partner Interface Layer (PIL)
	services only ever have to understand one data language.

Benefits for the Strategic Interface Layer (SIL)

Benefit	Why it Matters
Consistent Client Contract	The CDM ensures that all clients (mobile, web, partners) receive the same field names, formats, and structures for an entity like 'Customer', regardless of which backend system is supplying the data. This eliminates ambiguity and reduces integration costs for consumers.
Simplified Orchestration	When the SIL orchestrates a complex business process, data retrieved from one system (e.g., Billing) is immediately recognizable and usable when passed to another system (e.g., Provisioning). The CDM removes the need for multiple, complex data translations within the SIL itself.
Accelerated Migration	We can replace a legacy backend system with a new strategic one without changing the client-facing API contract. The old system was translated to the CDM, the new system will natively speak the CDM. This gives us immense technical and business flexibility.

Benefits for the Partner Interface Layer (PIL)

Benefit	Why it Matters
Internal Decoupling	Internal services only ever see and use our Canonical Model for a concept like a 'Port'. They are entirely shielded from the specific, often messy, data formats (XML, JSON, SOAP) required by partners like AT&T or Verizon.
Plug-and-Play Partner Integration	Partner Adaptors become simple two-way translators: Our CDM ⇔ Partner Format . This means integrating a new partner (e.g., Starlink) is fast, requiring only a new adaptor, with zero code change to the PIL or our internal services.
Vendor Neutrality	Business rules for partner selection (managed by the PIL) can be based on standardized CDM attributes (e.g., portType, locationID), not vendor-specific identifiers. This makes switching partners a business decision, not a major technical project.

Governance and Ownership

This is a partnership between Product and Architecture.

Area	Product Team Responsibility	Architecture Team Responsibility
Definition	Define the business fields, naming conventions, data types, and business rules for their domain's model. They are the ultimate authority on what a "Customer" or "Order" means to the business.	Define the technical format (e.g., JSON Schema, OpenAPI), security policies (e.g., Field-Level Authorization), and overall versioning strategy . They provide the guardrails and tools.
Version Control	Drive the roadmap for the next version of the model, communicating changes clearly to all consumers and stakeholders. They manage the "product backlog" for the data model.	Enforce Semantic Versioning (e.g., v1.0.0), ensuring no breaking changes are introduced without proper notification and deprecation plans. They act as the release managers for the technical standard.
Enforcement	Ensure all new strategic backends created by the domain team natively expose the CDM. Advocate for the CDM in all new development.	Mandate that all Adaptors (both SIL and PIL) perform the necessary translation to the CDM before data is passed to core services. They ensure compliance at the architectural level.

The CDM is a Product, Not a Project

To succeed, we must treat our Canonical Data Models as long-lived, strategic assets.

A. The CDM as an Internal Product

A project has a start and an end date. A product has a lifecycle, a roadmap, and dedicated owners who seek to maximize its value for its customers.

- **Your Consumers are Customers:** The "customers" of your domain's CDM are all the other development teams (internal and external) who will use it. Your goal is to make their job easier by providing a clear, stable, and well-documented data contract.
- **Create a Roadmap:** What new attributes will be needed in the next 6-12 months? Are there upcoming business initiatives that will require changes to the Order or Service entity? Plan for this evolution and communicate it transparently.
- **Prioritize and Justify Changes:** Every change to the CDM has a ripple effect. Changes must be justified by clear business value, not just technical preference. Use a backlog to manage and prioritize requested changes from consuming teams.

B. The Strategic Framework: Domain-Driven Design (DDD)

The principle of "Context Boundaries" mentioned earlier comes directly from Domain-Driven Design (DDD), a crucial framework for avoiding common CDM pitfalls. DDD helps us manage complexity by acknowledging that a single model cannot effectively serve the entire business.

- **Bounded Contexts are Your Friend:** The business is composed of different "Bounded Contexts" (e.g., Sales, Support, Billing), each with its own specific language and model. A "Customer" in the Sales context (with attributes like `leadScore`) is different from a "Customer" in the Billing context (with attributes like `paymentTerms`).
- **The CDM is the Shared Kernel:** Instead of a universal "God Model," the CDM should represent the **Shared Kernel**—the small, stable subset of the model that different contexts must agree on. The `customerID` and `legalName` are canonical and shared. The Sales team's `leadScore` is not; it belongs only within their Bounded Context.
- **Anti-Corruption Layers:** When contexts interact, DDD uses an "Anti-Corruption Layer" to translate between models. Our SIL and PIL adaptors are perfect examples of this. They protect our core services from the specific, and often messy, data models of other systems.

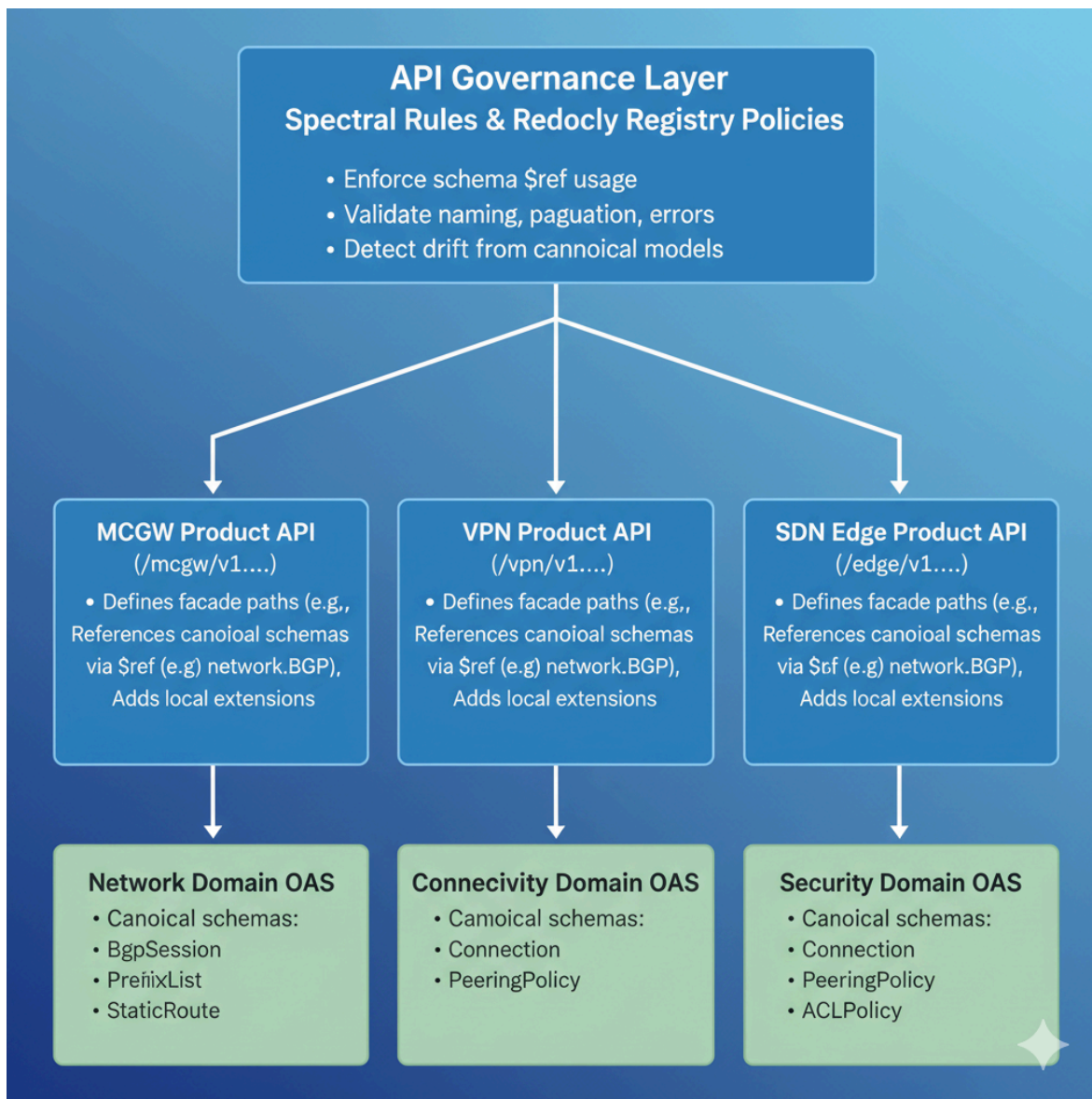
Canonical Model Creation Workflow

Introduction

This document provides the official blueprint and workflow for creating an API Canonical Model at Lumen. Its purpose is to guide architects and teams through a strategic, collaborative process that ensures every canonical model is a consistent, reusable, and valuable enterprise asset.

Following this workflow is essential for implementing our core architectural principles of decoupling, consistency, and agility. It transforms the creation of a model from a siloed technical task into a strategic design process that aligns with our business domains.

As illustrated in the diagram below, this workflow produces reusable **Domain Models** (the foundation), which are consumed by product-facing **Facade APIs** (the experience layer), and are all enforced by an automated **Governance Layer** (the contract).



Guiding Principles

Before starting the workflow, internalize these core principles that govern our approach:

- **Model the Domain, Not the System:** The model must represent the business reality (a "Customer," a "Service"), not the database tables or technical quirks of a legacy backend system.
- **Embrace Bounded Contexts:** Do not attempt to create a single model for the entire enterprise. Acknowledge that a "Port" in the Access domain has different attributes than a "Port" in the Assurance domain. Design models that are specific and unambiguous within their logical boundary.
- **Design for Stability and Extension:** Identify the stable, **Immutable Core** of a model and protect it. Allow for flexibility and future growth through well-defined **Optional Extensions**.
- **Treat the Model as a Product:** A canonical model is a long-lived asset with other development teams as its customers. It requires a product mindset, including dedicated

ownership, a roadmap, and a clear versioning strategy.

The Workflow

Phase 1: Discovery and Scoping 🗺️

In this phase, you define the problem space and establish the boundaries for your model.

1. **Define the Business Capability:** Clearly articulate the business process the model will support. Start with the "why."
2. *Example: "Provision a Last Mile Cloud Connect service" or "Retrieve a customer's billing history."*
3. **Establish the Bounded Context:** Work with product and enterprise architects to draw the conceptual boundary around the domain. Determine what entities and attributes are inside this context and what belongs to others. This is the most critical step for managing complexity.
4. **Identify Domain Experts and Stakeholders:** Identify the key people who have authoritative knowledge of this domain. This includes Product Managers, business stakeholders, and senior engineers from the relevant teams. Modeling cannot happen in isolation.

Phase 2: Collaborative Model Design 🧩

This is the core workshop phase where the model takes shape through collaboration.

1. **Host Design Workshops:** Bring the identified domain experts together. Use a whiteboard or digital collaboration tool to visually map out the entities and their relationships within the bounded context.
2. **Identify Core Entities:** Within the bounded context, identify the key "nouns."
3. *Example: For a Network domain, this might be Port, Circuit, LogicalDevice, and BgpSession.*
4. **Define the Immutable Core:** For each entity, define its essential, non-negotiable attributes that are stable and unlikely to change.
5. *Example: For a Port model, this would be its portID, siteID, and physicalInterfaceType.*
6. **Define Optional Extensions and Relationships:** Flesh out the model with attributes that provide flexibility or may not always be present. Define the relationships between entities.
7. *Example: A Circuit is composed of two Ports.*

Phase 3: Formalization and Review

This phase turns the conceptual model into a formal, ratified asset.

1. **Draft the Formal Specification:** Translate the workshop outputs into a formal, machine-readable contract using OpenAPI Specification (v3.x) and standalone JSON Schema files. These schemas will form the **Domain OAS** (e.g., Network, Connectivity) shown in Figure 1.
2. **Conduct Architectural Review:** Submit the draft specification for a formal review with the API Governance Guild. The model is checked for adherence to enterprise standards using automated tools like **Spectral** alongside peer review.
3. **Ratify and Version the Model:** Once approved, the model is formally ratified. It is assigned a version number (e.g., v1.0.0) following Semantic Versioning (SemVer) and becomes the official standard for that entity within its bounded context.

Phase 4: Publication, Implementation, and Governance 🚀

The final phase involves publishing the model and managing its lifecycle within our architecture.

1. **Publish to the Domain OAS Registry:** The ratified model specification is published to the official Lumen Schema Registry (a dedicated Git repository). This registry contains the versioned **Domain OAS** files (the green boxes in Figure 1) and is the single source of truth for all teams.
2. **Implement via Product API Facades:** The canonical models are consumed by the various **Product APIs** (e.g., MCGW, VPN), which act as facades. As shown in the middle layer of Figure 1, these APIs use `$ref` to reference the canonical schemas from the Domain OAS registry and can add their own local, product-specific extensions.
3. **Enforce via the Governance Layer:** The ratified model is a living asset. The **API Governance Layer** (the top box in Figure 1) uses tools like Spectral and the Redocly Registry to automatically validate the Product APIs. This ensures they are correctly implementing the canonical models, enforces consistent naming, and detects any "schema drift," protecting the integrity of our API ecosystem. Any proposed changes to a canonical model must follow this governance process before a new version is released.

Principles of Cloud-First API Design

These principles are the strategic solution to the very problems our customers and internal teams face. They provide the "North Star" that justifies our tactical API Style Guide.

How to Use These Principles: A Guide for Stakeholders

This section translates the high-level principles into actionable guidance for different roles.

For Product Managers

Your Role: Define the "What" and the "Why" You are the voice of the customer and the business. These principles empower you to define products, not just features.

- **Define Workflow Resources, Not Verbs:** When writing user stories, define the outcome the customer wants, not the steps they take. Your requirement should be "A customer can change bandwidth with a single API call" (**Principle 1 & 2**). This forces the creation of a simple orchestration façade.
- **Make Monetization a Requirement:** Every new API feature **MUST** have defined billing and entitlement logic. You must specify the `usageType` and `meteringUnit` as part of the feature definition (**Principle 5**).
- **Prioritize a "No-Ticket" Experience:** Your product vision **MUST** be geared toward 100% API-driven self-service. Any requirement for manual intervention or a support ticket is a design failure (**Principle 6**).
- **Own the API Metrics:** You are responsible for the API's success KPIs. You **MUST** define and track metrics for adoption, latency, and error rates to drive your roadmap and prove business value (**Principle 13**).

For Developers & Engineers

Your Role: Implement the "How" You build the reliable, consistent, and scalable platform. These principles are your blueprint for implementation.

- **Implement the Asynchronous Pattern:** For any operation that takes more than 2 seconds (e.g., provisioning), you **MUST** implement the async request-reply pattern. Return a 202 Accepted and a Location header pointing to a job/operation resource (**Principle 3**). Do not build long-polling, synchronous endpoints.
- **Enforce Standards in Your Pipeline:** You **MUST** integrate the provided Spectral linting rules into your CI pipeline. A build that violates the API Style Guide (naming, structure, etc.) **MUST** fail (**Principle 11**).
- **Use the Standard Patterns:** Do not invent your own error models, pagination, or authentication flows. You **MUST** use the platform-standard `problem+json` for errors, Link headers for pagination, and OAuth 2.0 for security (**Principle 12 & 8**).
- **Abstract Internal Complexity:** When building a façade, you are responsible for hiding the "identifier hell." Your implementation **MUST** handle the mapping between the single canonical API ID and all necessary internal system IDs.

For Architects & Platform Leadership

Your Role: Enforce the Platform Vision You are the guardians of platform cohesion, long-term strategy, and governance.

- **Own the Canonical Taxonomy:** You are responsible for defining and enforcing the uniform naming and versioning strategy (`/ {product} / v {version} / {resource}`). You **MUST** reject designs that create URI conflicts or inconsistent paths (**Principle 4 & 7**).

- **Champion Automation:** You **MUST** provide and maintain the automated governance tools (e.g., the master Spectral ruleset) that enable developers to comply with the standards at scale (**Principle 11**).
- **Design for the Ecosystem:** Your architectural reviews **MUST** ensure that new APIs are designed for federation and extensibility. A design that only works for a single use case and cannot support partners is a strategic failure (**Principle 10**).
- **Maintain Platform Trust:** You are responsible for enforcing policies on backwards compatibility and graceful evolution. You **MUST** ensure that teams follow proper deprecation procedures to avoid breaking client integrations (**Principle 15**).

1. Resource-Oriented Interface (The "How")

This is our core design philosophy. It means we model **all** interactions—from simple data management to complex, multi-step workflows—as "resources" (nouns) that a client can interact with.

This principle **explicitly forbids** exposing internal verbs or process steps (RPC-style).

It's crucial to understand that "resource-oriented" is **not** just for simple CRUD (Create, Read, Update, Delete). It is the *interface style* we use for two distinct patterns:

1. **CRUD-Style Resources:** For simple, state-based entities (e.g., `/ports`, `/users`).
2. **Workflow-Style Resources:** For complex, outcome-based actions (e.g., `/orders`, `/connections`).

Why: A consistent, resource-based interface enables self-service, idempotency, and alignment with all modern cloud provider conventions (AWS/GCP/Azure).

2. Desired-State & Outcome-Based (The "What")

This is the *message* you send to a resource. It's the **opposite of an imperative command**.

Instead of a step-by-step *process*, the client sends a *declarative payload* describing the **desired end state**. Our orchestration façade is then responsible for making the "actual state" match that "desired state."

Example: The Core Distinction

This is the most important concept. We **abstract** our internal process into a single, simple, resource-based API call.

✗ **Anti-Pattern: The RPC/Verb Process**

The client must know our internal steps. (**FORBIDDEN**)

```
POST /api/checkPrice
POST /api/createQuote
POST /api/submitOrder
```

✅ The "Workflow Resource" (Our Standard)

The client interacts with **one resource** (/v1/connections) and provides a **desired-state** body. Our façade hides the *entire* price/quote/order process.

```
POST /v1/connections
{
  "bandwidth": "10G",
  "src_port": "port-abc",
  "dst_port": "port-xyz"
}
```

3. Asynchronous and Operation-Tracked

What it means: Long-running provisioning and teardown tasks (like the POST /v1/connections example above) **MUST** return an operation resource immediately.

Example:

1. POST /fabric/v1/connections → 202 Accepted
2. **Response Header:** Location: /fabric/v1/operations/12345

Why: Mirrors GCP/Azure async operation patterns; avoids client timeouts and supports distributed job orchestration.

4. Product-Scoped Versioning

What it means: Versioning sits within the product namespace, never at the cross-domain layer.

Example:

- /fabric/v1/telemetry/events
- /mcgw/v1/connections

Why: Each product can evolve independently; prevents version collisions across domains.

5. Entitlement-Aware and Monetizable

What it means: Every resource request validates the caller's entitlements and records usage events for billing.

Example Metadata:

```
{
  "entitlementId": "ENT-123",
  "usageType": "bandwidth-hours",
  "meteringUnit": "GB"
}
```

Why: Supports SaaS-like pay-per-use and quota enforcement similar to cloud hyperscalers.

6. Self-Service and Declarative Lifecycle

What it means: Customers and partners can onboard, provision, and manage services fully via API or IaC templates—no ticketing.

Why: Enables elasticity, automation, and integration into DevOps pipelines.

7. Uniform Taxonomy and Namespacing

What it means: Product namespaces and resource naming follow a canonical structure: `/{{product}}/v{{version}}/{{resource}}/{{subresource}}`

Why: Drives predictability across APIs, allowing governance automation (Spectral rules, gateway routing) and consistent developer experience.

8. Strong Identity and Policy Integration

What it means: OAuth 2.0/OIDC as baseline; JWT claims carry entitlements, scopes, and tenant context.

Example Scopes: `fabric:read`, `mcgw:modify`, `wholesale:metrics`

Why: Enables fine-grained authorization and uniform policy enforcement across products.

9. Observable and Telemetry-Driven

What it means: APIs emit standardized metrics and events: latency, success rate, error codes, SLO violations, and usage.

Example:

```
POST /telemetry/events
{
  "api": "/fabric/v1/ports",
  "latencyMs": 245,
  "status": 200
}
```

Why: Provides the foundation for automated SLA tracking, billing accuracy, and health dashboards.

10. Extensible Through Federation and Partner APIs

What it means: Partner and wholesale APIs follow the same design principles, enabling multi-tenant federation.

Example:

- /wholesale/fabric/v1/connections
- /partner/mcgw/v1/orders

Why: Supports ecosystem integration (hyperscalers, value-added resellers) without duplicating backend logic.

11. Automation and Governance-Ready

What it means: Design patterns are machine-verifiable. URI structure, version placement, naming, and required metadata are enforced by Spectral linting and CI pipelines.

Why: Guarantees compliance at scale, reduces manual reviews, and allows self-serve publication to catalogs (e.g., Backstage).

12. Developer Experience Consistency

What it means: Consistent error models, pagination, and async patterns across all APIs; SDKs and documentation generated from OAS.

Example Error:

```
{
  "errorCode": "FABRIC-404",
  "message": "Port not found",
  "correlationId": "abc-123"
}
```

Why: Developers can reuse tooling and templates across products, just like AWS SDK parity across services.

13. Outcome Metrics and Continuous Improvement

What it means: APIs publish KPIs on adoption, latency, and SLA compliance to a governance dashboard.

Why: Drives data-driven evolution and productization—every API is treated as a measurable SaaS asset.

14. Compliance, Security, and Data Residency by Design

What it means: Security and compliance are embedded through consistent header policies, trace IDs, and encryption standards, not added after design.

Why: Supports multi-jurisdictional and regulated workloads (finance, healthcare).

15. Backwards Compatibility and Graceful Evolution

What it means: APIs evolve additively; breaking changes require a new major version. Deprecated endpoints remain discoverable with Deprecation headers and clear timelines.

Why: Ensures clients can safely migrate; matches cloud provider deprecation best practices.

=====

Reference Architectures (The "Playbooks")

Reference Architectures (The "Playbooks")

- [Proposed Sell Reference Architecture: : A Strategic Interface Layer Architecture](#)

- [Proposed "Buy" Reference Architecture for Partner Integration at Lumen: The PIL Model](#)

Proposed Sell Reference Architecture: : A Strategic Interface Layer Architecture

The Challenge

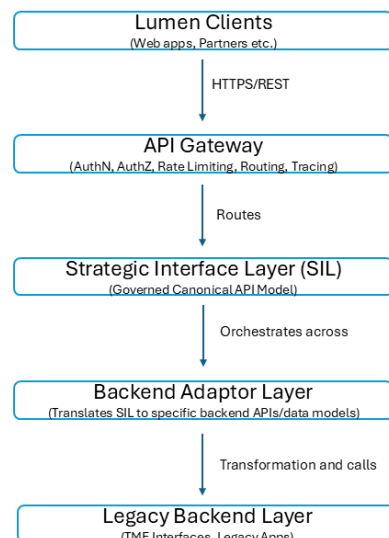
Lumen, a company with a history of frequent mergers and acquisitions, faces a common architectural dilemma: a fragmented backend landscape. Decades of integrating disparate systems have resulted in a lack of standardization, multiple APIs serving the same concepts with inconsistent data models, and a significant impediment to agility and unified customer experiences. The core challenge is to standardize Lumen's digital platform without requiring an immediate, disruptive "big bang" overhaul of all legacy systems

The Proposed Solution: A Strategic Interface Layer (SIL)

To address these challenges, we propose an architecture centered around a **Strategic Interface Layer (SIL)**. This layer acts as a crucial abstraction and orchestration point, designed to provide immediate standardization for clients while enabling a phased, long-term migration of legacy backends.

The architecture revolves around two key principles:

- **Client Abstraction:** Shielding client applications from the inherent complexity and inconsistencies of Lumen's diverse backend systems.
- **Phased Migration:** Allowing legacy backends to be refactored or replaced incrementally, without impacting the client-facing APIs.



Key Components:

1. **Lumen Clients:** Front-end applications (web, mobile, partner integrations) that consume Lumen's digital services. Their primary need is a stable, consistent, and well-documented API.
2. **API Gateway :** The external entry point for all client requests. After enforcing standard gateway functions (security, rate limiting, logging), this component will route all request to SIL Layer
3. **Strategic Interface Layer (SIL):** The is "the face of Lumen APIs". It is key interface, based off a canonical, well governed API model exposed to clients. This is the orchestration and standardization layer. Its responsibilities include:
4. **Standardized Contract:** Exposing a unified, domain-driven API contract for core Lumen concepts (e.g., Customer, Order, Service).
5. **Lean Orchestration:** For requests that involve multiple legacy backends or complex workflows not yet encapsulated by a single strategic backend.
6. **Data Model Transformation:** Ensuring that data returned to clients conforms to the strategic data model, regardless of the underlying backend's format.
7. **Backward Compatibility:** Managing API versions to ensure existing clients are not broken during backend transitions.
8. **Backend Adapter Layer:** A sub-component of the SIL (or a set of distinct services called by the SIL) responsible for translating the standardized SIL requests into the specific API calls and data formats required by individual legacy backends. Each adapter is purpose-built for a particular legacy system.
9. **Legacy Backends:** The existing, unstandardized systems resulting from Lumen's M&A history. These systems will eventually be deprecated, refactored, or replaced.

Operational Flow

- A **Lumen Client** makes a request to the **API Gateway**.
- The **API Gateway / Smart Router** routes the request to the **Strategic Interface Layer (SIL)**.
- The **SIL** then performs any necessary orchestration, calls the appropriate **Backend Adapter** for legacy systems.
- Responses are transformed by the SIL (if originating from legacy systems) to adhere to the strategic data model before being sent back to the client via the Gateway.

Interaction Principles (The Rules of Engagement)

This defines how the components **must** interact to ensure decoupling and maintain clear architectural boundaries.

- **External Consumers:** All external clients **must** only ever call the APIs exposed at the API Gateway. They are forbidden from calling any internal service directly.

- **API Gateway:** The Gateway after performing its function, routes the requests to SIL
- **Decoupling:** The public-facing API contract **must** remain stable, even when a backend system is migrated from the "Legacy Path" to the "Modern Path." This transition must be completely transparent to the external consumer.

Governance & Non-Functional Requirements (The Standards)

This ensures that every implementation is secure, consistent, and observable.

- **Security:** All endpoints exposed at the API Gateway **must** be secured using the enterprise standard (e.g., OAuth 2.0).
- **Observability:** All transactions **must** include a correlation ID that is passed through every layer for end-to-end tracing.
- **API Standards:** All public-facing APIs **must** adhere to the official Lumen API Style Guide.

The Colocation Principle: Context

The "Sell" Side Reference Architecture is designed as a layered, distributed system. Effective performance of this architecture relies on low-latency communication between its internal components, particularly between the Strategic Interface Layer (SIL) and its Backend Adapters.

However, Lumen's current enterprise constraint of a single, centralized API Gateway in GCP creates a "network hairpin" problem, introducing prohibitive latency (~130ms+ per hop) for any service-to-service communication that crosses cloud boundaries. While long-term solutions like a Federated Gateway or a Service Mesh are the strategic goal, they are not immediately available.

Therefore, to ensure the performance and viability of all near-term SIL implementations, the **Colocation Principle is adopted as the mandatory, pragmatic interim deployment pattern**

The Colocation Principle: Mandated Rules

Given the immediate need for a workable solution, the following rules are mandated for all SIL implementations until a federated gateway or service mesh is available.

- **SIL and Adapters as a Single Deployment Unit:** The SIL and its corresponding Backend Adapters **must** be deployed in the same cloud, same VPC, and ideally the same cluster. They should be treated as a single, cohesive deployment unit to guarantee low-latency communication.

- **Intra-Cloud Orchestration Only:** The SIL is strictly forbidden from performing real-time, synchronous orchestration that calls services or adapters across different cloud environments (e.g., a SIL in AWS cannot call an adapter in GCP). All real-time orchestration must be contained within a single cloud boundary.
- **Separate Logical Deployments Required:** While they must be colocated, the SIL and its Backend Adapters **must** remain logically separate deployments (e.g., distinct microservices or containers). Bundling adapters as a library (e.g., a JAR file) inside the SIL is considered an anti-pattern, as it creates a monolith and prevents independent deployment, scaling, and maintenance.
- **Multi-Cloud Aggregation via Asynchronous Data:** For any use case that requires a unified view of data from multiple clouds, the approved pattern is **asynchronous data aggregation**, not real-time orchestration. The SIL **must** perform a single, low-latency query against a pre-aggregated data store (e.g., a Data Hub or Lakehouse) rather than performing a live, cross-cloud "fan-out" to multiple services.

Proposed "Buy" Reference Architecture for Partner Integration at Lumen: The PIL Model

This document outlines the proposed **"Buy" Reference Architecture**—the strategy for integrating external partner capabilities (e.g., off-net ports, connectivity from AT&T, Verizon, Starlink) into Lumen's digital platform. This architecture is designed for **security, governance, and decoupling** to ensure that Lumen's internal services are protected from the complexity and potential volatility of third-party APIs.

The Need for Standardization

Just as the Strategic Interface Layer (SIL) standardizes Lumen's *selling* capabilities, a robust "Buy" architecture is needed to standardize *partner sourcing*. When buying capabilities, the goal is to abstract the partner's technology and data model from your internal systems. Without it, internal Lumen services would be tightly coupled to specific partner technologies, leading to:

- **High Maintenance Cost:** Any change to a partner's API would require updating every Lumen service that calls it.
- **Lack of Agility:** Adding a new partner (e.g., Starlink) would require extensive code changes across the organization.
- **Security Risk:** Scattered management of external partner credentials and security policies.

The Core Principle: A Playbook, Not a Single Solution

This Reference Architecture is not a single, monolithic system. It is a strategic **pattern** that will be implemented for each distinct "Buy" side business domain (e.g., an "OffNet

Connectivity PIL," a "Cloud Marketplace PIL").

The Reference Architecture is prescriptive about the **principles and components** (the "what" and "why") but flexible about the **implementation specifics** (the "how"). It provides strong guardrails while empowering delivery teams to make the right architectural choices for their specific business problem.

Structural Components (The "What")

This defines the non-negotiable building blocks that **must** be used in every implementation to ensure structural consistency across the enterprise.

Component	Primary Role	Rationale
Partner Interface Layer (PIL)	The internal-facing facade for a specific business domain.	Centralizes domain-specific business logic (e.g., partner selection) and exposes a single, clean API for internal consumers.
Partner Adaptors	A dedicated microservice for each individual partner.	Ensures technical isolation. If a partner's API changes, only its single adaptor is updated, protecting all other components.
External Partner Gateway	The single, governed exit point for all outbound traffic to partners.	Critical for governance. Enforces uniform security, rate limiting, and centralized logging, preventing unmanaged service-to-internet sprawl.
Webhook Ingestion Layer	The single, governed entry point for all inbound asynchronous events from partners.	Provides a secure and resilient "front door" for partner webhooks, authenticating and queuing events before they are processed.

Interaction Principles (The Rules of Engagement)

This defines how the components **must** interact to ensure decoupling and maintain clear architectural boundaries.

- **Internal Consumers:** Internal services **must** only ever call the PIL's interface. They are forbidden from calling a Partner Adaptor directly.
- **The PIL:** The PIL **must** expose a clean, domain-specific API that hides the underlying complexity of partner orchestration and selection.
- **Partner Adaptors:** An Adaptor's responsibility is limited to the technical concerns of a single partner (authentication, data transformation). It **must not** contain business logic that spans multiple partners.

Governance & Non-Functional Requirements (The Standards)

This ensures that every implementation is secure, consistent, and observable, regardless of the specific use case.

- **Security:** All endpoints exposed by the PIL and the Webhook Ingestion Layer **must** be secured using the enterprise standard (e.g., OAuth 2.0). All outbound traffic from the External Partner Gateway **must** use appropriate transport-layer security (e.g., mTLS).
- **Observability:** All transactions **must** include a correlation ID that is passed through every layer (PIL, Adaptor, Gateway) for end-to-end tracing.
- **Logging:** A standardized logging format **must** be used by all components to enable centralized analysis and alerting.

A Decision Framework for Extensions (The "How")

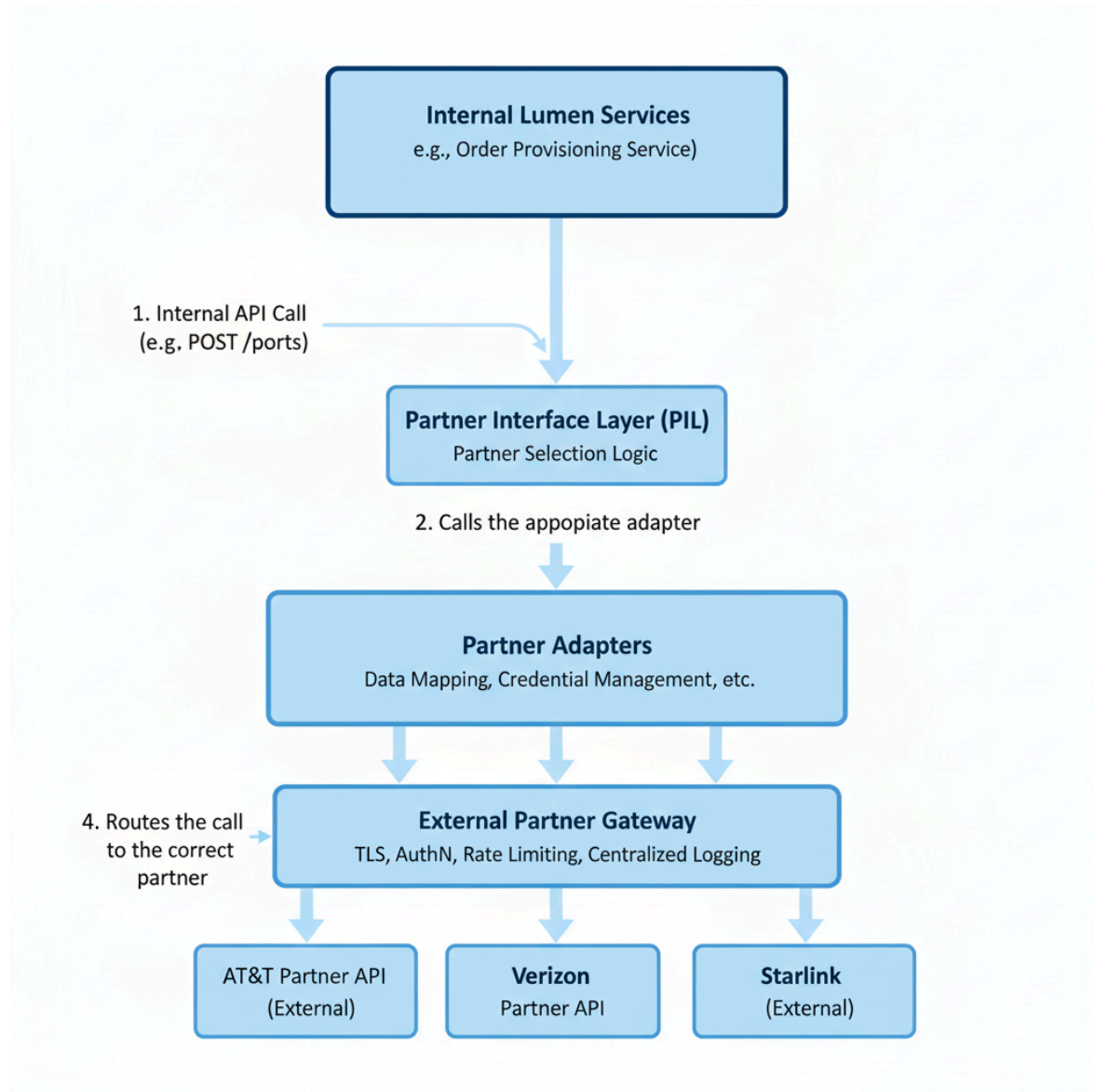
The Reference Architecture provides guided flexibility. Any team building an implementation **must** consciously make and document their decisions based on this framework.

The most critical decision is the state management strategy. The design **must** explicitly document whether it is using a "Stateful" or "Stateless" pattern and provide a clear justification.

- **Question 1: What is the Source of Truth?**
- If the **partner** is the absolute source of truth for the resource (like a cloud resource), a **Stateless** pattern is required.
- If **Lumen** is the source of truth for the *transaction* itself (like an OffNet order), a **Stateful** pattern is required.
- **Question 2: What is the Lifecycle of the Transaction?**
- Is it a short-lived, real-time request/response? -> **Stateless**.
- Is it a long-running, asynchronous process that could take days or weeks? -> **Stateful**.
- **Question 3: How are Out-of-Band Changes Handled?**
- The design **must** address how it will handle the "tricky case" where a resource's state is changed on the partner's side without a webhook (e.g., via their portal). A Stateful pattern, for example, must include a state reconciliation mechanism

The Secure Outbound Flow

All requests for partner services must follow a stringent, controlled path to ensure security and maintainability.



Internal Lumen Service → PIL → Adaptor → External Partner Gateway → 3rd Party Partner API

Step	Component	Action
Internal Call	Lumen Service	Makes a standardized request to the PIL (e.g., "Find Port in Dallas").
Business Logic	PIL	Applies business rules to select the best partner (e.g., "Verizon is cheapest here") and forwards the request to the corresponding Adaptor.
Transformation/Vendor specific AuthN	Adaptor	Transforms the standardized PIL request into the exact format and data model required by the chosen partner's API. Also

Step	Component	Action
		responsible for obtaining/caching the OAuth token for a specific external vendor.
Routing	External Partner Gateway	Manages mTLS/TLS, applies rate limits, and centrally logs the external transaction.
External Execution	3rd Party Partner	Receives and executes the request.

Key Benefits

This layered "Buy" architecture solves the core enterprise challenges of partner integration:

- **True Decoupling:** Lumen's internal systems only ever talk to the **PIL**'s standardized contract. They are completely ignorant of partner technology, meaning replacing 3rd party vendor with a new provider has zero impact on core Lumen code.
- **Centralized Security:** By forcing all outbound calls through the **External Partner Gateway**, Lumen ensures all traffic is uniformly secured and adheres to external partner agreements and rate limits.
- **Agile Onboarding:** Integrating a new partner is a focused project—it requires only building a new **Partner Adaptor**. The existing PIL, internal services, and Gateway remain untouched.
- **Resilience:** The **PIL** can implement failover logic. If the vendor Adaptor reports an API error, the PIL can automatically retry if the error is recoverable.

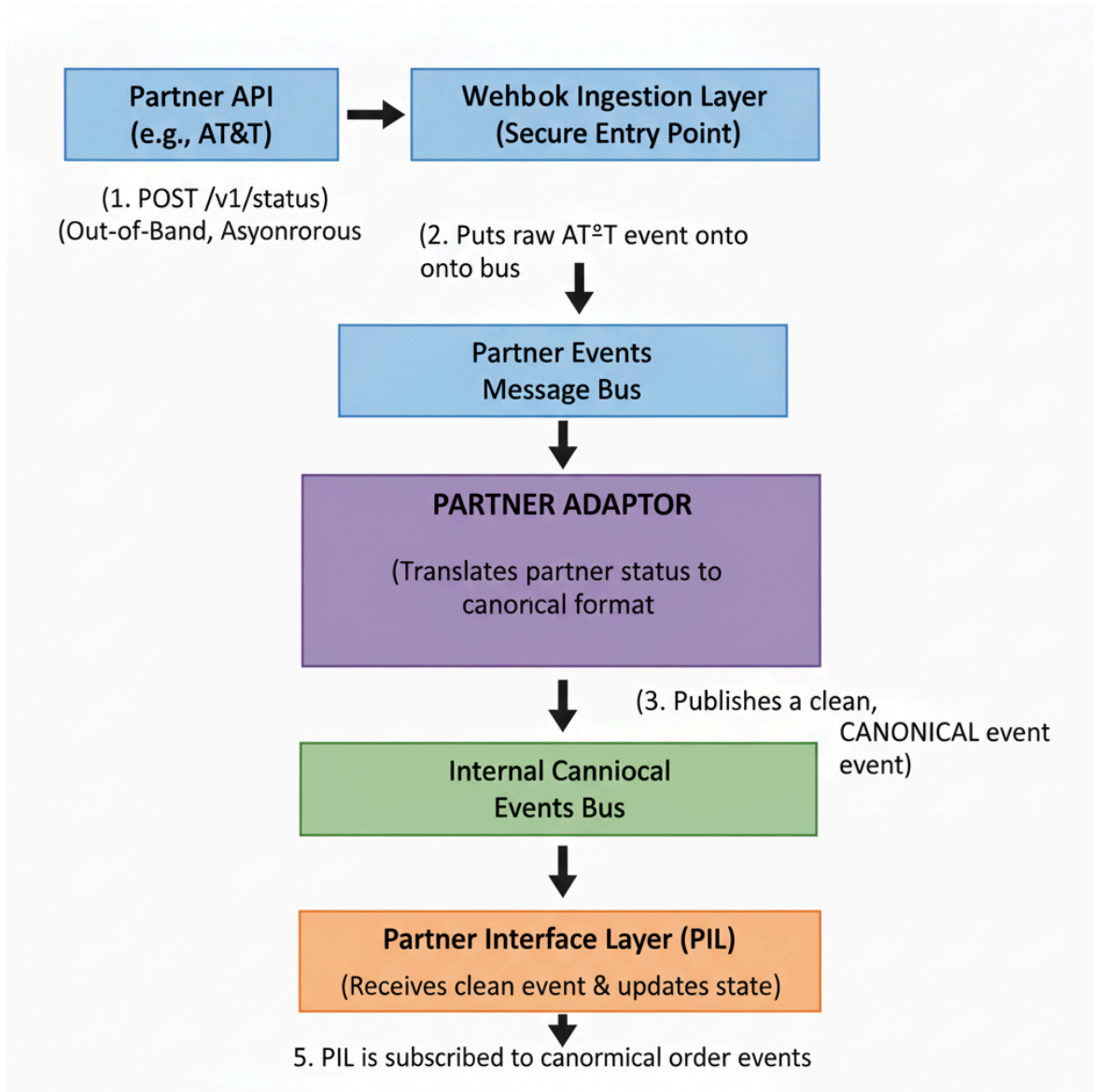
Inbound Flow - Handling Asynchronous Partner Events (Webhooks)

For long-running processes like service provisioning, partners provide asynchronous status updates via webhooks. The Reference Architecture mandates a secure, resilient, and decoupled pattern for ingesting and processing these events.

The flow is the mirror image of the outbound request, ensuring that internal systems are shielded from the complexity and proprietary formats of partner events.

1. **Ingestion:** A partner system sends a POST request to a single, secure endpoint exposed by the **Webhook Ingestion Layer**. This endpoint is specific to the partner (e.g., `https://webhooks.lumen.com/v1/att`). The payload is in the partner's native format.
2. **Authentication & Queuing:** The Ingestion Layer's only jobs are to authenticate the request (e.g., via an API key or HMAC signature) and immediately place the raw, untranslated event onto a dedicated **Partner Events Message Bus**. This ensures resilience and durability.

3. **Adaptor Translation:** The appropriate **Partner Adaptor** (e.g., the AT&T Adaptor) is subscribed to its topic on the message bus. It consumes the raw event and performs the critical translation, converting the partner's proprietary status codes and data fields into the official **Lumen Canonical Model**.
4. **Canonical Event Publication:** The Adaptor publishes a new, clean, **canonical event** (e.g., `lumen.order.status.updated`) onto a separate **Internal Canonical Events Bus**.
5. **Internal Consumption:** The appropriate **PIL** (or other internal service) is subscribed to the internal bus. It receives the clean, canonical event and can reliably take action, such as updating an order's state in its database.



Deployment Mandate: The Colocation Principle for the PIL

1. Context and Rationale

The "Buy" Side Reference Architecture is designed as a layered, distributed system that relies on high-chattiness, low-latency communication between its internal components, particularly between the **Partner Interface Layer (PIL)** and its **Partner Adaptors**. This is especially critical during the "fan-out" process for multi-partner quoting.

However, Lumen's current enterprise constraint of a single, centralized API Gateway in GCP creates a "network hairpin" problem, introducing prohibitive latency (~130ms+ per hop) for any service-to-service communication that crosses cloud boundaries. While long-term solutions like a Federated Gateway or a Service Mesh are the strategic goal, they are not immediately available.

Therefore, to ensure the performance and viability of all near-term PIL implementations, the **Colocation Principle is adopted as the mandatory, pragmatic interim deployment pattern.**

2. The Colocation Principle: Mandated Rules

Given the immediate need for a workable solution, the following rules are mandated for all PIL implementations until a federated gateway or service mesh is available.

- **PIL and Adapters as a Single Deployment Unit:** The PIL and the Partner Adapters it needs to orchestrate for a specific domain **must** be deployed in the same cloud, same VPC, and ideally the same cluster. They should be treated as a single, cohesive deployment unit to guarantee low-latency communication during quote fan-outs and other internal calls.
- **Intra-Cloud Orchestration Only:** The PIL is strictly forbidden from performing real-time, synchronous orchestration that calls Partner Adapters across different cloud environments (e.g., a PIL in AWS cannot call an adapter in GCP). All real-time orchestration **must** be contained within a single cloud boundary.
- **Separate Logical Deployments Required:** While they must be colocated, the PIL and its Partner Adapters **must** remain logically separate deployments (e.g., distinct microservices or containers). Bundling adapters as a library (e.g., a JAR file) inside the PIL is considered an anti-pattern, as it creates a monolith and prevents independent deployment, scaling, and maintenance.
- **Handling Multi-Cloud Data Requirements:** For any use case requiring a unified view of data from partners whose adaptors reside in different clouds, the primary principle is to **colocate all adaptors required for real-time orchestration**. Since Partner Adaptors are location-agnostic, they should be deployed in the same cloud environment as the PIL that consumes them.

=====

Lumen API Style Guide

Lumen API Style Guide

- [API Standard: Rest & Resource Design](#)
- [API Versioning Strategy](#)
- [API Standard: POST Method](#)
- [API Standard: JSON Payload](#)
- [API Standard: GET Method](#)
- [API Standard: Batch Operations](#)
- [API Standard: PATCH Method](#)
- [API Standard: Caching & Concurrency with ETag](#)
- [API Standard: Long-Running Operations \(LRO\)](#)
- [API Standard: Bulk Data Transfer \(Handling Massive Payloads\)](#)
- [API Standard: HTTP Headers](#)
- [API Standard: Delete Method](#)
- [API Style Guide Governance Tracker](#)
- [API Standard: Rate Limits & Error Handling Standard](#)
- [OAS Specification Best Practices](#)
- [API URI Standard Proposal](#)
- [API Pagination Standards](#)
- [API Standard: Lumen Problem Details — Minimal Profile \(LPDP-Mini v1.0\)](#)
- [API Standard: Idempotency](#)
- [API Standard: Partial Resource Retrieval](#)
- [API Standard: Standardized Data Types and Formats](#)
- [API Standard: Date & Time Naming](#)
- [API Standard: PUT Method](#)

API Standard: Rest & Resource Design

Purpose

This section defines how APIs at **Lumen** are structured around resources — the fundamental building blocks of RESTful architecture — ensuring consistency, discoverability, and governance automation across all API domains.

REST Principle — Resource-Centric Design

Governance Objective

APIs must be modeled around *resources*, not actions.

A *resource* represents a logical entity (e.g., customer, order, connection) that can be acted upon via standard HTTP methods.

“The key abstraction of information in REST is a resource.”

— Roy Fielding, *REST Dissertation* §5.2

Governance Rules

- Every REST API must identify its **primary resource(s)** before design begins.
- Resources are **nouns**, not verbs.
- CRUD behavior must be represented through **HTTP methods**, not in the URI.

✓ Example

Not Recommended	Recommended
POST /createUser	POST /users
POST /updateUser	PATCH /users/{id}
POST /deleteUser	DELETE /users/{id}

Resource Hierarchies and URI Scope

Governance Objective

Maintain intuitive, predictable, and domain-contained URI structures.

Governance Rules

- URIs must follow:

/ {domain} / v {n} / {resource} / {resource_id} / [subresource] * Each resource belongs to **one business domain**. * **No cross-domain fan-out:**

A resource in one domain cannot directly reference or nest resources from another.

✓ Example

Allowed	Disallowed
/customers/v1/customers/{id}/accounts	/projects/{id}/tasks/{taskId}/comments/{commentId}
/network/v1/connections/{id}	/billing/v1/customers/{id}/network/connections

Collections vs Singleton Resources

Governance Objective

Differentiate between **collections** (plural) and **singletons** (singular) to ensure consistent structure and expectations.

Governance Rules

- **Collections:** Plural noun, contain multiple resources (e.g., /customers).
- **Singletons:** Singular noun, one-per-parent (e.g., /projects/{id}/config).
- **Singleton resources MUST not define:**
 - id property
 - POST or DELETE operations
- **Singletons MUST define:** GET and PATCH.

Implementation Hint

Mark singletons explicitly in OAS:

```
x-singleton: true
```

✓ Example

Type	Path	Allowed Methods
Collection	/customers	GET, POST
Singleton	/projects/{id}/config	GET, PATCH

Resource Naming

Governance Objective

Enable predictable API navigation and uniform discoverability.

Governance Rules

- Use **nouns** for resources; avoid verbs.
- Pluralize collection names (e.g., /users, /invoices).
- Singular for singletons (/me, /company).
- **MUST** use lowercase kebab-case (/service-offers).
- Resource URIs must always be **URL-safe** and **lowercase**.

Actions on Resources (Non-CRUD)

Governance Objective

Support non-CRUD operations that represent *state transitions* or *processes* without breaking REST uniformity.

Governance Rules

- Use verbs **only** when:
- The action cannot be represented by standard HTTP methods.
- It represents a *state transition* or *long-running operation (LRO)*.
- Must be scoped to a resource (not root-level).
- Use **process/job** pattern for asynchronous operations.
- Allowed form:

POST /{resource}/{id}/action

✓ Example

Not Recommended	Recommended
POST /archiveUser	POST /users/{id}/archive
POST /reprovision	POST /connections/{id}/reprovision

If the operation is long-running, annotate:

```
x-lro: true
```

Resource Identifiers (IDs)

Governance Objective

Ensure resource identity is secure, unique, stable, and automation-friendly.

Governance Rules

- **MUST be opaque** — consumers should not infer meaning from IDs.
- **MUST be generated by provider**, never client-supplied.
- **MUST be globally unique** within API space.
- **MUST be URL-safe**.
- **MUST use consistent field naming convention**: <resource>_id.

- **MUST have fixed length** — changing length constitutes a breaking change.
- **MUST NOT** use sequential IDs.
- **MAY** use UUID v4 or Snowflake-style identifiers.
- **MAY** use short type prefixes (e.g., PAY_, ACC_) for troubleshooting.

Reusable Schema

```
ResourceId:
  type: string
  description: >
    Opaque, globally unique identifier for a resource.
    Generated by the API provider. Never sequential or client-supplied.
  pattern: '^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}$'
  example: "279fc665-d04d-4dba-bcad-17c865489dfa"
```

Resource Metadata

Governance Objective

Provide uniform lifecycle visibility (creation, updates) across all resources.

Governance Rules

Every **collection resource** MUST include:

- id
- created_at
- last_updated_at

Singletons exclude these (redundant with parent).

Reusable Schema

```
ResourceMetadata:
  type: object
  required: [id, created_at, last_updated_at]
  properties:
    id:
      $ref: '#/components/schemas/ResourceId'
    created_at:
      type: string
      format: date-time
      description: Creation timestamp (UTC)
    last_updated_at:
      type: string
```

format: date-time
description: Last update timestamp (UTC)

Governance Enforcement Rules

Rule ID	Enforcement Type	Description
path-domain-scope	Error	Paths must begin with a domain (e.g., /billing/, /network/).
path-versioned	Error	Paths must include major version segment (e.g., /v1/).
path-segment-depth	Error	Maximum path depth = 4 segments after version.
singleton-no-post-delete	Error	Singleton resources cannot define POST/DELETE.
id-schema-ref	Error	All *_id fields must use ResourceId schema.
resource-metadata-required	Warn	Collection schemas must include ResourceMetadata.
verb-endpoint-lro	Error	Verbs in path allowed only if x-action: true or x-lro: true. This explicitly permits action endpoints for complex state transitions.

Example Resource Model

```
paths:
  /customers:
    get:
      summary: List all customers
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                properties:
                  data:
                    type: array
                    items:
                      allOf:
                        - $ref: '#/components/schemas/ResourceMetadata'
                        - type: object
                          properties:
                            name:
                              type: string
```

```
example: "Acme Corp"

post:
  summary: Create a new customer
  responses:
    '201':
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Customer'

/customers/{customer_id}/accounts:
  get:
    summary: List accounts for a customer
    parameters:
      - name: customer_id
        in: path
        required: true
        schema:
          $ref: '#/components/schemas/ResourceId'
```

Governance Rationale

Principle	Outcome
Resource-centric modeling	Aligns APIs to REST uniform interface, improving predictability.
Strict URI domain boundaries	Enables per-domain governance, versioning, and automation.
Opaque, stable identifiers	Ensures consistency across distributed systems and prevents information leakage.
Singleton control	Simplifies lifecycle management for dependent configurations.
Action resource governance	Preserves REST semantics while supporting modern async/LRO workflows.

Summary

This REST & Resource section moves Lumen APIs from loosely interpreted REST conventions to a **governable, automatable, and externally predictable API architecture**.

- Consistent structure = easier discoverability and SDK generation.
- Enforced ID and resource schemas = audit-ready data lineage.
- Governed URI scope = scalable domain modeling without overlap.

API Versioning Strategy

Objective

To define the rules for modifying, versioning, and retiring APIs. This standard guarantees stability for consumers while enabling internal agility.

1. Versioning Principles

Our strategy uses Major, Minor, and Patch versions, but they are applied in different places to serve distinct purposes.

A. Major Versions: The Public Contract 📄

The major version represents the public contract with the client and **MUST** only change when a breaking change is introduced.

- **Implementation:** The major version **MUST** be included in the URI path and prefixed with a "v" (e.g., v1, v2).
- **Example:** `https://api.lumen.com/inventory/v1/connections`
- **Client Impact:** A client's integration is tied to this major version. We guarantee that no breaking changes will be made within a given major version.

B. Minor & Patch Versions: The Diagnostic Tool 🔗

Minor and Patch versions track non-breaking, additive changes (Minor) and bug fixes (Patch). They serve as a crucial tool for communication and diagnostics, specifically enabling the "Hotfix" workflow where Production may run v1.1.1 while Test runs v1.2.0.

- **Implementation:** The server **SHOULD** include the full semantic version (major.minor.patch) in an HTTP response header.
- **Example:** `API-Version: 1.2.1`
- **Client Impact:** This has zero impact on the client's code. Clients are not required to send or parse this header. Its purpose is to provide a precise reference point for debugging deployment lag in distributed systems.

C. Implementation Requirement (No Hardcoding)

To prevent drift between the documentation and the runtime behavior, developers **MUST NOT** hardcode the version number in source code.

- **The Source of Truth:** The value returned in the API-Version HTTP header **MUST** be derived directly from the `info.version` field in the `openapi.yaml` file (extracted during the build process).

2. Defining Breaking vs. Non-Breaking Changes

To ensure clarity, the following examples define what constitutes a breaking change versus a non-breaking change. When in doubt, a change SHOULD be treated as breaking.

● Breaking Changes (Requires New Major Version)

Modifications that can potentially break an existing client integration.

- **Request Changes:**

- Removing or renaming a parameter.
- Adding a new *required* parameter.
- Making a previously optional parameter required.
- Changing the data type or adding stricter validation rules.
- Changing authentication/authorization requirements.

- **Response Changes:**

- Removing or renaming a field in the response body.
- Making a previously required response field optional.
- Changing the data type or structure (e.g., nesting an existing field).
- Adding a new value to an enum (Breaks strict client parsers).
- Changing the primary HTTP Status Code.

● Non-Breaking Changes (Minor & Patch Increments)

Modifications that are safe to deploy without updating the URI.

- **Minor Version (Additive Features):**

- **Request:** Adding a new *optional* parameter or header.
- **Response:** Adding a new field, endpoint, or header.

- **Patch Version (Bug Fixes):**

- Internal logic corrections that do not alter the schema.
- Performance optimizations.
- Documentation typos or description updates.
- Security patches that do not alter the interface contract.

3. Governance Enforcement (Automated Gates)

Policies are enforced via the CI/CD pipeline to prevent accidental breakage and ensure documentation fidelity.

Breaking Change Detection (The Guard)

Every Pull Request to an active version (e.g., v1) runs an automated diff tool (e.g., openapi-diff) against the main branch.

- **Pass:** Change is additive (Minor/Patch).
- **Fail:** Change is breaking. The pipeline blocks the merge and instructs the developer to create a new Major Version (v2) directory.

Deprecation Assurance (The 3-Layer Check)

To ensure deprecated endpoints are correctly signaled to consumers, we utilize a "Define, Verify, Enforce" strategy:

1. **Lint (Design Quality):** A Spectral rule checks that any operation marked `deprecated: true` explicitly defines the `Deprecation` and `Sunset` headers in its response schema. This ensures the generated documentation is complete.
2. **Gateway (Runtime Enforcement):** We utilize a Governance-as-Code pattern to remove human error.
3. *Detection:* During the deployment build (Stage 5), the pipeline parses the `openapi.yaml` to identify operations where `deprecated: true`.
4. *Injection:* The pipeline automatically generates and injects an Apigee AssignMessage Policy into the Proxy Flow for those specific paths.
5. *Result:* The `Deprecation` and `Sunset` headers are injected into the response at the edge. The developer does not need to modify backend code.
6. **Contract Test (Verify):** The automated regression suite verifies that calling a deprecated endpoint in the Sandbox returns the required headers. If headers are missing, the release is blocked.

4. API Lifecycle Management

Announcing Major Changes (Pre-Release)

To provide consumers time to budget for migrations, breaking changes **SHOULD** be announced at least 6 months before a new major version is released.

- **Channel:** Official changelog, documentation, and direct email to affected consumers.
- **Content:** Must detail the changes, the reasoning, and a preliminary migration guide.

Active Version Support

All API products **MUST** support at least two active major versions during a migration period.

- **The 24-Month Rule:** When a new version (v2) is released, the previous version (v1) **MUST** be supported for at least **24 more months**.
- **Benefit:** This overlap allows clients to migrate at their own pace without service disruption.

5. Deprecating Endpoints and Features (Partial Deprecation)

When a specific endpoint or field is being removed while the rest of the API remains active.

Phase 1: Announce 🚨

The endpoint remains fully functional.

- **OpenAPI:** Mark the specific operation or field with `deprecated: true`.
- *Result:* The Developer Portal renders a visual strikethrough or warning badge.
- **HTTP Headers (RFC 8594):** The server (via Gateway Injection) SHOULD return headers on responses from that specific endpoint:
- `Deprecation:` Wed, 22 Oct 2025 13:30:00 GMT
- `Sunset:` Fri, 22 Oct 2027 13:30:00 GMT (The removal date).

Phase 2: Guide 🚀

Provide a clear path forward.

- **Link Header:** The server SHOULD return a link to the replacement:
- `Link: <.../new-endpoint>; rel="alternate"`

Phase 3: Remove ✖

After the support window, the endpoint is removed.

- **Constraint:** The deprecated endpoint MUST be removed in the next major version (v2), but stays in v1 until v1 is fully retired.

6. Sunsetting a Major Version (Total Deprecation)

When the entire Major Version (e.g., v1) is entering its end-of-life phase (usually after v2 is released).

- **Trigger:** The 24-month support clock has started for v1.
- **Action:**
- **Portal Metadata:** Now you MUST update `api-metadata.yaml` to signal the entire service is dying.

`lifecycleStage: Deprecated` `sunsetDate: "2027-10-22"` 2. **Result:** The Portal displays a global "Deprecation Warning" banner across the entire API documentation. 3. **Global Headers:** The Gateway is configured to inject Deprecation/Sunset headers on every endpoint in v1. 4. **Communication:** Consumers are notified via multiple channels.

* **Final Response:** Once decommissioned (T-0), endpoints SHOULD return 410 Gone to indicate the resource is permanently unavailable.

7. Special Handling

Experimental (Beta) Features

Beta features are not bound by the standard 24-month support policy.

- **Implementation:** Identified via URI (e.g., /v1-beta/) or Version Header (2.1.0-beta).
- **Governance:** Can be changed or removed without the standard deprecation process.

Updating Shared Data Models

Changes to common objects (e.g., Address) across APIs MUST use the "Expand and Contract" pattern:

1. **Expand (v1.2):** Add new fields as optional; mark old fields deprecated.
2. **Transition:** Monitor usage over 24 months.
3. **Contract (v2.0):** Remove deprecated fields in the next major version.

Emergency Security Exceptions

In the event of a Critical Security Vulnerability (CVSS > 9.0) where backward compatibility poses a severe risk:

- **Exception:** The "6-Month Notice" and "24-Month Support" requirements may be waived.
- **Approval:** Requires explicit written approval from the CISO and API Governance Chair.

Financial Accountability

Creating a new Major Version incurs a "Maintenance Tax." The Product Team is financially responsible for hosting infrastructure for both v1 and v2 during the mandatory 24-month overlap. Teams cannot "abandon" v1 early to save costs without a formal Exception.

API Standard: POST Method

Purpose

The POST method creates new resources or performs non-CRUD actions on existing resources.

It is **not inherently idempotent** and should be used when neither PUT nor PATCH semantics apply.

This page defines the required and recommended practices for all POST operations across Lumen APIs, ensuring consistency, predictability, and safe retries through **idempotency keys** and **standardized responses**.

Semantics

Property	Requirement
Safe	✗ No, may modify server state.
Idempotent	⚠ No by default; MUST use Idempotency-Key if retries are expected.
Typical Use	Resource creation or triggering an action.
Request Body	✓ Required, defines input parameters or resource representation.
Response Body	✓ Required, returns the created resource or operation result.

When to Use POST

Use Case	Example	Required Behavior
Create a new resource	POST /v1/orders	Returns 201 Created + Location header for the new resource.
Create multiple resources (batch / composite operation)	POST /v1/customers/batch	Performs creation of multiple items in one request. Returns 207 Multi-Status if results are mixed, or 202 Accepted if processed asynchronously via a job. Batch requests SHOULD complete within the synchronous timeout budget (≤ 30 s p95 / 60 s absolute) or follow the LRO pattern .
Trigger an action on a resource	POST /v1/invoices/{id}/void	Returns 200 OK or 202 Accepted depending on sync vs. async.
Start an asynchronous or workflow operation	POST /v1/network/connections/provision	Returns 202 Accepted + Location header for status tracking.
Perform search or query with complex filters (non-CRUD)	POST /v1/orders/search	Allowed when query parameters are too complex for GET; must be safe and side-effect free.
Submit sensitive criteria or large query payloads (PII, account numbers, tokens, advanced filters)	POST /v1/customers/search with body {"email":"...", "dob":"..."}	Use POST with a request body instead of putting sensitive data in the URL. MUST be side-effect free. MUST set Cache-Control:

Use Case	Example	Required Behavior
		no-store. MUST NOT log or echo sensitive fields. MUST NOT include PII or secrets in the path or query string. SHOULD return 413 Payload Too Large for oversized bodies and document limits. SHOULD include Vary: Authorization for authenticated results.

Non-Applicable Cases

Rule	Description
MUST NOT	Use POST for singleton resources (e.g., /profile, /configuration, /status). These resources are pre-defined by the system and not client-created. Use GET to retrieve and PATCH to update them.

Why the Sensitive-Data Rule Exists

- URLs appear in logs, browser history, and referrers; request bodies do not.
 - Shared or transparent caches may store GET responses; using POST + Cache-Control: no-store minimizes risk.
 - Request bodies allow schema-driven validation and redaction (x-lumen-sensitive: true in OAS).
-

Creation Semantics

Resource Creation

- **MUST** create new resource(s) under the collection URI.
Example:
POST /v1/customers → creates a customer resource.
- **MUST** return:
- 201 Created
- A Location header with the canonical URI of the new resource.
- The newly created resource in the response body.
- **MUST NOT** use POST to **replace** an existing resource; use PUT instead.
- **MAY** return 202 Accepted for **asynchronous creation**
- **MAY** return 207 Multi-Status if a **batch request** partially succeeds.

Example:

```
POST /v1/customers
Content-Type: application/json
{
  "name": "Jane Doe",
  "email": "jane@example.com"
}

HTTP/1.1 201 Created
Location: /v1/customers/c123
Content-Type: application/json
{
  "id": "c123",
  "name": "Jane Doe",
  "email": "jane@example.com"
}
```

Action Semantics

When performing an action on an existing resource that does not map to a CRUD operation:

Rule	Description
Pattern	POST <code>/resource/{id}/{action}</code> (e.g., <code>/invoices/{id}/approve</code>).
Response	200 OK for synchronous result, 202 Accepted for async.
Body	MAY contain parameters relevant to the action.
Idempotency	MUST support Idempotency-Key to handle retries safely.

Example:

```
POST /v1/invoices/123/void
Idempotency-Key: 4af7e7d9-91a6-4823-a1c0-8d112c884cb5
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": "123",
  "status": "voided"
}
```

Idempotency

POST is **not idempotent by default** — but Lumen APIs must provide an idempotent experience when requests are retried.

Header	Description
Idempotency-Key	A client-supplied opaque UUID identifying the request.
Scope	Applies to unsafe POSTs that could be retried.
Server Behavior	Deduplicate requests with the same key and identical payload; replay original result.
Retention	Store key and result for a minimum of 24 hours.
Conflict Handling	Return 409 Conflict if the same key is reused with a different payload.

Example:

```
POST /v1/payments
Idempotency-Key: 827dcf3e-44fb-4f07-94b3-6b47cf3b813d

→ 201 Created

# Retry (same key)
→ 201 Created (identical response)

# Retry (same key, different payload)
→ 409 Conflict
```

Headers

Header	Usage
Idempotency-Key	Ensures retry-safe behavior.
Location	Points to newly created resource or operation resource.
Retry-After	Used when returning 202 Accepted for async processing.
Content-Type	MUST be application/json or vendor subtype.
Accept	MUST be application/json (or compatible).

Status Codes

Code	Meaning	Applies To	Notes
201 Created	Resource successfully created	 Resource creation	MUST include Location header and representation.

Code	Meaning	Applies To	Notes
202 Accepted	Request accepted for asynchronous processing	✓ LRO / async	MUST include operation URL in Location.
207 Multi-Status	Batch/composite request produced mixed results	✓ Bulk / composite POSTs	Use when some items succeed and others fail, body MUST include per-item statuses and summary.
200 OK	Successful synchronous action	✓ Resource actions	Used only for non-creation actions.
400 Bad Request	Invalid request body or parameter	✓ Both	Syntax or schema violation.
401 Unauthorized	Authentication required	✓ Both	
403 Forbidden	Authenticated but lacks permission	✓ Both	
404 Not Found	Target resource or parent collection missing	✓ Both	
405 Method Not Allowed	Wrong HTTP method used	✓ Both	
409 Conflict	Resource already exists or idempotency key conflict	✓ Creation / Action	
422 Unprocessable Content	Semantic validation error	✓ Both	Schema valid but business rule failure.
500 Internal Server Error	Unexpected server failure	✓ Both	No internal details exposed.

OAS Authoring Requirements

When documenting POST operations in OpenAPI:

```

paths:
  /v1/orders:
    post:
      summary: Create an order
      parameters:
        - in: header
          name: Idempotency-Key
          schema: { type: string, maxLength: 255 }
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/OrderCreate' }
      responses:

```

```
'201':  
  description: Created  
  headers:  
    Location:  
      schema: { type: string, format: uri }  
'202': { description: Accepted (async creation) }  
'207': { description: Partial success (batch results) }  
'400': { description: Invalid request }  
'409': { description: Conflict (duplicate or idempotency) }  
'422': { description: Validation failed }
```

MUST

Include Idempotency-Key header definition.

Include Location header for 201 or 202 responses.

Constrain response codes to the approved matrix.

Document LRO patterns (202 + Location) when applicable.

Reference 207 for batch/composite POSTs if supported.

Governance Validation Checklist

Check	Expectation
✓ Correct verb usage	POST used only for create or action semantics
✓ Location header present	For 201 and 202 responses
✓ Idempotency supported	Idempotency-Key required for unsafe POSTs
✓ Proper status codes	Matches approved matrix
✓ Async workflows standardized	202 + Location pattern used
✓ OAS validated via Spectral	Governance enforcement ready

Developer Summary

- Use POST to **create** new resources or **trigger** actions — never for updates.
- Always return 201 Created with a Location header for creations.
- Use Idempotency-Key to protect clients from duplicate processing.
- For asynchronous operations, return 202 Accepted and an **operation URI**.
- For partial batch success, return 207 Multi-Status with results[] and summary.
- Adhere strictly to the **approved status-code matrix**.
- Document every POST in OpenAPI with consistent headers and response structure.

API Standard: JSON Payload

Overview

All Lumen APIs **must use JSON** as the canonical message format for request and response bodies.

From a governance standpoint, this ensures predictable interoperability across platforms, easier validation using OpenAPI 3.0.3, and forward-compatible payload design.

To achieve this, all JSON payloads must follow the structural, naming, and encoding rules defined in [RFC 7159 \(JSON\)](#) and [RFC 7493 \(I-JSON\)](#).

The goal is to enforce **clarity, safety, and uniformity** across all APIs regardless of team or domain.

Governance Principles

Governance Objective	Policy Requirement	Rationale
<i>Standardized Format</i>	All request and response bodies MUST be valid JSON objects.	Guarantees structured, parsable data with clear schema evolution paths.
<i>I-JSON Compliance</i>	JSON payloads MUST conform to the I-JSON profile (RFC 7493).	Ensures consistent behavior across clients, SDKs, and backend systems.
<i>Encoding</i>	All JSON content MUST use UTF-8 encoding.	Avoids encoding ambiguity and corruption across gateways.
<i>Uniqueness of Keys</i>	Each JSON object MUST contain unique property names .	Prevents parser ambiguity and security issues (duplicate key override).
<i>Content Type</i>	Media type MUST be declared as <code>application/json</code> .	Enables correct content negotiation and API discovery.
<i>Extensibility</i>	Payloads MUST support additive evolution (new fields tolerated).	Allows forward-compatible releases without breaking clients.

Structural Rules

Nested Structures

SHOULD prefer nested, well-scoped objects over flattened key sets. This improves logical grouping, maintainability, and future extensibility.



Recommended

```
{
  "customer_id": "C123",
```

```
"billing_address": {
  "line1": "100 Main St",
  "city": "Denver",
  "postal_code": "80202"
}
```

✗ Not Recommended

```
{
  "customer_id": "C123",
  "billing_address_line1": "100 Main St",
  "billing_address_city": "Denver",
  "billing_address_postal_code": "80202"
}
```

Field Naming Convention

Field names **MUST** follow **lower snake_case**, matching regex:

```
^[a-z][a-z0-9_]*$
```

Rules:

- First character must be lowercase.
- Use underscores to separate words.
- No hyphens, camelCase, or PascalCase.
- ASCII-only.

✓ **Recommended** → customer_name, invoice_number, created_at

✗ **Not Recommended** → salesOrderId, CustomerName, sales-order-id

Array Naming Convention

Array properties **MUST** use plural names to signal multiplicity.

Empty arrays must be represented as [], *not* null.

✓ **Recommended**

```
{
  "line_items": []
}
```

✗ **Not Recommended**

```
{
  "line_items": null
}
```

I-JSON Compliance Checklist

Category	Requirement	Description
Encoding	UTF-8	Required for all JSON payloads.
Duplicate Keys	Forbidden	Parsers must reject payloads where a key appears more than once in the same object.
Numbers	Finite, safe range only	No NaN, Infinity, or numbers $> 2^{53} - 1$.
Strings	Valid Unicode scalars only	No control or invalid Unicode characters.
Top-Level Type	Object only	Arrays, primitives, or raw strings as top-level payloads are not permitted.
Date/Time	RFC 3339 UTC format	Example: "2025-11-11T18:30:00Z"

Example — Valid Payload

```
{
  "order_id": "ORD_9876",
  "customer": {
    "customer_id": "C12345",
    "name": "Acme Corp"
  },
  "line_items": [
    {
      "item_id": "I567",
      "quantity": 3
    }
  ],
  "total_amount": 1500,
  "currency": "USD",
  "created_at": "2025-11-11T18:30:00Z"
}
```

Example — Invalid Payloads

Duplicate Keys

```
{
  "name": "Acme",
  "name": "Acme Inc"
}
```

Null Array

```
{
  "line_items": null
}
```

Unsafe Number

```
{
  "amount": 9999999999999999
}
```

Governance Enforcement

- **Spectral Linting Rules:**
Validate UTF-8 compliance, key uniqueness, and snake_case fields.
Disallow null arrays and duplicate keys.
 - **Gateway Validation:**
Reject invalid or non-I-JSON payloads at request-time.
 - **Schema Linting in CI:**
Automated pipelines must validate sample payloads and OpenAPI schemas for I-JSON compliance.
 - **Documentation Consistency:**
All examples in the Developer Portal must use valid, lint-clean JSON payloads.
-

Summary

JSON payload hygiene directly determines API quality and interoperability. Lumen APIs **must** adhere to I-JSON constraints, snake_case naming, predictable array behavior, and consistent UTF-8 encoding. This ensures that payloads are **machine-safe, forward-compatible, and consistent** across every domain and consumer.

API Standard: GET Method

Purpose

The GET method retrieves a resource or a collection of resources. It **MUST NOT** change server state and **MUST** be safe and idempotent as defined in [RFC](#)

This page defines how to design, document, and implement GET operations across Lumen APIs to ensure predictable, cache-friendly, and standards-compliant behavior.

Semantics

Property	Requirement
Safe	✓ YES, never modifies data or triggers side effects.
Idempotent	✓ YES, repeating the same GET yields the same result (unless data has changed).
Request Body	✗ NOT ALLOWED.
Response Body	✓ REQUIRED for 200 OK, MUST represent the requested resource(s).
Caching	✓ Strongly RECOMMENDED, use ETag and Cache-Control.

URI Design

Examples

- Single resource: `/v1/customers/{customer_id}`
- Collection: `/v1/customers`
- Filtered collection: `/v1/customers?status=active&country=US`

MUST

- Use plural nouns for collections.
 - Avoid action verbs (`/getCustomer`) — the HTTP verb already defines intent.
 - Preserve canonical noun hierarchy (**no verbs, no RPC style**).
-

Singleton Resources

Definition

A *singleton resource* represents a unique conceptual instance of a type that exists once per context (e.g., current user profile, organization settings, service status).

Singletons do not require an identifier or belong to a collection.

Examples

```
GET /v1/profile           → current user profile
GET /v1/organization/settings → organization configuration
```


GET /v1/configuration	→ global configuration
GET /v1/status	→ system status or health

Design Guidance

Rule	Description
MUST	Treat singletons as first-class nouns (/profile, /settings, /status).
MUST NOT	Nest singleton resources under collections if they represent contextual or global concepts (e.g., avoid /users/{id}/profile for the authenticated user's profile). Such resources have a single instance determined by the caller's identity or environment — not by path variables — and therefore belong at the root level (e.g., /profile).
MAY	Nest singleton resources only when they are scoped to a parent resource (e.g., /organizations/{orgId}/settings — one settings resource per organization).
MUST	Support GET for retrieval and PATCH for updates (if mutable). Never use POST or DELETE.
MUST	Return 200 OK with a body when available; 404 Not Found if temporarily unavailable.
SHOULD	Include ETag and support If-None-Match for cache validation.
MUST NOT	Return 204 for existing singletons — always return a representation unless explicitly empty.

Example

```
GET /v1/profile
If-None-Match: "v12"

HTTP/1.1 200 OK
ETag: "v13"
Cache-Control: private, max-age=60
{
  "id": "u123",
  "name": "Jane Doe",
  "email": "jane@example.com"
}
```

```
GET /v1/profile
If-None-Match: "v13"

HTTP/1.1 304 Not Modified
ETag: "v13"
```

Governance Note

Singletons must be documented as distinct resources (/profile, /settings, /status) and enforced through Spectral rules to verify ETag and If-None-Match support.

Response Structure

Every successful GET MUST return a JSON object.

Single Resource

```
{
  "id": "c123",
  "name": "Jane Doe",
  "email": "jane@example.com",
  "created_at": "2025-10-20T12:34:56Z"
}
```

Collection

```
{
  "data": [
    { "id": "c123", "name": "Jane Doe" },
    { "id": "c124", "name": "John Smith" }
  ],
  "pagination": {
    "limit": 20,
    "offset": 0,
    "total": 245
  }
}
```

MUST

- Wrap collections in a data array to allow metadata (pagination, links) at the top level.
 - Use consistent pagination keys (limit, offset, total).
-

Headers and Conditional Requests

ETag and Cache Validation

Header	Usage	Example
ETag	Server-generated version tag for the resource	ETag: "v7a2"
If-None-Match	Client cache validation header	If-None-Match: "v7a2"

Header	Usage	Example
Response on match	304 Not Modified (no body)	

MUST

- Return ETag on all cacheable responses.
- Support If-None-Match on all GET endpoints.
- Return 304 Not Modified if ETag matches.

SHOULD

- Add Cache-Control (e.g., private, max-age=60) to define freshness.
- Include Last-Modified when available for weak validation.

Query Parameters and Filtering

Principle	Guidance
Predictability	Parameter names must be consistent across APIs (status, sort, limit, offset).
Multiple Filters	Use & separated query pairs.
Sorting	?sort=+name ascending, ?sort=-created_at descending.
Pagination	Refer to pagination standards.
Empty Results	Return 200 OK with an empty data array, not 404.
Invalid Filters	Return 400 Bad Request with Problem Details.

Status Codes

Code	Meaning	Applies To	Usage Notes
200 OK	Successful retrieval	✔ Single / ✔ Collection	Always return for successful GETs; include response body.
304 Not Modified	Resource unchanged since client cache	✔ Single / ✔ Collection	Used with If-None-Match, no body.
400 Bad Request	Invalid query or parameter	✔ Single / ✔ Collection	Syntactic or invalid filter errors.
401 Unauthorized	Authentication required or invalid	✔ Both	
403 Forbidden	Authorized but lacks permission	✔ Both	
404 Not Found	Resource not found	✔ Single	Never use for empty collections.

Code	Meaning	Applies To	Usage Notes
405 Method Not Allowed	HTTP verb not supported	✔ Both	
422 Unprocessable Content	Valid syntax but invalid semantic input	✔ Both	
500 Internal Server Error	Unexpected server failure	✔ Both	No stack traces exposed.

Example Request / Response Set

Request

```
GET /v1/customers/123
If-None-Match: "v5"
Accept: application/json
```

Response (Resource Updated)

```
HTTP/1.1 200 OK
Content-Type: application/json
ETag: "v6"
Cache-Control: private, max-age=60
{
  "id": "123",
  "name": "Jane Doe",
  "email": "jane@example.com"
}
```

Response (Not Modified)

```
HTTP/1.1 304 Not Modified
ETag: "v6"
```

OAS Authoring Requirements

```
paths:
  /v1/customers/{id}:
    get:
      summary: Retrieve customer by ID
      parameters:
        - in: path
          name: id
          required: true
          schema: { type: string }
        - in: header
          name: If-None-Match
          schema: { type: string }
```

```
responses:
  '200':
    description: Successful response
    headers:
      ETag:
        description: Entity tag for versioning
        schema: { type: string }
  '304':
    description: Not Modified
  '400': { description: Invalid request }
  '404': { description: Not found }
```

MUST

- Include ETag in 200 response headers.
 - Include If-None-Match parameter.
 - Restrict status codes to the approved matrix.
 - Return application/json.
-

Governance Checklist

Check	Validation
✓ GET defined as safe and idempotent	No side effects
✓ ETag header returned	Required for cacheable responses
✓ If-None-Match supported	Enables 304 Not Modified
✓ No request body allowed	Specification enforces it
✓ Proper status codes used	200/304/400/401/403/404/405/422/500 only
✓ OAS validated via Spectral rules	Governance automation ready

Developer Summary

- Use GET exclusively for read-only operations.
- Never require or process a body in GET.
- Always include ETag and handle If-None-Match for conditional caching.
- Return 200 OK for success, 304 for cache hits, 404 only for missing single resources.
- Treat collection queries as successful even when empty.
- Keep response structure predictable (data, pagination).
- Follow the approved status-code matrix and Problem Details format.

API Standard: Batch Operations

1. Purpose

Batch operations enable clients to perform **the same action on multiple resources** in a single request (e.g., create, update, or trigger actions across many items).

This guide defines **Lumen's governance standards** for batch APIs—latency thresholds, idempotency, status codes, response shape, and OpenAPI documentation.

Batch APIs must behave **predictably**, support **safe retries**, and meet **bounded synchronous latency**. If those guarantees can't be met, use the **Long-Running Operation (LRO)** pattern.

2. Semantics

Property	Requirement
Verb	POST
Safe	✗ No — modifies server state
Idempotent	⚠ Recommended for synchronous; required for asynchronous
Response Type	application/json
Behavior	Processes multiple items of the same resource type atomically per item (not per batch)

3. When to Use Batch Operations

Use Case	Example	Behavior
Batch Create or Update	POST /v1/customers/batch	201 if all succeed, 207 if mixed results, 202 if deferred.
Batch Action	POST /v1/invoices/batch/void	200/207/202 depending on outcome.
Large Imports / Heavy Processing	POST /v1/orders/import	202 Accepted + Location to a job; follow the LRO guide.

4. Execution Policy

4.1 Synchronous Mode

A batch **may** execute synchronously when:

- **Predictable latency:** $\leq 30\text{ s p95} / 60\text{ s absolute}$

- **Deterministic outcome:** same input → same observable result (conflicts included)
- **Retry safety (recommended):** supports Idempotency-Key for deduping retries

4.2 Asynchronous Mode

If synchronous boundaries can't be guaranteed:

- Return **202 Accepted**
- Include Location to a job resource
- Follow the **LRO Style Guide** for polling, job states, lifecycle

5. Status Codes

Code	Meaning	Applies To	Notes
201 Created	All items created	Batch create	May return representation or summary
200 OK	All items succeeded (non-create)	Batch action	Uniform success
207 Multi-Status	Mixed outcomes	Create/action	Return per-item results + summary
202 Accepted	Deferred job	Any	Location → job URI
4xx/5xx	Uniform failure	Any	Use RFC 7807 Problem Details

Rules: Use **207** only when per-item outcomes differ. Use **201/200** for uniform success. Use **202** when work is deferred.

6. Response Structure

6.1 207 Multi-Status (Partial Success)

```
HTTP/1.1 207 Multi-Status
Content-Type: application/json
{
  "results": [
    { "id": "c101", "status": 201, "message": "Created" },
    { "id": "c102", "status": 409, "message": "Duplicate customer" }
  ],
  "summary": { "total": 2, "succeeded": 1, "failed": 1 }
}
```

Rules

- **MUST** include summary (total/succeeded/failed)
 - **MUST** use numeric HTTP status codes (100–599) for each item's status field
 - **MAY** include message or failure details
 - **MAY** offer **compact** (failures-only) vs **full** modes
 - **MUST NOT** truncate without pagination metadata
 - **MAY** expose a **results** endpoint (optional, use-case dependent)
-

7. 202 Accepted (Asynchronous Jobs)

Submit

```
POST /v1/customers/batch
→ 202 Accepted
Location: /v1/jobs/9827
Retry-After: 10
```

Poll

```
GET /v1/jobs/9827
→ 200 OK
{
  "id": "9827",
  "state": "in_progress",
  "accepted": 1000,
  "processed": 300,
  "succeeded": 295,
  "failed": 5,
  "links": {
    "results": "/v1/jobs/9827/results"
  }
}
```

Rules

- **MUST** include Location
 - **SHOULD** include Retry-After
 - **MUST** dedupe via Idempotency-Key (same key + payload → same job)
 - **MUST NOT** embed resource URLs or per-item data in Job
 - See **LRO Style Guide** for states & polling
-

8. Headers

Header	Usage
Idempotency-Key	Required for all unsafe batch POSTs; dedupes retries

Header	Usage
Location	Required for 201/202
Retry-After	Recommended for 202/throttling
Cache-Control	no-store for sensitive/transient results
Vary	Include Authorization when results depend on user context

9. Idempotency

Requirement	Description
MUST	Deduplicate by (method, canonical path, Idempotency-Key, request hash)
MUST	Return identical outcome on same key + payload
MUST	Retain key + response metadata $\geq$ 24h (72h recommended for async)
MUST	409 Conflict if key reused with different payload
RECOMMENDED	Retry-After on transient failures

Async specifics:

- Same key+payload while job queued/in-progress → **return same job** (don't create a new one)
- After completion → **replay** stored outcome / job status

10. OpenAPI Authoring (Using Reusable Schemas)

10.1 Batch Endpoint (sync/async)

```

paths:
  /v1/customers/batch:
    post:
      summary: Create multiple customers
      parameters:
        - in: header
          name: Idempotency-Key
          required: true
          schema: { type: string, maxLength: 255 }
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: array

```

```

        items: { $ref: '#/components/schemas/CustomerCreate' }
responses:
  '201': { description: All created successfully }
  '200': { description: All items succeeded (non-creation action)' }
  '207':
    description: Partial success (mixed outcomes)
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/BatchMultiStatusResponseCompact'
  '202':
    description: Accepted for asynchronous processing
    headers:
      Location:
        schema: { type: string, format: uri }
      Retry-After:
        schema: { type: integer, minimum: 1 }
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/JobStatus'
  '400': { description: Invalid input or schema error }
  '409': { description: Conflict or idempotency-key mismatch }

```

If you also support **full** results inline for small batches, offer an alternate 207 response using `BatchMultiStatusResponseFull`.

10.2 Job Polling

```

paths:
  /v1/jobs/{jobId}:
    get:
      summary: Get job status
      parameters:
        - in: path
          name: jobId
          required: true
          schema: { type: string }
      responses:
        '200':
          description: Job status
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/JobStatus'

```

10.3 (Optional) Exported Full Results

```

paths:
  /v1/exports/{exportId}:
    get:

```

```
summary: Download exported batch results
parameters:
  - in: path
    name: exportId
    required: true
    schema: { type: string }
responses:
  '200':
    description: Export ready
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ExportManifest'
  '202': { description: Still generating; retry later }
  '410': { description: Export expired }
```

11. Reusable Schemas (Components)

```
components:
  schemas:

    BatchSummary:
      type: object
      description: Counts for a batch operation
      required: [total, succeeded, failed]
      properties:
        total: { type: integer, minimum: 0 }
        succeeded: { type: integer, minimum: 0 }
        failed: { type: integer, minimum: 0 }

    BatchFailure:
      type: object
      description: Per-item failure record (used in compact and full)
      required: [index, status, code, message]
      properties:
        index:
          type: integer
          minimum: 0
          description: Zero-based position of the item in the submitted array
        id:
          type: string
          nullable: true
          description: Server-assigned or client key if available
        status:
          type: integer
          description: HTTP status for this item (e.g., 409, 412, 422)
        code:
          type: string
          description: Stable application error code (string, not numeric)
        message:
          type: string
```

```

    description: Human-readable summary (safe for logs)
    details:
      $ref: '#/components/schemas/ProblemDetails'

BatchSuccess:
  type: object
  description: Per-item success record (used only in 'full' view)
  required: [index, status]
  properties:
    index:
      type: integer
      minimum: 0
    id:
      type: string
      nullable: true
      description: Created/affected resource identifier if available
    status:
      type: integer
      description: HTTP status for this item (e.g., 200, 201)
    resource:
      type: object
      additionalProperties: true
      description: Optional representation of the created/updated resource (domain-s

BatchMultiStatusResponseCompact:
  type: object
  description: Default compact batch response (summary + failures only)
  required: [summary]
  properties:
    summary:
      $ref: '#/components/schemas/BatchSummary'
    failures:
      type: array
      description: Failed items only (paged if large)
      items: { $ref: '#/components/schemas/BatchFailure' }
    page:
      $ref: '#/components/schemas/PageMeta' # refer to pagination style guide

BatchMultiStatusResponseFull:
  type: object
  description: Full batch response (summary + successes + failures). Use only for sm
  required: [summary]
  properties:
    summary:
      $ref: '#/components/schemas/BatchSummary'
    successes:
      type: array
      description: Successful items (optional and typically limited)
      items: { $ref: '#/components/schemas/BatchSuccess' }
    failures:
      type: array
      description: Failed items (may be paged)
      items: { $ref: '#/components/schemas/BatchFailure' }
    page:

```

\$ref: '#/components/schemas/PageMeta' # refer to pagination style guide

JobStatus:

type: object
description: Operational status for an asynchronous batch job (no per-item data)
required: [id, state, submitted_at, progress, links]
properties:
 id: { type: string }
 type: { type: string, enum: [batch] }
 state: { type: string, enum: [queued, in_progress, completed, failed, canceled] }
 submitted_at: { type: string, format: date-time }
 progress:
 type: object
 required: [total, processed, succeeded, failed]
 properties:
 total: { type: integer, minimum: 0 }
 processed: { type: integer, minimum: 0 }
 succeeded: { type: integer, minimum: 0 }
 failed: { type: integer, minimum: 0 }
 links:
 type: object
 required: [self]
 properties:
 self:
 type: string
 format: uri
 description: Canonical job URI for polling
 results:
 type: string
 format: uri
 nullable: true
 description: Optional link to compact or exported results (no per-item pay
idempotency_key:
 type: string
 description: Echo of the submission key to support replay semantics
request_hash:
 type: string
 description: Hash/fingerprint of original request body for conflict detection
error:
 \$ref: '#/components/schemas/ProblemDetails' # refer to LPDP

ExportManifest:

type: object
description: Descriptor for large exported batch results
required: [id, state]
properties:
 id: { type: string }
 state: { type: string, enum: [pending, generating, completed, failed, expired] }
 file_type: { type: string, enum: [application/jsonl, application/json, text/csv] }
 size_bytes: { type: integer, minimum: 0 }
 expires_at: { type: string, format: date-time }
 download_url: { type: string, format: uri }

```
error:
  $ref: '#/components/schemas/ProblemDetails'
```

12. Governance Validation Checklist

-  Latency boundaries: ≤ 30 s p95 / ≤ 60 s absolute (else LRO)
-  Status codes: 200 / 201 / 207 / 202 only
-  207 response uses **BatchMultiStatusResponseCompact** (or **...Full** when allowed)
-  Idempotency: Idempotency-Key required (async); recommended (sync)
-  OAS coverage: 201 + 207 + 202 documented; job uses **JobStatus** schema
-  Spectral rules: enforce headers, responses, and schemas

API Standard: PATCH Method

Purpose

The PATCH method partially updates an existing resource at its canonical URI.

Unlike PUT, it does not require the full representation, only the fields being changed.

PATCH is **not inherently idempotent**, but can be made so when combined with If-Match and deterministic patch semantics.

This page defines Lumen's governance requirements and best practices for designing, documenting, and implementing PATCH operations across APIs.

Semantics

Property	Requirement
Safe	 No, modifies server state.
Idempotent	 Not guaranteed, SHOULD use If-Match for concurrency control.
Typical Use	Partial modification of resource fields.
Request Body	 Required, describes changes to apply.
Response Body	 Required for 200 OK, returns updated representation.

When to Use PATCH

Use Case	Example	Required Behavior
Modify selected fields	PATCH /v1/customers/{id}	Updates only given attributes; returns 200 OK with updated object.
Change workflow state	PATCH /v1/orders/{id}	Updates only status or stage.
Partial configuration change	PATCH /v1/configurations/{id}	Applies incremental changes.
Partial update of singleton	PATCH /v1/organization/settings	Updates subset of fields in a singleton resource.

Governance Note — When PATCH Is Business-Critical

The PATCH method **MUST NOT** be used merely for developer convenience.

Teams **SHOULD** use PATCH **only when partial updates are business-critical**

— meaning that sending or overwriting the entire resource representation (via PUT) would introduce data-loss, concurrency, or operational issues.

PATCH is business-critical when:

- Multiple systems or users own different parts of a resource (e.g., shared ownership of a profile).
- Partial mutation prevents overwriting external or read-only fields.
- Resource size or update frequency makes PUT inefficient.
- Incremental updates are required for workflows, IoT configuration, or multi-step state transitions.

PATCH is *not* business-critical when:

- The resource is small and easily replaced with PUT.
- The update semantics are simple (single owner, no concurrency risk).
- PATCH is used only to avoid sending redundant fields.

Situation	Recommended Method	Rationale
Toggle a single user preference or flag	PATCH	Partial update is isolated and critical to UX.
Update multiple independent fields in a shared resource	PATCH	Prevents data overwrite from concurrent systems.
Replace a complete entity such as customer, invoice, or address	PUT	Simple and predictable full-replacement semantics.
Minor API optimization (no concurrency risk)	PUT	Avoids unnecessary PATCH complexity.

Singleton Resources

Definition

A singleton resource (e.g., /configuration, /profile, /settings, /status) represents a unique, addressable entity that may be mutable.

Guidance

Rule	Description
MAY use PATCH	For partial updates to existing or pre-created singletons.
MAY use PUT	When replacing the entire representation.
MUST include If-Match	To prevent overwriting concurrent changes.
MUST return 200 OK	With updated representation on success.
MUST return 412 Precondition Failed	If If-Match does not match current ETag.
SHOULD include ETag	Every GET/PATCH response must include new ETag.
MUST NOT	Use PATCH to create a singleton — creation uses PUT.

Example

```
PATCH /v1/organization/settings
If-Match: "v1"
Content-Type: application/json

{
  "auto_approve": false
}

HTTP/1.1 200 OK
ETag: "v2"
Content-Type: application/json
{
  "auto_approve": false,
  "timezone": "America/Chicago"
}
```

Patch Semantics

JSON Merge Patch (RFC 7396) — Preferred

Simplified document-based merge:

present keys → overwrite,

null → delete,

absent → no change.

```
PATCH /v1/customers/123
Content-Type: application/merge-patch+json

{ "status": "inactive" }
```

JSON Patch (RFC 6902) — Optional

Explicit, ordered operations for atomic control.

```
PATCH /v1/customers/123
Content-Type: application/json-patch+json

[
  { "op": "replace", "path": "/status", "value": "inactive" },
  { "op": "remove", "path": "/temporary_flag" }
]
```



Governance Callout — PATCH Format Standards

Lumen Default Standard:

All Lumen APIs **MUST** implement PATCH using **JSON Merge Patch (RFC 7396)** with

Content-Type: application/merge-patch+json.

Rationale

- Aligns with standard REST partial-update semantics.
- Easy for developers; payload mirrors resource shape.
- Fully idempotent when combined with ETag + If-Match.

Optional Alternative (Advanced Use Only)

application/json-patch+json (**RFC 6902**) **MAY** be supported for:

- Explicit add / remove / replace operations.
- Ordered multi-field or audited mutations.
- Complex structured documents.

Governance Enforcement

Rule	Description
MUST	Default Content-Type = application/merge-patch+json.
MAY	Support application/json-patch+json only after design review.
MUST NOT	Implicitly accept both without declaring them in OAS.
MUST	Validate PATCH media type via Spectral rule.

Spectral Example

```
rules:
  patch-must-use-merge-patch:
    description: "PATCH operations must default to application/merge-patch+json"
    given: "$.paths[*].patch.requestBody.content"
    then:
      field: "application/merge-patch+json"
      function: truthy
```

Summary

Aspect	Governance Standard
Default RFC	RFC 7396 — JSON Merge Patch
Optional RFC	RFC 6902 — JSON Patch
Default Content-Type	application/merge-patch+json
Optional Content-Type	application/json-patch+json (advanced)
Idempotency	ETag + If-Match required
Documentation	OAS MUST explicitly list supported types

Conditional Requests & Concurrency

Header	Usage	Example
ETag	Returned with GET/PATCH responses.	ETag: "v10"
If-Match	Required for optimistic concurrency.	If-Match: "v10"

Rules

- **MUST** return new ETag after successful PATCH.
- **MUST** reject updates without If-Match when concurrent writes possible.
- **MUST** return 412 Precondition Failed on mismatch.

- **MUST NOT** overwrite concurrent updates.

Idempotency

PATCH is not guaranteed idempotent but SHOULD be made so where possible.

Technique	Description
ETag + If-Match	Ensures update applies only to current version.
Idempotency-Key	Optional for safe retry behavior.
Deterministic Patch	Same payload → same final state.

Example

```
PATCH /v1/customers/123
If-Match: "v4"
Content-Type: application/merge-patch+json

{
  "email": "jane.doe@example.com",
  "status": "inactive"
}

HTTP/1.1 200 OK
Content-Type: application/json
ETag: "v5"

{
  "id": "123",
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "status": "inactive"
}
```

Status Codes

Code	Meaning	Applies To	Notes
200 OK	Resource partially updated	✔ Existing	MUST return updated representation.
204 No Content	Update succeeded (no body)	⚠ Optional	Only if representation unchanged.
400 Bad Request	Invalid JSON or schema	✔ Both	Syntax errors.
401 Unauthorized	Authentication required	✔ Both	

Code	Meaning	Applies To	Notes
403 Forbidden	Authenticated but not permitted	✓ Both	
404 Not Found	Resource missing	✓ Both	
405 Method Not Allowed	Wrong verb	✓ Both	
409 Conflict	Version/state conflict	✓ Both	Patch cannot apply.
412 Precondition Failed	If-Match mismatch	✓ Both	Stale ETag.
422 Unprocessable Content	Semantic validation failure	✓ Both	Violates business rule.
500 Internal Server Error	Unexpected failure	✓ Both	No internal details.

OAS Authoring Requirements

```

paths:
  /v1/customers/{id}:
    patch:
      summary: Partially update customer
      parameters:
        - in: path
          name: id
          required: true
          schema: { type: string }
        - in: header
          name: If-Match
          schema: { type: string }
      requestBody:
        required: true
        content:
          application/merge-patch+json:
            schema: { $ref: '#/components/schemas/CustomerPatch' }
      responses:
        '200':
          description: Successful partial update
          headers:
            ETag:
              description: Entity tag for versioning
              schema: { type: string }
        '412': { description: Precondition failed (stale ETag) }
        '400': { description: Invalid request }
        '404': { description: Not found }

```

Governance Checklist

Check	Validation
✓ Partial update only	PATCH does not require full resource.
✓ Concurrency control	ETag + If-Match required.
✓ Media type governed	Default RFC 7396 only.
✓ Status codes approved	200/204/400/401/403/404/405/409/412/422/500.
✓ Error model compliant	RFC 7807 Problem Details.
✓ OAS validated via Spectral	Governance automation ready.

Developer Summary

- Use PATCH for **partial updates** only.
- Default to **JSON Merge Patch (RFC 7396)**.
- Use If-Match and ETag for safe concurrency.
- Return 200 OK with updated representation or 204 No Content if unchanged.
- For **singleton resources**, use PATCH for partial changes, PUT for full replacement.
- Follow approved status codes and RFC 7807 error structure.
- Declare supported patch type(s) explicitly in OAS.
- All PATCH endpoints must pass Spectral linting for governance compliance.

API Standard: Caching & Concurrency with ETag

1. What is an ETag?

An **ETag (Entity Tag)** is an HTTP header that provides an identifier for a specific version of a resource.

Think of it as a "**fingerprint**" or "**version hash**" for a resource's state. When a resource (like a JSON object for a user) is created or modified, the server **MUST** generate a new, unique ETag value for it.

Clients **MUST** treat the ETag as an opaque string. They should store it, but never try to parse or interpret its contents.

Example ETag Response Header:

```
HTTP/1.1 200 OK
Content-Type: application/json
ETag: "abc-123-xyz-789"
{
  "id": "user-42",
```

```
"name": "Jane Doe"
}
```

1.1. Standard Mandate: Strong vs. Weak ETags

While all ETags serve a purpose, our standard **MANDATES** the use of **Strong ETags** because they are required for reliable concurrency control.

Feature	Strong ETag (Required for our API)	Weak ETag (Not allowed for Concurrency)
Primary Use	Concurrency Control (Optimistic Locking) and reliable caching.	Caching only (when byte-for-byte fidelity is not critical).
Guarantee	The resource is byte-for-byte identical . Any change, no matter how small, MUST generate a new ETag.	The resource is semantically equivalent , but minor presentation changes (like a footer timestamp) may not generate a new ETag.
Format	Enclosed in standard quotes (no prefix). "v1-hash-xyz"	Preceded by W/. W/"v1-hash-xyz"

Crucial Note: Only the **Strong ETag** provides the guarantee necessary for the server to safely allow a destructive update (PUT, PATCH, DELETE). The server **MUST NOT** accept an If-Match header containing a Weak ETag.

2. Why This Standard is Required

ETags are not optional. They are our platform's standard for solving two critical, high-level API problems:

- **Efficient Caching:** Prevents clients from re-downloading data they already have, saving bandwidth and improving client-side performance.
- **Concurrency Control:** Prevents the "lost update" problem, ensuring that clients don't accidentally overwrite each other's changes.

This guide defines the two ways ETags **MUST** be used.

3. Use Case 1: Caching (Conditional GET)

This flow prevents a client from re-downloading a resource that has not changed.

The Flow

1. Server **MUST** Send ETag: All GET responses for a single, cacheable resource **MUST** include an ETag header.
2. Client **SHOULD** Store ETag: The client application (web app, mobile app) **SHOULD** cache the resource's body and its ETag value.
3. Client **MUST** Send If-None-Match: The next time the client wants to GET that same resource, it **MUST** include the stored ETag value in an If-None-Match request header.
4. Server **MUST** Validate: The server receives the request and compares the If-None-Match value with the resource's current ETag.

The Result

- If the ETags **MATCH**: The client's version is up-to-date. The server **MUST** stop and return an empty **304 Not Modified** response. This saves the server from computing the response and saves bandwidth by not sending the data again.
 - If the ETags **DO NOT MATCH**: The resource has changed. The server **MUST** proceed as normal, returning a **200 OK** with the full, new resource and the new ETag.
-

4. Use Case 2: Concurrency Control (Optimistic Locking)

This is the most critical function of ETags. It prevents clients from overwriting each other's work.

The "Lost Update" Problem

1. Alice GET /users/123. She receives the user's data and the ETag: "v1".
2. Bob also GET /users/123. He receives the same data and the same ETag: "v1".
3. Alice sends a PATCH to change the user's name. Her request succeeds. The server updates the user and generates a new ETag: "v2".
4. Bob now sends a PATCH to change the user's phone number. His request is based on the old "v1" data. Bob's update overwrites Alice's name change. Alice's work is lost.

The ETag Solution (Mandatory)

All state-changing methods (PUT, PATCH, DELETE) **MUST** be protected against this.

1. Server **MUST** Enforce If-Match: Servers **MUST** require the If-Match header on all PUT, PATCH, and DELETE requests.
2. Client **MUST** Send If-Match: A client **MUST** include the ETag of the resource it thinks it's updating in the If-Match header.

The Correct Flow

1. Alice GET /users/123. She receives ETag: "v1".
2. Bob GET /users/123. He also receives ETag: "v1".
3. Alice sends PATCH /users/123 with the header If-Match: "v1".
4. The server checks: If-Match value ("v1") matches the current ETag ("v1").
5. The server accepts the update, processes it, and generates a new ETag: "v2".
6. Bob now sends PATCH /users/123 with his header If-Match: "v1".
7. The server checks: If-Match value ("v1") **does not match** the current ETag ("v2").
8. The server **rejects** the request with **412 Precondition Failed**.
9. Bob's update fails, which is the correct behavior. He is notified that the resource changed underneath him. He must now re-GET the resource to get the new data (and the "v2" ETag) and then retry his change.

5. Server Response Summary

This table summarizes the server's mandatory behavior when ETag-related headers are used.

Request Header Sent	Use Case	Client ETag vs. Server ETag	Server Response	What It Means
If-None-Match	Caching (GET)	Match	304 Not Modified	"The client's cached version is still good. Don't send the body."
If-None-Match	Caching (GET)	No Match	200 OK	"The resource has changed. Here is the new version and new ETag."
If-Match	Concurrency (PUT, PATCH, DELETE)	Match	200 OK (or 204)	"The client was working on the correct version. The update is accepted."
If-Match	Concurrency (PUT, PATCH, DELETE)	No Match	412 Precondition Failed	"Conflict!" The resource was changed by someone else. The update is rejected."

6. OpenAPI Specification (OAS 3.1) Examples

Example: Defining ETag on a GET Response


```

paths:
  /users/{userId}:
    get:
      summary: Get a single user
      responses:
        '200':
          description: The user resource.
          headers:
            etag:
              schema:
                type: string
              description: The ETag for this version of the resource.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'

```

Example: Requiring If-Match on a PATCH Request

```

paths:
  /users/{userId}:
    patch:
      summary: Update a user
      parameters:
        - in: header
          name: if-match
          description: The ETag of the resource to update, for concurrency control.
          required: true
          schema:
            type: string
      requestBody:
        # ... request body definition
      responses:
        '200':
          description: Update successful.
          headers:
            etag:
              schema:
                type: string
              description: The new ETag for the updated resource.
        '412':
          description: Precondition Failed. The resource has been modified.
          content:
            application/problem+json:
              schema:
                $ref: '#/components/schemas/Problem'

```

API Standard: Long-Running Operations (LRO)

1. Summary

This document defines the standard asynchronous Long-Running Operation (LRO) pattern for all Lumen APIs. This pattern **must** be used when an API operation cannot be completed within a standard, synchronous HTTP request-response cycle (e.g., > 2-3 seconds).

Holding an HTTP connection open is fragile and resource-intensive. The LRO pattern solves this by immediately acknowledging the client's request, returning a "job" or "operation" resource the client can poll for status, and decoupling the request's acceptance from its final completion.

This pattern provides:

- Deterministic asynchronous behavior
 - Standard client polling and progress tracking
 - Reliable retries via idempotency
 - A clear separation of **resource state** vs. **operation state**
 - Structured error reporting (per our **LPDP-Mini v1.0** profile) for failures
-

2. Why This Pattern Matters for Lumen

Many of our core business processes are complex and not instantaneous. Synchronous APIs are unsuitable and unreliable for these tasks. This pattern is the standard for:

- **Service Provisioning:** Activating a new network service involves orchestration across multiple systems and can take minutes or hours.
 - **Wholesale & Bulk Operations:** A partner submitting a POST for a wholesale order containing 5,000 circuits cannot be handled in a single synchronous transaction.
 - **Network Decommissioning:** Tearing down a complex service requires a series of ordered steps that must be tracked.
 - **Large Data Exports:** Generating a large-scale report (e.g., "export all billing data for the last 12 months").
-

3. The LRO Pattern Explained

3.1 Workflow

Step	Description	Example
1. Initiate	Client sends request (POST, PUT, PATCH, DELETE) to start a long-running transaction.	DELETE /ports/{id}
2. Accept	Server validates, starts background job; responds 202	Headers: Location: /operations/{opId}

	Accepted.	
3. Poll	Client polls GET /operations/{operationId} until terminal.	Payload: status, percent, error?
4. Result	On success, link to result; on failure, return Problem Details.	resource.href: "/ports/123"
5. Cleanup	Operation record is purged after a defined TTL.	GET /operations/{opId} → 404

3.2 Mandatory Operation Resource Schema

The GET /operations/{operationId} endpoint is the "source of truth" for the job. It **must** return a response body adhering to the following fixed schema.

Here is an **example** of a FAILED operation response:

```
{
  "operationId": "a1b2-c3d4-e5f6",
  "status": "FAILED",
  "percent": 33,
  "startedAt": "2025-10-29T16:00:00Z",
  "completedAt": "2025-10-29T16:00:15Z",
  "metadata": {
    "message": "Failed during partner verification.",
    "step": "PARTNER_VERIFY"
  },
  "resource": null,
  "error": {
    "title": "Partner timeout",
    "detail": "The operation failed because a downstream partner did not respond.",
    "errors": [
      {
        "code": "PARTNER_TIMEOUT",
        "message": "Partner did not confirm teardown within SLA",
        "meta": {
          "traceId": "traceId-abc123"
        }
      }
    ]
  }
}
```

3.2.1 Formal OAS 3.1 Schema Definitions

```
# These schemas MUST be used for the GET /operations/{id} response
components:
  schemas:
    Operation:
      type: object
```

```
required: [operationId, status, startedAt]
properties:
  operationId:
    type: string
    description: Unique identifier for the operation.
    example: "a1b2-c3d4-e5f6"
  status:
    type: string
    description: The current state of the operation.
    enum: [PENDING, RUNNING, SUCCEEDED, FAILED, CANCELLED]
  percent:
    type: integer
    format: int32
    description: Estimated progress percentage (0-100).
    example: 33
  startedAt:
    type: string
    format: date-time
    description: UTC timestamp when the operation was accepted.
    example: "2025-10-29T16:00:00Z"
  completedAt:
    type: string
    format: date-time
    description: UTC timestamp when the operation entered a terminal state.
    example: "2025-10-29T16:00:30Z"
  resource:
    type: object
    description: Present on SUCCEEDED for POST/PUT/PATCH.
    properties:
      id:
        type: string
        description: The ID of the created/modified resource.
        example: "port-1234"
      href:
        type: string
        format: uri
        description: The full URI to GET the resource.
        example: "/v1/ports/1234"
  error:
    description: Present on FAILED. Follows the LPDP Problem Details profile.
    $ref: '#/components/schemas/LpdpProblem'
  metadata:
    type: object
    description: >
      Service-specific metadata. This object contains rich, real-time
      status information.
    properties:
      message:
        type: string
        description: The current human-readable status message.
        example: 'Step 2 of 4: Verifying configuration...'
      step:
        type: string
        description: A machine-readable code for the current sub-state.
```

```

        example: 'VERIFYING_CONFIG'
        additionalProperties: true

# ----- LDPD Problem Details Profile Schemas -----
LdpdProblem:
  type: object
  additionalProperties: false
  description: Minimal Problem-Details envelope (RFC 9457-compliant)
  properties:
    title:
      type: string
      description: Short, human-readable summary.
    detail:
      type: string
      description: Optional longer explanation.
    errors:
      type: array
      minItems: 1
      items: { $ref: '#/components/schemas/LdpdError' }
  required: [ title, errors ]

LdpdError:
  type: object
  additionalProperties: false
  description: Individual error item.
  properties:
    code:
      type: string
      description: Stable, machine-readable identifier (string form preferred).
    message:
      type: string
      description: Human-readable explanation of the issue.
    meta:
      type: object
      description: >
        Optional unregulated extension for machine-readable context.
        Lumen does not prescribe its structure. Teams may define custom
        schemas if needed.
      additionalProperties: true
  required: [ code, message ]

```

3.3 Industry Standard Schemas (Google & Microsoft)

Major cloud providers use fixed schemas for operations response. This predictability is why the pattern is successful.

- **Google Cloud (AIP-151):** Uses a fixed schema for `google.longrunning.Operation`. It has standard fields: `name` (the op ID), `done` (boolean), a `result` field (for error or response), and a `metadata` object. This `metadata` object is used to hold the exact kind of qualitative, real-time status (e.g., 'Verifying configuration...') that our standard now requires.

- **Microsoft Azure:** Also uses a fixed schema. The polling URL (from Location or Azure-AsyncOperation headers) returns a standard object with fields like status, id, startTime, endTime, percentComplete, and error.

Our schema in 3.2 is an industry-aligned "best-of-both" approach, standardizing on the operationId, status, and our LpdpProblem error structure.

3.4 HTTP Semantics by Use Case

Use Case	URI	Expected Codes
Async Create	POST /resource	201 (sync) or 202 (async)
Async Replace/Update	PUT /resource/{id}	200/204 or 202
Async Partial Update	PATCH /resource/{id}	200 or 202
Async Delete	DELETE /resource/{id}	204 (sync) or 202 (async)
Poll for Status	GET /operations/{id}	200 (status) → 404 after TTL
List Operations	GET /operations	200
Request Cancellation	DELETE /operations/{id}	202

3.5 State Machine

3.5.1 State Definitions

- **RUNNING:** The operation is actively being processed.
- **SUCCEEDED:** The operation completed successfully.
- **FAILED:** The operation failed due to an error. The error field in the response **must** be populated.
- **CANCELLED:** The operation was successfully cancelled by a client request (DELETE /operations/{id}) and did not complete.

3.5.2 State Transitions

Operation states are terminal. Once an operation is SUCCEEDED, FAILED, or CANCELLED, its state **must not** change.

- RUNNING → SUCCEEDED, FAILED, CANCELLED
- SUCCEEDED → (No transitions)
- FAILED → (No transitions)
- CANCELLED → (No transitions)

3.6 Client Polling Strategy

Clients **must** implement a robust polling strategy.

- **Backoff:** Clients **should** use **exponential backoff with jitter** to avoid overwhelming the server (e.g., 1s, 2s, 4s, 8s... capped at 30-60s).
 - **Retry-After Header:** If the 202 Accepted or 200 OK (on a poll) response includes a Retry-After header, the client **must** respect it. This is the server's primary backpressure mechanism.
 - **Rate Limiting:** Polling endpoints (GET /operations/{id}) **are** subject to API rate limits. Excessive polling (e.g., ignoring backoff) **will** result in a 429 Too Many Requests error, which should also include a Retry-After header.
 - **Header Format:** Per RFC 9110, Retry-After can be seconds (Retry-After: 30) or an HTTP-date (Retry-After: Wed, 29 Oct 2025 16:30:00 GMT). Clients **must** support both formats.
-

4. Explaining the Pattern: Operation Polling vs. Resource Polling

This section details *why* this LRO pattern (polling /operations) is the standard and *why* the common anti-pattern (polling the resource itself) is disallowed.

4.1 The Correct Pattern: Polling the Operation Resource

The core of this pattern is the separation of the *job's* state from the *resource's* state. The GET /operations/{id} endpoint is the **only** source of truth for the *job*.

This section defines what a client should expect from the *business resource* (GET /resource/{id}) while an operation is in progress.

4.1.1 Behavior of GET /resource During an LRO

This is the standard for what a GET on the resource itself must return while a DELETE operation is in progress.

1. **While In-Progress (Job status: "RUNNING"):**
2. The GET /resource/{id} endpoint **must** return 200 OK with the full, normal resource representation.
3. **Caching:** The response **should** include Cache-Control: no-store or max-age=10.
4. **(Optional) Discoverability:** The response **may** include a Link: </operations/{operationId}>; rel="operation".
5. **(Optional) Lifecycle State:** You **may** add a *minimal, read-only* flag: "lifecycleStatus": "DELETING". Clients **must not** poll this field for completion.
6. **After Completion (Job status: "SUCCEEDED"):**

7. GET /resource/{id} **must** return 404 Not Found or 410 Gone.

4.1.2 Policy: 410 Gone vs. 404 Not Found

- 410 Gone is **preferred** for a short "tombstone" period (e.g., 1-24 hours) after a successful DELETE. It asserts the resource *did* exist. The response body **may** include an LpdpProblem body with the deletedAt time and operationId in the meta block.
- After the tombstone TTL, the server **should** revert to returning a standard 404 Not Found.

4.1.3 Concurrency During Deletion

While a resource is being deleted (LRO status: "RUNNING"), all access to that resource must be handled explicitly.

Request	Expected Response	Error Code (code)
GET /resource/{id}	200 OK	N/A
PUT or PATCH /resource/{id}	409 Conflict or 423 Locked	RESOURCE_LOCKED
DELETE /resource/{id} (duplicate)	202 Accepted	N/A (Returns original Location header)
POST (dependent sub-resources)	409 Conflict	OPERATION_IN_PROGRESS

4.2 The Anti-Pattern (Disallowed): Polling the Business Resource

A common but flawed alternative is to return 202 Accepted and then have the client poll GET /resource/{id}, waiting for its state to change (e.g., for status to become DELETED or for it to return 404). This anti-pattern is **disallowed** for the following reasons.

4.2.1 Flaw: Poor Observability

- **Problem:** The LRO pattern provides a "job log" (the operation resource). The anti-pattern only provides the final state of the resource. You cannot debug a failed process by only looking at the final object.
- **Example (Anti-Pattern):**
 - A DELETE /ports/123 fails. The team adds a status: "DELETE_FAILED" field to the GET /ports/123 response.
- **The Issue:** An operations team tries to debug. They have no idea *why* it failed (the error message doesn't belong on the port model). They don't know *when* the delete was attempted, *how long* it ran before failing, or *which* trace ID to look up. They can't answer: "What is our p95 time-to-delete?" or "What is our error rate for port deletions?"
- **Example (LRO Pattern):**

- The GET `/operations/op-abc` response shows `status: "FAILED"`, `startedAt: "..."`, `completedAt: "..."`, and a full error object with code: `"PARTNER_TIMEOUT"` and `meta: { "traceId": "..." }`.
- **The Benefit:** The operations team can now build a dashboard. They can query GET `/operations?status=FAILED` to find all failures. They can aggregate `errors[0].code` to see the top-most failure reason. They can compute exact durations (`completedAt - startedAt`) to get p95 latency SLOs.

4.2.2 Flaw: Security Risk & Information Leaks

- **Problem:** The resource (e.g., `/ports/{id}`) and the operation (e.g., `/operations/{op-abc}`) often have different security audiences. The resource is for *anyone* who can read it. The operation's *result* is often only for the *initiator*.
- **Example (Anti-Pattern):**
- To "fix" the observability flaw, a team adds an `errorInfo: "Downstream partner 'XYZ-Corp' timed out (trace-id: 99a-88b)"` field to the GET `/ports/123` response.
- **The Risk:** A low-privilege, read-only user from a *different tenant* (who just has read-access to the port) now polls the resource. They have just learned:
 1. A delete was attempted (which they shouldn't know).
 2. The internal name of our downstream partner (XYZ-Corp).
 3. An internal `trace-id` they can use for other attacks.
 4. This is a major information leak.
- **Example (LRO Pattern):**
- The read-only user polls GET `/ports/123` and sees the normal `200 OK` response. They have no idea a delete is in progress.
- The initiator (with correct auth) polls GET `/operations/op-abc` and gets the `FAILED` status and error.
- The read-only user tries to access GET `/operations/op-abc` and gets a `404 Not Found` (or `403 Forbidden`). The sensitive error details are protected.

4.2.3 Flaw: High Server Load

- **Problem:** A GET on a business resource (e.g., `/ports/{id}`) is often a "heavy" query. A GET on an operation is a "light" query. Forcing clients to poll a heavy endpoint creates unnecessary load.
- **Example (Anti-Pattern):**
- A client polls GET `/ports/123` every 5 seconds. This endpoint is "heavy" because it's designed for UI/customer use. To build the response, the server must:

1. Join 5 tables in the database (port details, customer info, location data, child circuits, config).
2. Call 2-3 downstream microservices.
3. Serialize a large 100KB JSON payload.
4. A thousand clients polling this every 5 seconds creates massive read load on the main business database and downstream services.
5. **Example (LRO Pattern):**

- A client polls GET /operations/op-abc every 5 seconds. This endpoint is "light" because it's *designed* for polling.
 - The server does one thing: SELECT * FROM operations_table WHERE id = 'op-abc' (or even better, a Redis GET).
 - It returns a tiny 1KB JSON payload ({ "status": "RUNNING", "percent": 30 }).
 - This has almost zero impact on the core system. Furthermore, the server can use the ETag header. If the status hasn't changed, it returns 304 Not Modified (an empty body), saving bandwidth and processing.
-

5. Applying the Pattern to Workflows

5.1 Individual Example (DELETE)

1. Client: DELETE /connections/con-123
2. Server: → 202 Accepted, Location: /operations/op-abc
3. Client: GET /operations/op-abc (polls until terminal)
4. Server: → 200 OK, {"status": "FAILED", "error": {...}}

5.2 Bulk / Wholesale Example

This LRO pattern is the *same* one used for bulk interfaces; no new pattern is needed. The operation resource simply contains a list of sub-operations. (Note: Detailed bulk API design is a separate topic).

- **Partial Failure Policy:** The overall operation status reflects the batch:
- SUCCEEDED: All sub-operations succeeded.
- FAILED: *Any* sub-operation failed.
- **Client Resubmission:** The client is responsible for inspecting the subOperations array in the response and re-submitting *only* the items that FAILED, using new Idempotency-Key headers for the new batch request.
- **Limits:** Services **must** define and enforce:

- A maximum number of items per bulk request (e.g., 200).
 - A per-tenant concurrent operation limit (e.g., 10 active RUNNING operations).
-

6. Governance Requirements

All LRO-enabled APIs **must** adhere to these standards.

6.1 Idempotency

- **Requirement:** All LRO-initiating requests (POST, PUT, PATCH, DELETE) **must** require an Idempotency-Key header.
- **Scope:** The key's uniqueness is scoped *per tenant* and *per operation*.
- **Lifetime:** The server **must** cache the idempotency key and its result for a minimum of 24-72 hours.
- **Behavior:**
 - If a request is retried *while the operation is active* (RUNNING), the server **must** return the 202 Accepted with the *same* Location header.
 - If a request is retried *after the operation is terminal* (SUCCEEDED/FAILED), the server **must** return the final stored result (e.g., 200 OK with the final op body, or 410 Gone if the resource was deleted) until the idempotency key expires.

6.2 Cancellation

- **Mechanism:** Clients **may** request cancellation by sending DELETE /operations/{id}.
- **Allowable States:** Cancellation can only be attempted on operations in RUNNING state.
- **Response:**
 - 202Accepted: The cancellation request was accepted (best-effort). The client must poll the operation, which will eventually transition to CANCELLED or FAILED.
 - 409 Conflict: The operation cannot be cancelled. The server must return an application/problem+json response body. The detail or message should state: "The operation is already in a terminal state (SUCCEEDED, FAILED, CANCELLED) and cannot be cancelled, or cancellation is not supported for this RUNNING operation."
 - 404 Not Found: The operation is unknown or has expired.

6.3 Security & Tenancy

- **Visibility:** An operation resource GET /operations/{id} **must** be protected by the same authorization rules as the resource it's acting upon. An operation **must only** be visible to the initiator, resource owners, or tenant admins.

- **Listing:** GET /operations **must** be filtered by tenancy. A user must **never** see operations from another tenant.
- **Data Redaction:** Error messages (message and detail) **must not** contain PII, secrets, or internal stack traces.

6.4 Rate Limiting & Quotas

- **Requirement:** All LRO endpoints **must** be protected by rate limits.
- **Limits:** Services **must** enforce limits on:
 - Max concurrent RUNNING operations per tenant.
 - Max items per bulk request.
 - Polling frequency.
- **Response:** When a limit is exceeded, the server **must** return 429 Too Many Requests with a Retry-After header.

6.5 Error Taxonomy (Problem Details)

All errors **must** use application/problem+json and our LPDP-Mini v1.0 profile. The code is the canonical identifier.

Error Code (code)	HTTP Status	Meaning
OPERATION_EXPIRED	404	Polled an operation ID that is past its TTL.
RESOURCE_LOCKED	409 / 423	PUT/PATCH failed, delete in progress.
OPERATION_IN_PROGRESS	409	POST (sub-resource) failed, parent delete in progress.
VALIDATION_ERROR	400	The initial request body failed validation.
PARTNER_TIMEOUT	504	A downstream system failed to respond in time.
RATE_LIMIT_EXCEEDED	429	Too many requests or concurrent operations.

6.6 Caching & Headers

- GET /operations/{id}: **Must** return Cache-Control: no-store. **Should** support ETag and If-None-Match to allow clients to poll efficiently and receive a 304 Not Modified if the status hasn't changed.
- GET /resource/{id} (during delete): **Must** return Cache-Control: no-store or a very small max-age (e.g., 10 seconds).

6.7 Operation Resource Lifecycle

Phase	Description	Response Code
Active	Operation created (PENDING, RUNNING).	200 OK
Terminal	Operation reaches SUCCEEDED, FAILED, or CANCELLED.	200 OK (for the duration of the TTL)
Expired (purged)	Operation record past retention TTL (7-30 days).	404 Not Found with code: "OPERATION_EXPIRED"
Manually deleted	Client performs DELETE /operations/{id}.	202 Accepted, then 404 Not Found

7. Industry References (for Architects)

- **Google Cloud:** [AIP-151 – Long-Running Operations](#)
- **Microsoft Azure:** [Asynchronous Operations](#)
- **RFC 9457:** [Problem Details for HTTP APIs](#)

8. OAS 3.1 Snippets (For Implementation)

8.1 LRO Initiator (e.g., Async DELETE)

```
paths:
  /resources/{id}:
    delete:
      summary: Deletes a resource asynchronously
      parameters:
        - in: header
          name: Idempotency-Key
          required: true
          schema: { type: string, minLength: 8 }
      responses:
        '202':
          description: Accepted; poll the operation resource for status.
          headers:
            Location:
              schema: { type: string, format: uri }
              description: The URL of the /operations/{operationId} resource.
            Retry-After:
              schema: { type: integer, example: 10 }
        '404': # ... standard errors
        '409': # ... concurrency/lock error
        '429':
          description: Rate limit exceeded (e.g., too many concurrent ops).
          headers:
```

```

      Retry-After: { schema: { type: integer, example: 60 } }
    content:
      application/problem+json:
        schema: { $ref: '#/components/schemas/LpdpProblem' }
  '503':
    description: Service Unavailable (e.g., job queue is full).
    headers:
      Retry-After: { schema: { type: integer, example: 120 } }
    content:
      application/problem+json:
        schema: { $ref: '#/components/schemas/LpdpProblem' }

```

8.2 Get Operation Status (Polling)

```

paths:
  /operations/{operationId}:
    get:
      summary: Gets the status of a long-running operation.
      parameters:
        - in: header
          name: If-None-Match
          schema: { type: string }
          description: ETag for a 304 Not Modified response.
      responses:
        '200':
          description: Operation status.
          headers:
            ETag: { schema: { type: string } }
            Retry-After: { schema: { type: integer, example: 10 } }
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Operation' }
        '304':
          description: Not Modified. Operation status has not changed.
        '404':
          description: Operation expired or not found.
          content:
            application/problem+json:
              schema: { $ref: '#/components/schemas/LpdpProblem' }

```

8.3 Cancel Operation

```

paths:
  /operations/{operationId}:
    delete:
      summary: Requests cancellation of an operation.
      responses:
        '202':
          description: Cancellation request accepted (best-effort).
        '404':
          description: Operation not found or expired.

```

```
'409':  
  description: Conflict. Operation is already in a terminal state.
```

9. Implementation & Observability

9.1 Time & Consistency

- **Timestamps:** All timestamps (`startedAt`, `completedAt`) **must** be in UTC and formatted as **ISO-8601**.
- **Consistency:** A small delay (eventual consistency) is permitted between the LRO terminal state (`SUCCEEDED`) and the resource state (`GET /resource/{id}` returning `404`). The `GET /operations/{id}` endpoint is **always** the source of truth for the *job's* status.

9.2 Observability & SLOs

- **Required Metrics:** Services **must** emit metrics for:
 - `operation_started_total` (Counter, tagged by type, tenant)
 - `operation_completed_total` (Counter, tagged by type, tenant, status: [`SUCCEEDED`, `FAILED`, `CANCELLED`])
 - `operation_duration_seconds` (Histogram, tagged by type, status)
 - **Tracing:** The `operationId` **must** be correlated with the distributed trace context (e.g., in `error.errors[0].meta.traceId`).
 - **Target SLOs:** Teams **must** define and publish SLOs for LROs (e.g., p95 completion for `DELETE` < 5 minutes).
-

10. Optional Extensions: Event-Based Notifications

Polling is the default, but services **may** offer event-based notifications for advanced clients.

- **Methods:** Server-Sent Events (SSE), Webhooks, or Kafka topics.
- **Payload:** The event payload **should** match the Operation schema.
- **Webhooks:** If webhooks are used, the service **must** provide a way for clients to register a callback URL and **must** use an HMAC signature (e.g., in `Webhook-Signature` header) for verification.

API Standard: Bulk Data Transfer (Handling Massive Payloads)

1. Purpose & Scope

This standard is an extension of the [API Standard: Long-Running Operations \(Async API Pattern\)](#).

The "Long-Running" guide solves the **TIME** problem (jobs > 60 seconds). This guide solves the **SIZE** problem (payloads > 5MB).

Together, they form the complete, mandatory pattern for any asynchronous job that processes bulk data.

2. The Problem (The Anti-Pattern)

Synchronous REST APIs and API Gateways (like Apigee) are designed for small, fast, metadata-based transactions (typically < 5MB).

Teams **MUST NOT** attempt to use synchronous API calls for bulk data jobs. This anti-pattern includes:

- Sending multi-megabyte JSON/XML payloads in a POST or PUT.
- Expecting a multi-megabyte JSON/XML payload from a GET.
- Requesting gateway timeouts greater than the 60-second default.

This practice creates critical stability, security, and resource-exhaustion risks for the entire platform, affecting all other services. **Requesting longer timeouts or larger payload limits is not the correct solution.**

3. The Standard Pattern (Decoupled "Claim Check")

For any operation that involves a request or response payload larger than the 5MB gateway limit, the **Decoupled Data Transfer** pattern described below **MUST** be used.

This pattern separates the small, fast *control plane* (the API call) from the large, slow *data plane* (the file transfer). The API Gateway **MUST** only handle the control plane. The data plane is delegated to the cloud storage provider.

Use Case A: Bulk Request (Massive Upload)

This flow is used when a client needs to *send* a massive file to an API.

1. Step 1: Client Requests Upload URL (Control Plane)

The client makes a small, fast, and authenticated API call to the gateway to request a secure upload location.

POST /v1/reports

(See Metadata Schema below for required request/response formats) 2. **Step 2: API Responds with Pre-Signed URL (Control Plane)**

The API service generates a secure, one-time-use upload URL from GCS/S3 or any cloud storage bucket and returns it to the client.

```
{ "fileId": "abc-123-xyz", "preSignedUploadUrl": "[https://storage.googleapis.com/bucket/upload?signature=..." } 3. Step 3: Client Uploads File (Data Plane)
```

The client uploads the massive payload directly to the preSignedUploadUrl. This traffic bypasses the API Gateway. 4. **Step 4: Client Commits the File (Control Plane)**

Once the upload is complete, the client makes a second small API call to notify the service that the file is ready for processing.

POST /v1/reports/abc-123-xyz/process 5. **Step 5: API Responds Asynchronously (Control Plane)**

The API Gateway now follows the API Standard: Long-Running Operations, returning an HTTP 202 Accepted with a statusUrl.

```
{ "jobId": "job-999", "status": "Queued", "statusUrl": "/v1/reports/status/job-999" }
```

Use Case B: Bulk Response (Massive Download)

This flow is used when a client makes a request that *produces* a massive result.

1. **Step 1: Client Starts the Job (Control Plane)**

The client makes a small, fast POST request to start the job, per the Long-Running Operations standard.

POST /v1/reports/generate 2. **Step 2: API Responds Asynchronously (Control Plane)**

The API Gateway instantly returns an HTTP 202 Accepted with a statusUrl. The 60-second timeout problem is solved.

```
{ "jobId": "job-777", "status": "InProgress", "statusUrl": "/v1/reports/status/job-777" } 3. Step 3: Server Produces File (Background Work)
```

The backend service runs the 5-minute query and saves the massive payload as a file in a temporary cloud storage bucket, adhering to the security requirements below. 4. **Step 4: Client Polls for Status (Control Plane)**

The client polls the statusUrl.

GET /v1/reports/status/job-777 5. **Step 5: API Responds with Result URL (Control Plane)**

Once the job is done, the API returns a small JSON payload containing a new, pre-signed download URL for the file in GCS/S3 or any cloud storage bucket. The massive payload problem is solved.

200 OK

```
{ "jobId": "job-777", "status": "Complete", "resultUrl": "[https://storage.googleapis.com/bucket/download/report-777.csv?signature=..." } 6. Step 6: Client Downloads File (Data Plane)
```

The client follows the resultUrl to download the massive file directly from the cloud storage bucket. This traffic bypasses the API Gateway.

Use Case C: Failure Handling & Retries

The process is designed to be resilient.

Upload Failure Logic (Use Case A)

If the upload in **Step 3** fails (e.g., network error, URL expires), the preSignedUploadUrl is considered invalid.

- **DO NOT** retry the PUT to the same failed URL.
- **DO** restart the process from **Step 1** to request a **brand new** preSignedUploadUrl.

Download Failure Logic (Use Case B)

If the download in **Step 6** fails (e.g., network error, resultUrl expires), the job itself (job-777) is **still complete**.

- **DO NOT** queue a new job (do not call POST /v1/reports/generate again).
- **DO** restart the process from **Step 4** and call the statusUrl (GET /v1/reports/status/job-777) again.
- The server will see the job is "Complete" and generate a **brand new, fresh** resultUrl with a new expiration, allowing the client to retry the download.

4. Security Requirements

All services implementing this pattern **MUST** adhere to the following security controls:

- **Short Expiry (TTL):** Pre-signed URLs MUST be generated with the shortest possible Time-to-Live (TTL).

- **Upload URL (Use Case A):** Recommended TTL is **15 minutes**.
- **Download URL (Use Case B):** Recommended TTL is **60 minutes**.
- **Scoped Access:** The URL **MUST** be scoped to the *minimum* required operation. For `preSignedUploadUrl`, this **MUST** be PUT. For `resultUrl`, this **MUST** be GET. List/Delete/Write permissions are forbidden.
- **Encryption:** All data at rest in the storage bucket (even temporary data) **MUST** be encrypted using server-side encryption (AES-256 or equivalent).
- **Auditing:** The API service **MUST** log the generation of every pre-signed URL, including the associated `fileId/jobId` and the authenticated client context (ideally including the `W3C trace_id`).

5. Metadata Schema Requirements

The JSON payloads for the control plane (Steps 1, 2, 5) **MUST** be part of the API's formal OpenAPI 3.1 specification.

Example Schema for Upload Response (Step 2)

```
{
  "type": "object",
  "properties": {
    "fileId": {
      "type": "string",
      "description": "The unique identifier for the uploaded file.",
      "example": "abc-123-xyz"
    },
    "preSignedUploadUrl": {
      "type": "string",
      "format": "uri",
      "description": "The temporary URL to PUT the file to."
    }
  },
  "required": ["fileId", "preSignedUploadUrl"]
}
```

Example Schema for Download Response (Step 5)

This schema defines the successful response (status: "Complete") and the failed job response (status: "Failed").

```
{
  "type": "object",
  "properties": {
    "jobId": {
      "type": "string",
      "example": "job-777"
    }
  }
}
```

```

    },
    "status": {
      "type": "string",
      "enum": ["InProgress", "Complete", "Failed"],
      "example": "Complete"
    },
    "resultUrl": {
      "type": "string",
      "format": "uri",
      "description": "Present only when status is 'Complete'. This is the pre-signed URL"
    },
    "error": {
      "description": "Present only when status is 'Failed'. Follows the LPDP-Mini v1.0 s"
      "$ref": "#/components/schemas/LpdpProblem"
    }
  },
  "required": ["jobId", "status"]
}

```

Dependent Schemas (Error Profile)

The following schemas are required to support the error property above, per the **API Standard: Lumen Problem Details (LPDP-Mini v1.0)**.

```

{
  "components": {
    "schemas": {
      "LpdpProblem": {
        "type": "object",
        "additionalProperties": false,
        "description": "Minimal Problem-Details envelope (RFC 9457-compliant) per LPDP-M"
        "properties": {
          "title": {
            "type": "string",
            "description": "Short, human-readable summary."
          },
          "detail": {
            "type": "string",
            "description": "Optional longer explanation."
          },
          "errors": {
            "type": "array",
            "minItems": 1,
            "items": { "$ref": "#/components/schemas/LpdpError" }
          }
        },
        "required": [ "title", "errors" ]
      },
      "LpdpError": {
        "type": "object",
        "additionalProperties": false,

```

```

    "description": "Individual error item.",
    "properties": {
      "code": {
        "type": "string",
        "description": "Stable, machine-readable identifier (string form preferred).",
      },
      "message": {
        "type": "string",
        "description": "Human-readable explanation of the issue."
      },
      "meta": {
        "type": "object",
        "description": "Optional unregulated extension for machine-readable context.",
        "additionalProperties": true
      }
    },
    "required": [ "code", "message" ]
  }
}
}
}

```

6. Related Standards

This standard **MUST** be implemented in conjunction with the following governance standards:

- [API Standard: Long-Running Operations](#): This is a required component for the asynchronous flow.
- [API Standard: Lumen Problem Details \(LPDP-Mini v1.0\)](#): This defines the mandatory error structure for a failed job.
- [API Standard: Pagination](#): Teams should first attempt to use standard pagination to handle large *list-based* responses. This Bulk Data standard is for *single-object* massive payloads (like a generated file).

7. Automation Notes (for Governance Tools)

To support automated governance and enforce this standard, the following checks **MUST** be implemented in our tooling.

1. Design-Time Check (OAS & Metadata)

This check validates the API's *design* and *metadata* to ensure the correct pattern is being used.

- **Metadata Usage**: Developers **MUST** add a tags array containing "bulk-data" to any OpenAPI operation that follows this standard.

- **Automation Rule:** The Spectral linter in the CI pipeline **MUST** enforce the following:
- If tags contains "bulk-data", the operation **MUST NOT** have a synchronous 200 OK or 201 Created response with a content body (it must be an async 202 Accepted).
- If tags contains "bulk-data", the operation **MUST** reference the standard schemas defined in this guide (e.g., for preSignedUploadUrl or resultUrl).

2. Build-Time Check (CI Pipeline & Gateway Config)

This check is the "hard guardrail" that validates the *as-built configuration* during deployment.

- **Automation Rule:** The CI/CD pipeline **MUST** scan the API Gateway (Apigee) proxy configuration files.
- **Action:** The build **MUST FAIL** if the configuration attempts to set the payload limit above the **5MB** enterprise standard.
- **Exception:** Any request for an exception (e.g., for a legacy system) **MUST** be flagged for manual review and approval by the API Governance Council.

8. Industry Precedent (The "Why")

This is the standard, non-negotiable pattern used by all major cloud providers to protect their platforms. They all separate the metadata API from the data plane.

- **Amazon (AWS):** Uses [S3 Pre-Signed URLs](#) to allow temporary, secure access to storage buckets.
- **Google (GCP):** Uses [Cloud Storage Signed URLs](#) for the same purpose.
- **Microsoft (Azure):** Uses [Shared Access Signatures \(SAS\)](#) to provide delegated access to storage.

By adopting this standard, we are aligning with industry best practices for building stable, secure, and scalable services.

API Standard: HTTP Headers

1. Naming & Formatting

- **MUST** use lowercase kebab-case ASCII for all HTTP header names.
 -  idempotency-key, traceparent, content-type, if-match
 -  X_Lumen_Entity_ID, X-Lumen-Entity-Id, RequestId, non-ASCII
 - **MUST NOT** use the legacy x- prefix for new headers (per RFC 6648).
 - **MUST** document any legacy interoperability exceptions in the API's local specification.
-

2. Header Registry (Allow-list)

To ensure client simplicity and enforceable governance, APIs **MUST** restrict headers to the following allow-list. Additions require a formal exception via the API Governance process.

2.1 Standard Headers (Allowed)

- **Content & Negotiation:** accept, content-type, content-length, content-encoding
- **Authentication:** authorization
- **Caching:** cache-control, etag, last-modified, expires, vary
- **Conditional Requests:** if-match, if-none-match, if-modified-since, if-unmodified-since
- **Conformance & Links:** link, location, retry-after, date
- **Range (when applicable):** range, content-range
- **Tracing & Correlation:** traceparent, tracestate (W3C)

Any new custom headers **MUST** be defined with a purpose, schema, max length, examples, and security considerations before being added to this list.

3. Authentication & Security

- **MUST** use authorization: Bearer <access_token> for OAuth2/JWT authentication.
 - **MUST NOT** introduce proprietary authentication headers without a formal review from the Security team.
 - **MUST NOT** place secrets or PII in headers or URLs. Always use request bodies over TLS.
 - **SHOULD** include the traceparent in all responses for correlation.
-

4. Content Negotiation & Errors

- **Clients SHOULD** send accept: application/json. Clients must only request media types the API supports.
 - **Servers MUST** set content-type: application/json for all JSON response bodies.
 - **Servers MUST** use content-type: application/problem+json and conform to **RFC 9457 (Problem Details)** for all 4xx and 5xx error responses.
 - **Servers MUST** return a 406 Not Acceptable if the client's accept header does not match a supported media type.
-

5. Caching & Conditional Requests

- ETag: Servers **MUST** include an ETag header on all GET responses for cacheable resources. This is preferred over Last-Modified.
 - If-None-Match: Servers **MUST** honor the If-None-Match request header, returning a 304 Not Modified if the ETag matches.
 - If-Match: Servers **MUST** use the If-Match header to enable optimistic concurrency control for PATCH, PUT, or DELETE requests.
 - Cache-Control: **MUST** be set according to the resource's caching policy (e.g., no-store for sensitive data, max-age for public metadata).
 - Location: **MUST** be returned with a 201 Created response, containing the canonical URI of the newly created resource.
-

6. Observability & Correlation

- **MUST** implement W3C Trace Context (traceparent, tracestate).
 - **Clients SHOULD** generate and send the traceparent header.
 - **Gateways MUST** ensure every request has a traceparent. If a client request is missing the traceparent header, the gateway **MUST** generate one before forwarding the request to internal services.
 - **Servers MUST** propagate the traceparent header across all internal service boundaries.
 - **Servers MUST** include the traceparent value in all error responses for end-to-end tracing.
-

7. Rate Limiting

Gateway should handle ratelimiting and use below headers.

- **SHOULD** use the IETF-standardized RateLimit response fields:
 - ratelimit-limit: The quota for the time window.
 - ratelimit-remaining: The number of requests remaining.
 - ratelimit-reset: The remaining time in seconds until the quota resets.
 - **MUST** include the Retry-After header (in seconds) when returning a 429 Too Many Requests or 503 Service Unavailable status due to throttling.
-

8. Size Constraints

- **Size Limits:** Header values **MUST** be ASCII. Total header size **SHOULD** remain within gateway limits (e.g., 8KB).
-

9. Governance & Linting (Spectral Snippets)

The following Spectral rules SHOULD be used to enforce these header standards.

Header Name Format

```
rules:
  lumen-header-name-format:
    given: "$..parameters[?(@.in=='header')].name"
    then:
      function: pattern
      functionOptions: { match: "^[a-z][a-z0-9-]*$" }
```

10. Change Control

- Additions to the allow-list or new custom headers **MUST** go through API Governance with Security review.

API Standard: Delete Method

DELETE Method – Request and Response Standards

Summary

DELETE represents a request to remove the entire resource identified by the URI. It is *not* intended for partial removal, filtered deletion, or passing instruction payloads.

Request Rules

Rule	Requirement	Rationale
MUST NOT include a request body	DELETE has undefined semantics for a body per RFC 9110 §9.3.5	Ensures predictable behavior across proxies, SDKs, and gateways
MUST target a single resource URI	DELETE /{resource}/{id}	Full deletion semantics, not partial update
MUST NOT support conditional filters or lists in the body	Use PATCH or POST /{resource}/delete instead	Keeps REST semantics pure and idempotent
MUST be idempotent	Multiple identical DELETES should have the same effect	Required for retry safety

Example (Correct)

```
DELETE /fabric/v1/connections/12345
```

Response

```
HTTP/1.1 204 No Content
```

Example (Incorrect)

```
DELETE /mcgw/v1/routes
Content-Type: application/json

{
  "routeIds": ["r1", "r2", "r3"]
}
```

Why This Is Non-Compliant:

This DELETE request attempts partial modification via a request body, which has no defined semantics under RFC 9110.

Use PATCH or a POST action endpoint instead.

Response Rules

Status Code	Meaning	Use Case
204 No Content	 Successful deletion Or Resource does not exist	Resource deleted successfully Or Target resource already deleted or invalid
400 Bad Request	Invalid URI, malformed query parameter	Syntax or parameter validation failure
401 Unauthorized	Authentication required	Caller not authenticated
403 Forbidden	Insufficient permission	Caller lacks permission to delete
405 Method Not Allowed	DELETE not supported for this resource	Used when DELETE not implemented
409 Conflict	Discouraged. If 409 is possible during delete, model it as POST with cancel action.	e.g., "Resource locked" or "pending operation"
422 Unprocessable Content	Discouraged. If 422 is possible during delete, model it as POST with cancel action.	e.g., "Cannot delete active policy" — prefer PATCH or POST action
500 Internal Server Error	Platform/system failure	Server-side unexpected error

Response Body Rules

- **MUST NOT** include a response body for Delete.

Alternatives for Non-Standard Deletion Use Cases

Use Case	Recommended Design	Example
Delete subset of items	PATCH with remove operation	PATCH /mcgw/v1/routes { "remove": ["r1","r2"] }
Batch or filtered deletion	POST to action endpoint	POST /fabric/v1/connections/delete { "ids": ["c1","c2"] }
Conditional deletion	POST to /(resource);terminate with conditions	POST /mcgw/v1/sessions/terminate { "force": true }

Governance Enforcement Examples

Rule 1 – No DELETE Request Body

```
rules:
  lumen-no-delete-body:
    description: "DELETE requests must not define requestBody"
    given: "$.paths[*].delete"
    then:
      field: requestBody
      function: falsy
```

Rule 2 – Valid DELETE Responses

```
rules:
  lumen-delete-status-codes:
    description: "Allowed HTTP status codes for DELETE"
    given: "$.paths[*].delete.responses"
    then:
      function: lumen-validate-status-code
      functionOptions:
        validCodes: [204, 400, 401, 403, 404, 405, 500]
```

Summary Table

Category	Guideline
Request	DELETE MUST target a single resource and have no body
Response	Use only predefined codes (204, 400–500)
Governance	Enforced via Spectral rules for DELETE behavior and status codes

Key Takeaways

- DELETE is for **entire resource removal only** — not partial, conditional, or batch operations.
- No request body allowed — maintain idempotency and HTTP compliance.
- Use **PATCH** for partial deletion or **POST /action** for complex or conditional deletions.
- Responses should be predictable and minimal (typically 204 No Content).
- All error responses should follow **RFC 7807** for consistency and automation.

API Style Guide Governance Tracker

1. Overview

This page tracks the progress of the API Working Group in reviewing, consolidating, and approving the official Lumen API Style Guide. The goal is to create a single, unified set of standards that will be used for all API development and governance.

Status Legend

-  **Approved:** The standard has been reviewed and approved by the working group.
-  **Pending:** The standard is under review, has open issues, or is awaiting a formal decision.
-  **Draft:** The standard exists but has not yet been formally reviewed by the group.

2. Master Governance Tracker

Subject Area	Source	Status	Open Issues / Questions	Notes / Rationale
Principles of Cloud-First API Design	Confluence	 Approved		
OAS Specification Best Practices	Confluence	 Approved	3.1.x (limited support). Current support 3.0.x	Updated support for 3.0.x
API Versioning Strategy	Confluence	 Approved	Deprecation: date format to follow RFC. Modify status code	Updated
API Pagination Standards	Confluence	 Approved	next/prev is not possible at this time	Removed

API Standardized Error Responses	Confluence	 Approved	transparent, 9457 (RFC 9457 Problem Details)	Created RFC 9457 Problem Detail profile for Lumen
API URI Standards and Design Patterns	Confluence	 Approved		
API Standard: Idempotency	Confluence	 Approved		
API Standard: Partial Resource Retrieval	Confluence	 Approved		
API Standard: Delete Method	Confluence	 Approved		
API Standard: Caching & Concurrency	Confluence	 Approved		
API Standard: HTTP Headers	Confluence	 Approved		
API Standard: Date & Time Naming	Confluence	 Approved		
API Standard: Standardized Data Types	Confluence	 Approved		
API Standard: Rest & Resource Design	Confluence	 Approved		
API Standard: JSON Payload	Confluence	 Approved		extracted date standards separately from json payload
API Standard: GET Method	Confluence	 Approved		
API Standard: POST Method	Confluence	 Approved		
API Standard: PUT Method	Confluence	 Approved		
API Standard: PATCH Method	Confluence	 Approved		
webhook	(New Item)	☐ Draft	jacob(webhooks - undermetristic), casey (sre)	Lower priority, not required for base API Styleguide
API Governance Process	Confluence	☐ Draft		Lower priority, not required for base

				API Styleguide
API Standard: Long-Runing Operations (LRO)	Confluence	 Approved		Lower priority, not required for base API Styleguide
API Standard: Bulk Data Transfer	Confluence	 Approved		Lower priority, not required for base API Styleguide
API Standard: Batch Operations	Confluence	 Approved		

Export to Sheets

API Standard: Rate Limits & Error Handling Standard

1. Executive Summary

This standard defines how Lumen APIs should apply rate limiting and error handling using Apigee X, IETF `RateLimit-*` headers, and LPDP-Mini.

In most cases, every public-facing API should combine **two complementary layers**:

- **Layer 1 – Client Fairness:**
Per-client limits enforced with **Quota** (cluster-aware, contractual).
- **Layer 2 – Global Safety:**
Infrastructure protection using **SpikeArrest** (node-local, non-contractual).

1.1 Quick Decision Guide

Scenario	Recommended Approach	Rationale
Per-client or per-application product limits	Quota (Layer 1)	Accurate, cluster-wide counters and contractual semantics
Backend with fixed capacity (DB, mainframe)	SpikeArrest – Legacy (Layer 2)	Smooths concurrency and protects fragile systems
Autoscaling backend (Kubernetes, FaaS, microservices)	SpikeArrest – Modern (Layer 2)	Allows bursts up to a safe per-node ceiling
Authenticated APIs	Quota + SpikeArrest	Combines fairness with safety
Anonymous public APIs	IP-based SpikeArrest	Basic protection against bots and abusive traffic
No reliable identity for clients	SpikeArrest only	Quota requires a trusted identifier

Golden principle:

For most production APIs, use **Quota (Layer 1)** for fairness and **SpikeArrest (Layer 2)** for infrastructure protection.

2. Core Model: The Two-Layer Defense

To balance **user experience**, **fair usage**, and **backend stability**, all public-facing APIs should be designed using a two-layer funnel:

Layer	Scope	Policy	Primary Objective
Layer 1 — Client Fairness	Per application (<code>client_id</code>)	Quota (short interval)	Enforce contract-grade, per-client limits across the cluster (e.g., 1000 requests per 5 minutes)
Layer 2 — Global Safety	Aggregate traffic	SpikeArrest	Protect backends from traffic bursts, thundering herds, and gateway autoscaling events

2.1 Governance Intent

- **Quota** represents the **contract** between Lumen and the client (fairness, tiers, SKUs).
 - **SpikeArrest** represents **infrastructure protection** and should not be interpreted as a customer-facing limit or product feature.
-

3. Platform Behavior and Cluster Awareness

3.1 Quota (Cluster-Aware – Required for Layer 1)

- Quota counters are **distributed across all MPs**.
- Limits are preserved even as Apigee scales horizontally.

Required configuration:

```
<Distributed>true</Distributed>
```

Quota window semantics:

- Quota uses a **fixed window** model.
 - At the end of each interval, the counter resets completely.
 - Unused quota **does not carry over** to the next window.
-

3.2 SpikeArrest (Node-Local – Required for Layer 2)

- SpikeArrest executes **per MP** and is **not cluster-aware**.
- Each MP enforces its own local rate limit; total cluster throughput is a product of the configured rate and the number of MPs.

Effective cluster rate:

$$\text{Effectivelimit} = \text{ConfiguredRate_per_MP} \times \text{Active_MPs}$$

Governance guidance:

- Use SpikeArrest solely for **traffic shaping and backend safety**.
- Do not treat SpikeArrest as a customer-facing or contractual limit.
- When sizing SpikeArrest, use the **expected peak number of MPs**, not the current or minimum count, and validate via load testing.

4. Pattern 1 — Layer 1: Client Fairness (Quota)

4.1 Choosing an Appropriate Quota Interval

Quota intervals should balance fairness, user experience, and datastore overhead.

Interval	Advantages	Trade-offs	Recommended Usage
1 minute	Very tight fairness, quick recovery	Higher datastore traffic and possible “reset spikes”	Premium tiers and strict SLAs
5 minutes	Balanced fairness and overhead	Moderate burst tolerance	Recommended default
10 minutes	Lower load on datastore	Looser fairness, longer wait on exhaustion	High-volume, lower-value APIs
1 hour	Minimal storage pressure	Poor user experience on exhaustion	Legacy or transitional scenarios only; avoid for new designs

Default: For most APIs, a **5-minute** interval is recommended.

4.2 Example: Client Fairness Quota Policy

Purpose:

Enforces a contract-grade limit of 1000 requests per 5 minutes per verified client application.

```
<Quota name="Q-Client-Fairness">
  <Allow count="1000"/>
  <Interval>5</Interval>
  <TimeUnit>minute</TimeUnit>
  <Distributed>true</Distributed>
  <!-- Identifier must be derived from verified credentials, not client-provided headers -->
</Quota>
```



```
<Identifier ref="apigee.client_id"/>
</Quota>
```

Identifier best practices:

- `apigee.client_id` should be populated by `VerifyAPIKey` or `OAuth` flows.
- Avoid using client-controlled headers (e.g., `X-Client-Id`) as identifiers for Quota.
- If `apigee.client_id` is missing, that condition should be treated as an **authentication/authorization failure**, not as a rate-limit violation.

5. Pattern 2 — Layer 2: Global Backend Defense (SpikeArrest)

5.1 Option A: Legacy / Fixed-Capacity Backends (Strict Mode)

Use this approach when protecting systems that are sensitive to concurrency and cannot scale elastically, such as mainframes or certain relational databases.

Sizing guidance (Strict):

```
ConfiguredRate_per_MP =
  (Backend_Safe_TPS × Safety_Margin) / Target_MP_Count_at_Peak_Load
```

Where:

- `Backend_Safe_TPS` = safe throughput determined by testing
- `Safety_Margin` = 0.7–0.8 (20–30% headroom)
- `Target_MP_Count_at_Peak_Load` = expected MP count during peak traffic

Example:

```
<SpikeArrest name="SA-Global-Legacy">
  <!-- Example value; teams must derive actual values through load tests -->
  <Rate>200ps</Rate>
  <UseEffectiveCount>false</UseEffectiveCount>
</SpikeArrest>
```

5.2 Option B: Modern Autoscaling Backends (Burst Mode)

Use this approach when backends can scale horizontally (e.g., Kubernetes, FaaS, microservices) and primarily need protection against unbounded surges.

Sizing guidance (Burst):

```
ConfiguredRate_per_MP =  
  (Backend_Max_TPS × Safety_Margin) / Target_MP_Count_at_Peak_Load
```

Example:

```
<SpikeArrest name="SA-Global-Modern">  
  <!-- Example value; teams must derive actual values through load tests -->  
  <Rate>1000ps</Rate>  
  <UseEffectiveCount>true</UseEffectiveCount>  
</SpikeArrest>
```

These numeric values are illustrative. Teams should derive real values from performance baselines and capacity planning.

5.3 Anonymous or Unauthenticated APIs (IP-Based SpikeArrest)

For endpoints that do not have a reliable authenticated identity, IP-based rate limiting can provide basic protection.

5.3.1 Example: IP-Based SpikeArrest

```
<SpikeArrest name="SA-Anonymous-IP">  
  <Rate>10pm</Rate>  
  <Identifier ref="client.ip.normalized"/>  
</SpikeArrest>
```

IP extraction and normalization (simplified example):

```
<ExtractVariables name="EV-Extract-Client-IP">  
  <Source>request.header.x-forwarded-for</Source>  
  <Pattern>{clientip},</Pattern>  
  <VariablePrefix>extracted</VariablePrefix>  
</ExtractVariables>  
  
<Javascript name="JS-Normalize-IPv6" timeLimit="100">  
  <Source>  
    // In production, use a robust library for IPv6 normalization  
    var ip = context.getVariable("extracted.clientip") ||  
              context.getVariable("client.ip");  
    context.setVariable("client.ip.normalized", ip);  
  </Source>  
</Javascript>
```

Considerations:

- Carrier-grade NAT (CGNAT) can cause many users to share a single IP address.
- IPv6 address rotation within a /64 subnet can be used to evade simple IP-based limits.

- For higher assurance, consider augmenting IP-based limits with additional signals such as User-Agent or device fingerprinting.
-

6. Rate Limit Response Headers (RFC 7231 + draft-ietf-httpapi-ratelimit-headers)

Lumen adopts the IETF standard `RateLimit-*` **response headers**. These headers should only be exposed when the data is accurate at a cluster level.

6.1 Principles

- **Quota-based limits** have accurate distributed counters and should publish `RateLimit-*` headers.
 - **SpikeArrest-based limits** are node-local and should not publish rate-limit usage headers. They may only provide a generic `Retry-After`.
-

6.2 Infrastructure-Level Limits (SpikeArrest)

Scenario: SpikeArrest has blocked a request due to high load or excessive bursts.

Header behavior:

Header	Requirement	Notes
<code>Retry-After</code>	Recommended	A small fixed value (e.g., 5–10 seconds) to signal temporary backoff
<code>RateLimit-Limit</code>	Must not be sent	Not accurate cluster-wide
<code>RateLimit-Remaining</code>	Must not be sent	Not accurate
<code>RateLimit-Reset</code>	Must not be sent	Not accurate

6.3 Client Fairness or Business Limits (Quota)

Scenario: A client has reached its contractual or fairness limit for the current window.

Header behavior:

Header	Requirement	Description
<code>RateLimit-Limit</code>	Required	Total requests allowed in the window (e.g., 1000)
<code>RateLimit-Remaining</code>	Required	Remaining requests in the current window (commonly 0 at violation)

Header	Requirement	Description
RateLimit-Reset	Required	Seconds until the quota window resets
Retry-After	Required	Same value as RateLimit-Reset

7. Error Format: LPDP-Mini for 429 Responses

All 429 Too Many Requests responses must comply with **LPDP-Mini v1.0**, using `application/problem+json`.

7.1 Required Structure

- `title` — short, human-readable summary.
- `errors[]` — array of error objects containing:
 - `code` — machine-readable error identifier.
 - `message` — clear explanation for human consumers.
 - `meta` — optional, structured details that do *not* reveal internal or sensitive information.

7.2 Example LPDP-Mini 429 Response

```
{
  "title": "Rate Limit Exceeded",
  "errors": [
    {
      "code": "traffic.limit_exceeded",
      "message": "You have exceeded the allowed request rate.",
      "meta": {
        "retry_after_seconds": "60"
      }
    }
  ]
}
```

8. Implementation Pattern (Shared Flow Recommended)

To ensure consistency, rate limiting and error handling should be implemented as **Shared Flows** and attached via FlowHooks to relevant proxies.

8.1 Fault Handling Structure

The following pattern illustrates how to route different fault conditions:

```

<FaultRules>
  <!-- Authentication/authorization issues -->
  <FaultRule name="Handle-Missing-ClientID">
    <Condition>(apigee.client_id = null) or (apigee.client_id = "")</Condition>
    <Step><Name>AM-Set-401-Unauthorized</Name></Step>
  </FaultRule>

  <!-- SpikeArrest violations -->
  <FaultRule name="Handle-Spike-Violation">
    <Condition>(fault.name = "SpikeArrestViolation")</Condition>
    <Step><Name>AM-Set-429-SpikeArrest</Name></Step>
  </FaultRule>

  <!-- Quota violations -->
  <FaultRule name="Handle-Quota-Violation">
    <Condition>(fault.name = "QuotaViolation")</Condition>
    <Step><Name>JS-Calculate-Reset</Name></Step>
    <Step><Name>AM-Set-429-Quota</Name></Step>
  </FaultRule>
</FaultRules>

```

8.2 Authentication Fault Response (401 Unauthorized)

```

<AssignMessage name="AM-Set-401-Unauthorized">
  <Set>
    <StatusCode>401</StatusCode>
    <ReasonPhrase>Unauthorized</ReasonPhrase>
    <Headers>
      <Header name="Content-Type">application/problem+json</Header>
    </Headers>
    <Payload contentType="application/problem+json">
      {
        "title": "Authentication Required",
        "errors": [
          {
            "code": "auth.missing_credentials",
            "message": "A valid API key or OAuth token is required to access"
          }
        ]
      }
    </Payload>
  </Set>
</AssignMessage>

```

8.3 Reset Calculation Logic (Reusable JavaScript Policy)

This policy computes the `flow.reset_seconds` value using the Quota expiry time. It is designed to be reusable across multiple Quota policies.

JavaScript policy configuration:

```
<Javascript name="JS-Calculate-Reset" timeLimit="200">
  <ResourceURL>jsc://calculate-reset.js</ResourceURL>
  <Properties>
    <!-- Default policy name; can be overridden by per-proxy configuration -->
    <Property name="quotaPolicyName">Q-Client-Fairness</Property>
  </Properties>
</Javascript>
```

Script (calculate-reset.js):

```
var defaultPolicyName = properties.quotaPolicyName || "Q-Client-Fairness";

// Prefer dynamic policy name if provided by the runtime
var policyName = context.getVariable("quota.policy.name") || defaultPolicyName;

var expiryTimeVar = "ratelimit." + policyName + ".expiry.time";
var expiryTime = context.getVariable(expiryTimeVar); // expected in ms epoch
var currentTime = Date.now();

if (expiryTime) {
  var deltaMs = expiryTime - currentTime;
  var resetSeconds = Math.ceil(deltaMs / 1000);
  if (resetSeconds < 0) resetSeconds = 0;
  context.setVariable("flow.reset_seconds", resetSeconds);
} else {
  // Fallback: approximate reset using an interval value if available, otherwise default
  var intervalSeconds =
    context.getVariable("ratelimit." + policyName + ".interval") ||
    60;
  context.setVariable("flow.reset_seconds", intervalSeconds);
}
```

8.4 SpikeArrest 429 Response (Infrastructure Protection)

```
<AssignMessage name="AM-Set-429-SpikeArrest">
  <Set>
    <StatusCode>429</StatusCode>
    <ReasonPhrase>Too Many Requests</ReasonPhrase>
    <Headers>
      <Header name="Content-Type">application/problem+json</Header>
      <Header name="Retry-After">5</Header>
    </Headers>
    <Payload contentType="application/problem+json">
      {
        "title": "Rate Limit Exceeded",
        "errors": [
          {
            "code": "traffic.limit_exceeded",
```

```

        "message": "System load is currently too high to process your re
        "meta": {
            "retry_advice": "Please wait a few seconds before retrying."
        }
    }
]
}
</Payload>
</Set>
</AssignMessage>

```

8.5 Quota 429 Response (Client Fairness / Business Limits)

```

<AssignMessage name="AM-Set-429-Quota">
  <Set>
    <StatusCode>429</StatusCode>
    <ReasonPhrase>Too Many Requests</ReasonPhrase>
    <Headers>
      <Header name="Content-Type">application/problem+json</Header>

      <Header name="RateLimit-Limit">{ratelimit.Q-Client-Fairness.allowed.count}</Header>
      <Header name="RateLimit-Remaining">{ratelimit.Q-Client-Fairness.remaining.co

      <Header name="RateLimit-Reset">{flow.reset_seconds}</Header>
      <Header name="Retry-After">{flow.reset_seconds}</Header>
    </Headers>
    <Payload contentType="application/problem+json">
      {
        "title": "Quota Exceeded",
        "errors": [
          {
            "code": "traffic.quota_exceeded",
            "message": "You have reached your request allowance for the curr
            "meta": {
              "policy": "Quota"
            }
          }
        ]
      }
    </Payload>
  </Set>
</AssignMessage>

```

Teams may generalize the policy name similarly to the JavaScript policy if multiple Quota definitions are used.

9. Monitoring and Alerting

Observability is essential to effective traffic management. Teams are expected to configure dashboards and alerts using Apigee Analytics and backend metrics.

9.1 Key Metrics

- Count and percentage of requests resulting in `fault.name = "SpikeArrestViolation"`.
- Count and percentage of `fault.name = "QuotaViolation"` by `client_id`.
- Backend performance indicators (e.g., database CPU, latency, thread utilization).
- Effective SpikeArrest rate (configured rate per MP × number of MPs).

9.2 Recommended Thresholds

Metric	Warning Threshold	Critical Threshold	Suggested Action
SpikeArrest violations (share of traffic)	> 2%	> 5%	Investigate traffic pattern and backend capacity
Aggregate Quota violations (across clients)	> 10% of active clients	> 25% of active clients	Review quota settings and client usage profiles
Per-client Quota violations	> 100 per hour	> 1000 per hour	Investigate for abuse or misconfiguration
Backend latency (P95) during SpikeArrest activity	> 2× baseline	> 5× baseline	Review capacity, queries, and rate limits

10. Security Considerations

- Quota identifiers should always be derived from **verified credentials** (e.g., `apigee.client_id`) and not from user-controlled headers.
- Rate limiting configuration and error responses should not expose internal implementation details such as MP counts, absolute backend thresholds, or internal resource identifiers.
- For anonymous traffic, IP-based limits can help mitigate abuse but should not be relied upon as a strong identity mechanism.

11. Client Behavior Expectations

To interact responsibly with Lumen APIs, client implementations should:

1. Inspect `RateLimit-*` headers on all responses, not just on 429 responses.
2. Implement preemptive throttling when the remaining quota becomes low:

```
``` remaining = int(response.headers.get("RateLimit-Remaining", "0")) reset =  
int(response.headers.get("RateLimit-Reset", "0"))
```



if remaining < 10 and reset > 0: time.sleep(reset) `` 3. Respect Retry-After` values on 429 responses and use exponential backoff:

```
wait = int(response.headers.get("Retry-After", "1")) wait = min(wait * (2
** attempt), 300) # cap at 5 minutes time.sleep(wait) 4. Avoid tight retry
loops that disregard Backoff and Retry-After.
```

---

## 12. Testing and Validation

Before an API is promoted to production, rate limiting behavior should be validated through both targeted tests and load evaluations.

### 12.1 Pre-Production Tests

#### 1. Quota accuracy test

2. Send exactly Allow requests within a single window.
3. Confirm that the (Allow + 1)th request receives a 429 with correct RateLimit-\* headers.
4. After the RateLimit-Reset duration, confirm that the quota is available again.

#### 5. SpikeArrest load test

6. Increase traffic up to and beyond ConfiguredRate\_per\_MP × Active\_MPs.
7. Confirm that SpikeArrest violations occur before backend saturation.
8. Monitor backend latency and error rates.

#### 9. Autoscaling test

10. Trigger MP scale-up to the expected peak count.
  11. Verify that Quota behavior does not change and remains accurate.
  12. Verify that effective SpikeArrest behavior matches expectations.
- #### 13. Multi-region test (if applicable)
14. Generate concurrent traffic from multiple regions.
  15. Confirm that Quota counters behave correctly and that any replication lag is within acceptable tolerance.

### 12.2 Production Smoke Checks

After production deployment:

- Confirm that 429 responses produced by Quota include `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, and `Retry-After`.
  - Confirm that 429 responses produced by SpikeArrest include `Retry-After` only.
  - Verify that `flow.reset_seconds` is consistent with actual reset timings (within a small tolerance).
  - Confirm that configured alerts trigger appropriately under controlled conditions.
- 

## 13. Common Pitfalls and Preferred Practices

The following table summarizes common misconfigurations and the recommended alternatives:

Pattern to Avoid	Preferred Practice	Rationale
Using <code>request.header.client_id</code> as a Quota identifier	Use <code>apigee.client_id</code> populated by <code>VerifyAPIKey</code> or <code>OAuth</code>	Ensures the identifier is trusted and not spoofed
Setting very long Quota intervals (e.g., 1 hour) for fairness	Use 5–10 minute windows for most APIs	Reduces the impact of exhaustion on clients while maintaining control
Using <code>SpikeArrest</code> as a product tier or contractual limit	Use Quota for contractual limits and tiers	<code>SpikeArrest</code> is node-local and not suitable for contractual guarantees
Retrying 429 responses immediately without delay	Honor <code>Retry-After</code> and implement exponential backoff	Avoids retry storms and wasted capacity
Hard-coding policy names in JavaScript	Use runtime variables ( <code>quota.policy.name</code> ) or policy properties	Improves reusability and reduces configuration drift
Testing rate limits with only one MP	Test at the expected peak MP count	Ensures behavior is realistic under scaled conditions

## OAS Specification Best Practices

This document provides a set of standardized guidelines for creating high-quality, maintainable, and developer-friendly OpenAPI specifications. Adhering to these practices ensures consistency and clarity across all our APIs.

---

## OAS Version and Format

### Adopt OAS 3.0.3 as the Standard

- Utilize OAS 3.0.3 for all new API specifications to ensure alignment with ApigeeX industry practices.
- Include the correct version declaration at the start of the document:

openapi: 3.0.3

## Use YAML as the Documentation Format

- Employ YAML for writing OAS documents for better readability and maintainability.
  - Use a consistent indentation of two spaces to structure the document clearly.
- 

## Global Base-Path Versioning

Versioning must be handled at the **Server** level, not the **Path** level. This separates deployment configuration from resource definition and allows for easier refactoring and proxying.

Requirement 1: Server URLs

All entries in the servers array MUST end with a version identifier (e.g., /v1).

Requirement 2: Resource Paths

Endpoint paths MUST NOT contain version numbers. They should represent pure resources.

Component	❌ Bad Practice	✅ Good Practice
Server URL	https://api.lumen.com	https://api.lumen.com/v1
Path	/v1/services	/services

### Example:

```
servers:
 - url: https://api.lumen.com/v1 # ✅ Correct
paths:
 /inventory/services: # ✅ Correct (Clean path)
 get:
 summary: List services
```

---

## API Metadata (info object)

The info object is the first thing a developer sees. It must clearly articulate the API's purpose and business value.

- `info.description`: This field **MUST** describe the business capability the API unlocks. It should answer the question, "What problem does this solve for the consumer?" Avoid technical jargon.
  - **Bad**: "An API for managing MCGW connections."
  - **Good**: "Use the Fabric Connections API to programmatically create, manage, and delete high-speed, private connections between your on-premises infrastructure and cloud providers like AWS, Azure, and GCP."
- 

## API Documentation Structure (tags and summary)

These fields control how your API is organized and displayed in documentation tools, making them critical for usability and context. -->

- `tags`: All operations **MUST** be grouped using tags. The tags **MUST** be based on the resource name (e.g., a "Connections" tag for all operations related to connections) to organize the API around its core business objects.on-summary-exist -->
  - `summary`: Every operation **MUST** have a summary. It must be a short, action-oriented phrase from the user's perspective that clearly states what the endpoint does.
  - **Bad**: "GET connections"
  - **Good**: "List all active connections"
- 

## Operation IDs (`operationId`)

The `operationId` is used by code generation tools to create method names in SDKs. Inconsistent or missing IDs lead to messy, unpredictable code and a poor developer experience.

- **Requirement**: Every operation **MUST** have a unique `operationId`.
  - **Convention**: The `operationId` **MUST** follow a consistent naming convention, such as `verbResource` (e.g., `listConnections`, `createConnection`). This ensures the generated code is predictable and intuitive.
- 

## Security

Apply security requirements at the individual path or operation level rather than globally. This approach provides precise control and accommodates varying security needs for different endpoints.

```
paths:
 /items:
 get:
 security:
 - OAuth2:
 - read
```

---

## Reusability and Naming

- **Reuse Schemas:** Leverage the components section to define and reuse schemas, parameters, and headers. This ensures consistency and makes the specification easier to maintain.
- **Descriptive Naming Conventions:** Use meaningful names for paths, parameters, and other elements, and follow the company's established naming standard.

## How to Document Reusable Components

The components section is the central repository for all reusable definitions.

### Reusable Request Parameters (Query & Path)

Define reusable query and path parameters under

```
components.parameters.
```

### Declare Reusable Parameters example

```
components:
 parameters:
 ResourceId:
 name: id
 in: path
 description: The unique identifier for the resource.
 required: true
 schema:
 type: string
 format: uuid
 LimitParam:
 name: limit
 in: query
 description: The maximum number of items to return per page.
 schema:
 type: integer
 default: 20
```

Reference them in your paths

```
paths:
 /v1/items/{id}:
 parameters:
 - $ref: '#/components/parameters/ResourceId'
```

## Reusable Request Headers

Request headers are treated as parameters with "in: header" and are also defined under

```
components.parameters
```

### Define the reusable header as a parameter

```
components:
 parameters:
 CorrelationIdHeader:
 name: X-Correlation-ID
 in: header
 description: Tracks a request across multiple services for debugging.
 required: true
 schema:
 type: string
 format: uuid
```

### Reference it in your path

```
paths:
 /v1/items:
 post:
 parameters:
 - $ref: '#/components/parameters/CorrelationIdHeader'
```

## Reusable Response Headers

Response headers have their own dedicated section for reusability:

```
components.headers
```

### Define the reusable response header

```
components:
 headers:
 RateLimitRemaining:
 description: The number of requests left for the current time window.
 schema:
 type: integer
```

### Reference it in a response definition

```

paths:
 /v1/items:
 get:
 responses:
 '200':
 description: A successful response.
 headers:
 RateLimit-Remaining:
 $ref: '#/components/headers/RateLimitRemaining'

```

---

## Data Types and Examples

- **Provide Realistic Examples:**
- Include examples that cover various real-world scenarios (common, edge, and error cases).
- Use the example or examples keyword within schema definitions for both requests and responses.
- Ensure examples use realistic and relevant data.

### OAS examples Example

```

paths:
 /v1/items:
 post:
 summary: Create a new item
 requestBody:
 required: true
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/Item'
 examples:
 # First example: A simple item with only required fields
 simpleItem:
 summary: A basic item
 description: An example of creating an item with the minimum required in
 value:
 name: "Standard Widget"
 price: 19.99

 # Second example: An item with optional fields included
 itemWithOptionalFields:
 summary: An item with optional data
 description: An example of creating an item that includes optional field
 value:
 name: "Premium Widget"
 price: 29.99

```

```
description: "A high-quality, durable widget."
sku: "PREM-WID-001"
```

Tool	Rule	Example
Spectral Linter	Ensure fields is declared in GET operations where allowed	oas-parameter-name: fields
Governance Rule	Lint rule naas-partial-response-allowed	Enforced only for GET endpoints returning resources > 10 fields

## Performance Considerations

- Reduces payload size by **30–80 %** for high-volume telemetry endpoints.
- Lowers serialization/deserialization overhead in API Gateway and MCGW controllers.
- Prevents over-fetching in portal UI and CLI integrations.

## Security & Compliance

- Projection **cannot** bypass authorization. The API **MUST** validate that each requested field is permitted under the caller's security scope.

## Summary

Benefit	Impact
Reduced payloads and latency	Better API responsiveness for UI and CLI clients
Standardized pattern across products	Consistent developer experience
Governance-ready	Easily enforced via OAS and Spectral rules

## API Standard: Standardized Data Types and Formats

### Purpose

To ensure consistency, readability, and interoperability across all APIs published by Lumen, this section defines the canonical data types, formats, and reusable schemas to be used in all OpenAPI 3.0.3 specifications.

### Principles

- APIs **MUST** use only the data types and formats listed here.
- All fields that accept structured values (e.g., country, currency, language, etc.) **MUST** include:



- a description explaining the expected standard,
- a pattern (regex) to make the rule self-explanatory to external developers, and
- an example value.
- Custom format identifiers (e.g., country-code) are **advisory only**.
- They are intended for internal governance, linting, and future migration to OAS 3.1.
- External consumers should be able to use the schema **without** referencing any style guide.

## Rationale: Custom format vs Regex

Aspect	Custom format	Regex + Example
<b>Purpose</b>	Governance keyword for Spectral, SDKs, portals	Human & machine validation rule
<b>External Developer Value</b>	Low – tooling hint only	High – immediately clear
<b>Governance Value</b>	High – centralized semantics	Medium – must duplicate everywhere
<b>Recommended Use</b>	Together – format + pattern + example	

## Standard Data Types (OAS 3.0.3)

Type	OAS type	OAS format	Description	Example
Boolean	boolean	—	true or false	true
Integer (32 bit)	integer	int32	–2,147,483,648 ... 2,147,483,647	42
Integer (64 bit)	integer	int64	–9,223,372,... 7, use ≤ 2 <sup>53</sup> – 1 for I-JSON	9007199254740991
Number (float)	number	float	IEEE-754 single precision	3.14
Number (double)	number	double	IEEE-754 double precision	3.1415926535
String	string	—	UTF-8 text	"Lumen rocks!"
Date	string	date	RFC 3339 date	"2025-11-11"
Date-Time	string	date-time	RFC 3339 timestamp (UTC Z)	"2025-11-11T14:05:00Z"
Byte (Base64)	string	byte	Base64 RFC 4648 § 4	"VGZzdA=="

Type	OAS type	OAS format	Description	Example
Binary	string	binary	Raw octet stream (upload/download)	<i>(file body)</i>

## Custom (Governed) Formats

✅ These are optional keywords for internal tooling and do **not** alter wire format.

Logical Type	OAS type	OAS format	Description / Rule	Example
URI	string	uri	URI per RFC 3986	"https://example.com/x"
Email	string	email	RFC 5322 address	"user@example.com"
UUID	string	uuid	RFC 4122 UUID	"279fc665-d04d-4dba-bcad-17c865489dfa"
Time	string	time	RFC 3339 partial time	"08:26:40Z"
Decimal	string	decimal-string	Arbitrary precision string	"1234.5678"
Language	string	language-tag	ISO 639-1 / BCP 47 code	"en-US"
Country	string	country-code	ISO 3166-1 alpha-2 uppercase	"US"
Currency	string	currency-code	ISO 4217 alpha-3 uppercase	"USD"
Bandwidth	integer	bandwidth-mbps	Normalized Mbps integer	1000 (1 Gbps)

Each field **must also define a pattern and example** so the meaning is clear even if clients ignore format.

## Reusable Schema Definitions

### CountryCode

```
CountryCode:
 type: string
 description: ISO 3166-1 alpha-2 country code (uppercase).
 format: country-code
 pattern: '^[A-Z]{2}$'
 example: "US"
```

### CurrencyCode

CurrencyCode:  
type: string  
description: ISO 4217 currency code (uppercase).  
format: currency-code  
pattern: '^[A-Z]{3}\$'  
example: "USD"

## LanguageTag

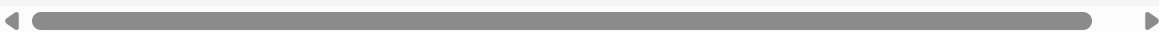
LanguageTag:  
type: string  
description: Language tag per ISO 639-1 / BCP 47.  
format: language-tag  
pattern: '^[A-Za-z]{2,3}(-[A-Za-z0-9]{2,8})\*\$'  
example: "en-US"

## Email

Email:  
type: string  
description: Email address per RFC 5322.  
format: email  
pattern: '^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+\$'  
example: "user@example.com"

## UUID

UUID:  
type: string  
description: Universally Unique Identifier per RFC 4122.  
format: uuid  
pattern: '^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}\$'  
example: "279fc665-d04d-4dba-bcad-17c865489dfa"



## URI

URI:  
type: string  
description: URI string per RFC 3986.  
format: uri  
pattern: '^(https?|ftp)://[^\s/\$. ?#].[^\s]\*\$'  
example: "https://api.lumen.com/v1/services"

---

## BandwidthMbps (Canonical Integer)

Represents bandwidth capacity.

- **Standard:** Value MUST be an integer normalized to **Megabits per second (Mbps)**.
- **Naming:** Fields referencing this schema MUST include the suffix Mbps in the property name (e.g., bandwidthMbps, uploadSpeedMbps) to avoid ambiguity.

```
BandwidthMbps:
 type: integer
 format: int64
 minimum: 0
 description: >
 Bandwidth capacity measured in Megabits per second (Mbps).
 1 Gbps = 1000. 10 Gbps = 10000.
 example: 10000
```

---

## Money Object ( Integer Minor Units)

Represents a monetary amount using ISO 4217 minor units (integer-based).

```
Money:
 type: object
 required: [amount, currency]
 properties:
 amount:
 type: integer
 minimum: 0
 maximum: 9007199254740991 # I-JSON safe (2^53 - 1)
 description: >
 Monetary amount expressed in ISO 4217 minor units.
 Example: USD 12.34 → amount=1234, currency="USD"
 currency:
 $ref: '#/components/schemas/CurrencyCode'
 example:
 amount: 1234
 currency: "USD"
```

---

## Address Object

A common postal address representation used consistently across all domains.

```
Address:
 type: object
 description: Postal address (country-specific formatting may apply).
 additionalProperties: false
 properties:
 line1:
 type: string
 description: Primary street line (street, number, unit).
 maxLength: 256
 example: "100 Main St Apt 5B"
 line2:
```

```

 type: string
 description: Secondary line (building, suite, floor) if needed.
 maxLength: 256
 example: "Building A, Floor 3"
 city:
 type: string
 description: City, town, or locality.
 maxLength: 128
 example: "Denver"
 state:
 type: string
 description: State, region, or province (country-specific).
 maxLength: 128
 example: "CO"
 postal_code:
 type: string
 description: Postal/ZIP code (country-specific format).
 maxLength: 32
 example: "80202"
 country:
 $ref: '#/components/schemas/CountryCode'
example:
 line1: "100 Main St Apt 5B"
 city: "Denver"
 state: "CO"
 postal_code: "80202"
 country: "US"

```

### Context-specific variants:

```

UsShippingAddress:
 allOf:
 - $ref: '#/components/schemas/Address'
 - type: object
 required: [line1, city, state, postal_code, country]

```

```

InternationalBillingAddress:
 allOf:
 - $ref: '#/components/schemas/Address'
 - type: object
 required: [line1, city, postal_code, country]

```

## Governance Validation Rules

Rule	Enforcement
Disallow unrecognized format values	Spectral lint
Require pattern, example, description when custom format used	Spectral lint
Validate $\text{Money.amount} \leq 2^{53} - 1$	Spectral + CI schema check

Rule	Enforcement
Enforce ISO regex patterns for country/currency/language	Spectral rule
Warn if float/double used for monetary values	Spectral rule

## Summary

- **External developers** should understand schemas immediately through **description + regex + example**, without needing to read the style guide.
- **Governance tooling** (Spectral, SDK generators, portals) can rely on **custom formats** for consistency and automation.
- **Money, Address, Country, Currency, BandwidthMbps and Language** fields are standardized across all domains.

# API Standard: Date & Time Naming

## 1. Purpose & Scope

This standard defines how **date and date-time fields** must be named, structured, and represented in API payloads.

Consistent date naming and typing improves discoverability, reduces ambiguity, and enables automated validation through governance tools such as **Spectral**.

This convention applies to:

- All API resource representations (request and response payloads)
- Both system metadata (e.g., auditing fields) and business fields (e.g., expiration, activation)
- All date and date-time fields defined in OpenAPI specifications

## 2. Core Rules

### 2.1 Field Naming Suffixes

Suffix	Meaning	OpenAPI Format	Example
_at	Timestamp with time component (RFC 3339 date-time)	format: date-time	created_at, updated_at, starts_at
_on	Calendar date only (RFC 3339 date)	format: date	effective_on, expires_on
*_from / *_until	Time or date ranges	format: date-time or date	valid_from, valid_until

⚠️ The OpenAPI schema **MUST** align the suffix to its proper format:  
\_at → format: date-time, \_on → format: date.

---

## 2.2 State vs Schedule Semantics

Use timestamp names that clearly express **intent** — whether the value represents a **state change** (what has happened) or a **scheduled event** (what will happen).

Category	Description	Examples	Verb Tense	Mutability
State-based	Reflect past events that changed system state. Used for audit, versioning, or ordering.	created_at, updated_at, deleted_at, completed_at	Past	Immutable
Schedule-based	Define future or ongoing timeframes within a business process.	starts_at, ends_at, expires_at, effective_on, renews_on	Present / Future	Mutable
Range-based	Define valid windows for temporal entities.	valid_from, valid_until	Mixed	Usually Mutable

### 💬 Clarification:

- *State fields* answer "**When did this happen?**"
  - *Schedule fields* answer "**When will or should this occur?**"
- 

## 2.3 Examples

### ✅ Recommended

```
{
 "created_at": "2025-10-12T15:45:33Z",
 "updated_at": "2025-10-15T19:02:11Z",
 "starts_at": "2025-11-01T00:00:00Z",
 "ends_at": "2025-11-30T23:59:59Z",
 "valid_from": "2025-11-01T00:00:00Z",
 "valid_until": "2026-01-01T00:00:00Z"
}
```

### ❌ Not Recommended

```
{
 "created": "2025-10-12",
}
```

```
"modification_date": "2025-10-15T19:02:11Z",
"start_date": "2025-11-01",
"expire_at": "2025-11-30"
}
```

#### Issues:

- Mixed and inconsistent suffixes (\_date, \_at)
  - Some fields missing time component where expected
  - Ambiguous intent (is expire\_at a schedule or state?)
- 

### 3. Governance & Enforcement (Spectral Rule)

The following Spectral rule enforces this convention during API design-time validation.

#### Rule ID: lumen-date-field-naming

```
rules:
 lumen-date-field-naming:
 description: "Date/time fields must use '_at' for RFC 3339 date-time and '_on' for R
 message: >-
 "{{property}}" field name does not follow Lumen Date/Time naming convention.
 Use '_at' for RFC 3339 date-time, '_on' for RFC 3339 date.
 For ranges, use *_from / *_until."
 given: "$.components.schemas.*.properties.*"
 severity: warn
 then:
 - function: pattern
 field: "@key"
 functionOptions:
 match: ".*(_at|_on|_from|_until)$"
 - function: truthy
 field: "format"
 - function: enumeration
 field: "format"
 functionOptions:
 values:
 - date
 - date-time
```

#### ✅ What it checks:

- Ensures all date-related fields end with \_at, \_on, \_from, or \_until.
  - Validates that \_at fields use format: date-time.
  - Validates that \_on fields use format: date.
  - Flags inconsistent or ambiguous names (created, start\_date, expire\_at).
-



## 4. Automation Notes

- Integrate this rule into the **Spectral ruleset** used by the API linter in the CI pipeline.
  - Apply it to all repositories that define OpenAPI 3.x specifications.
  - Violations are classified as **warnings** (initial rollout) and will be escalated to **errors** once adoption reaches >80%.
- 

## 5. Related Standards

- [RFC 3339: Date and Time on the Internet](#)

# API Standard: PUT Method

## Purpose

The PUT method **replaces the full representation** of a resource at its canonical URI. It is **idempotent** — identical requests produce the same final state.

This page defines Lumen’s standards for designing, documenting, and implementing PUT operations, including creation semantics, conditional updates, and approved response codes.

---

## Semantics

Property	Requirement
Safe	✗ No, modifies server state.
Idempotent	✓ Yes, repeating the same request yields the same result.
Typical Use	Full replacement of a resource.
Request Body	✓ Required, represents the complete, updated resource.
Response Body	✓ Required for 200 OK, returns the latest representation.

---

## When to Use PUT

Use Case	Example	Required Behavior
Replace an existing resource completely	PUT /v1/customers/{id}	Replaces the full customer object with the provided payload; returns 200 OK and the updated resource.

Use Case	Example	Required Behavior
Create a resource at a client-known URI (rare)	PUT /v1/buckets/{id}	MAY be used if the client controls ID generation. Return 201 Created if the resource did not exist before.
Update large static configurations or documents	PUT /v1/configurations/{id}	Appropriate when sending an entire configuration file or template.
Atomic overwrite of immutable document	PUT /v1/policies/{id}	Old version replaced entirely, new ETag generated.

## Singleton Resources

### Definition

A **singleton resource** represents a unique instance that exists once per logical context (for example, /configuration, /profile, /settings, /status, /organization/{id}/policy).

It is not part of a collection. A singleton **may or may not pre-exist**, if it does not, the client can create it by PUTting a full representation at its canonical URI.

### Guidance

Rule	Description
<b>MAY use PUT to create a singleton</b>	When the resource has a known, fixed URI and does not yet exist. PUT is idempotent, repeating the same request yields the same result. Return <b>201 Created</b> on first creation, <b>200 OK</b> on subsequent replacements.
<b>MUST</b>	Require If-Match on updates when the singleton already exists to prevent concurrent overwrites.
<b>MUST return 201 Created</b>	When the singleton did not exist previously and has now been created.
<b>MUST return 200 OK</b>	When replacing an existing singleton with a new representation.
<b>SHOULD include ETag</b>	Every GET and PUT response for a singleton must include an ETag for version control.
<b>MUST NOT</b>	Use PUT for actions or only for full replacement. Use PATCH for partial updates.
<b>MUST NOT</b>	Allow DELETE on singleton endpoints.

## Examples

### Create (first-time PUT)

```
PUT /v1/organization/settings
Content-Type: application/json

{
 "auto_approve": true,
 "timezone": "America/Chicago"
}

HTTP/1.1 201 Created
ETag: "v1"
Location: /v1/organization/settings
```

### Replace (subsequent update)

```
PUT /v1/organization/settings
If-Match: "v1"
Content-Type: application/json

{
 "auto_approve": false,
 "timezone": "America/Chicago"
}

HTTP/1.1 200 OK
ETag: "v2"
```

### Conflict (stale ETag)

```
HTTP/1.1 412 Precondition Failed
Content-Type: application/problem+json
{
 "type": "https://api.lumen.com/errors/conflict",
 "title": "Precondition Failed",
 "status": 412,
 "detail": "The resource has been modified by another client."
}
```

## Governance Note

- Singleton PUT operations follow the same idempotency, full-replacement, and ETag/If-Match rules as normal resources.
- Example Spectral validation:

```
rules:
 singleton-put-must-support-etag:
 given: "$.paths[?(@property.match(/^\\v1\\/(configuration|profile|settings|status|o
```

```
then:
 field: "responses['200'].headers.ETag"
 function: truthy
```

## Summary

Singleton resources can be created or replaced using PUT when they have a fixed URI and no existing representation.

Return 201 on first creation, 200 on subsequent updates.

Always include ETag and require If-Match for concurrency control.

## General Rules

Rule	Description
<b>MUST</b>	Replace the <b>entire resource representation</b> ; omitted fields are treated as deleted.
<b>SHOULD NOT</b>	Use PUT for partial updates — use PATCH instead.
<b>MUST</b>	Return the updated resource in the response body.
<b>MUST</b>	Preserve the resource identifier and canonical URI.
<b>MUST NOT</b>	Allow structural mutations (e.g., changing resource type or parent linkage).
<b>MUST</b>	Be <b>idempotent</b> — re-submitting the same payload must not create duplicates.

## Conditional Requests & ETag

Header	Usage	Example
ETag	Returned with GET/PUT responses.	ETag: "v12"
If-Match	Required header for optimistic concurrency.	If-Match: "v12"

## Rules

- **MUST** return a new ETag on every successful modification.
- **SHOULD** reject updates without If-Match when concurrent edits are possible.
- **MUST** return 412 Precondition Failed if If-Match does not match.
- **MUST NOT** accept stale updates.

## Example

```
PUT /v1/customers/c123
If-Match: "v4"
Content-Type: application/json
```

```
{
 "id": "c123",
 "name": "Jane Doe",
 "email": "jane@example.com",
 "status": "active"
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json
ETag: "v5"
Cache-Control: private, max-age=60
```

```
{
 "id": "c123",
 "name": "Jane Doe",
 "email": "jane@example.com",
 "status": "active"
}
```

## Status Codes

Code	Meaning	Applies To	Notes
<b>200 OK</b>	Resource successfully replaced	✓ Existing resource	MUST include updated representation.
<b>201 Created</b>	Resource newly created at client-specified URI or singleton first creation	✓ Creation	MUST include Location header.
<b>204 No Content</b>	Update succeeded, no body returned	⚠ Optional	Only if representation unchanged.
<b>400 Bad Request</b>	Invalid or missing fields	✓ Both	Schema/validation errors.
<b>401 Unauthorized</b>	Authentication required	✓ Both	
<b>403 Forbidden</b>	Authenticated but not allowed	✓ Both	
<b>404 Not Found</b>	Resource does not exist	✓ Both	
<b>405 Method Not Allowed</b>	Wrong HTTP verb	✓ Both	
<b>409 Conflict</b>	Version/state conflict	✓ Both	For concurrent updates or integrity violations.
<b>412 Precondition Failed</b>	If-Match mismatch	✓ Both	Stale ETag.

Code	Meaning	Applies To	Notes
<b>422 Unprocessable Content</b>	Semantically invalid	✔ Both	Valid JSON, violates business rule.
<b>500 Internal Server Error</b>	Unexpected server failure	✔ Both	No internal details exposed.

## Headers

Header	Usage
If-Match	Required for conditional updates.
ETag	Returned in every successful PUT.
Cache-Control	MUST specify private or no-store.
Content-Type	MUST be application/json.
Accept	MUST be application/json or compatible.

## OAS Authoring Requirements

```

paths:
 /v1/customers/{id}:
 put:
 summary: Replace a customer resource
 parameters:
 - in: path
 name: id
 required: true
 schema: { type: string }
 - in: header
 name: If-Match
 schema: { type: string }
 requestBody:
 required: true
 content:
 application/json:
 schema: { $ref: '#/components/schemas/Customer' }
 responses:
 '200':
 description: Successful replacement
 headers:
 ETag:
 description: Entity tag for versioning
 schema: { type: string }
 '201': { description: Created (first-time PUT) }
 '412': { description: Precondition failed (stale ETag) }

```

```
'400': { description: Invalid request }
'404': { description: Not found }
```

## MUST

- Define If-Match and ETag headers.
- Restrict status codes to the approved matrix.
- Use application/json.

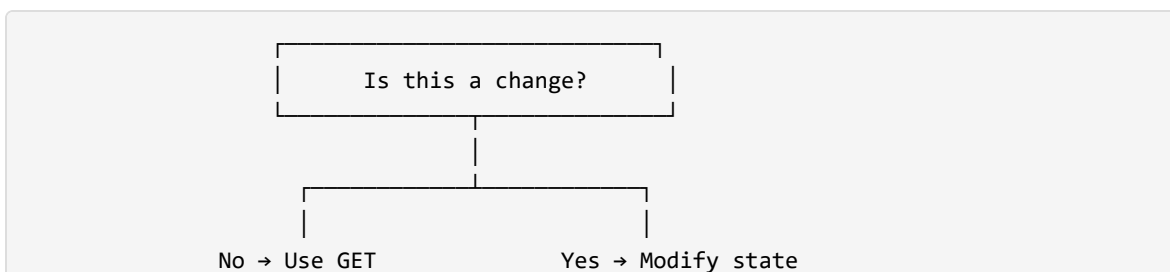
## Governance Checklist

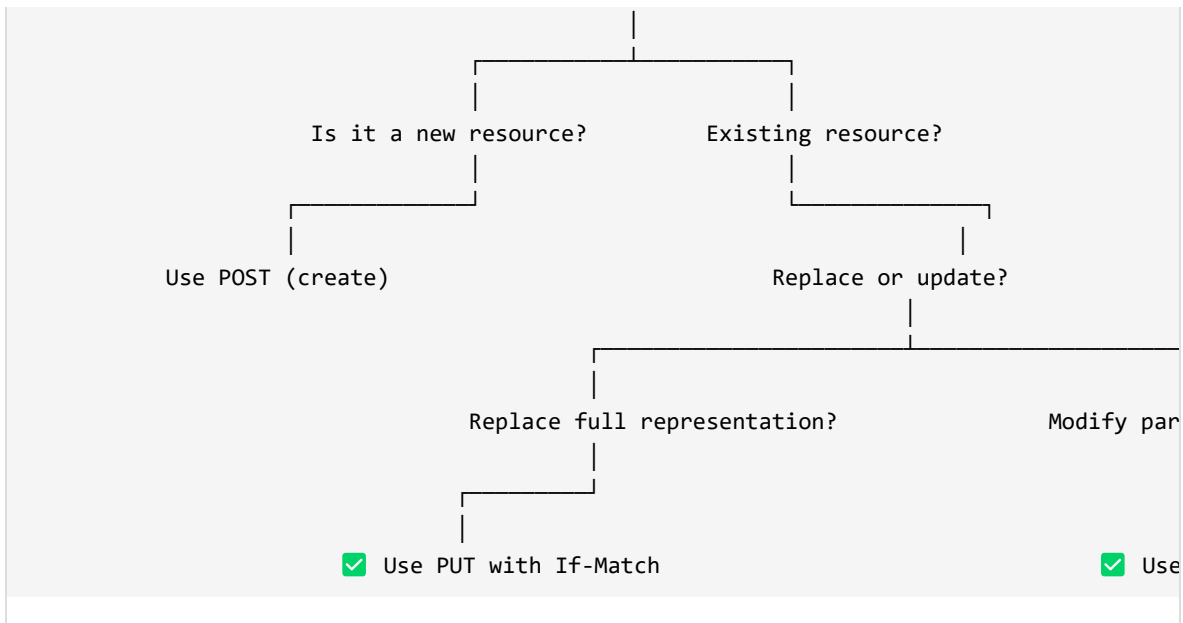
Check	Validation
✓ Idempotency guaranteed	PUT deterministically replaces the same resource
✓ Full resource in body	Required
✓ Returns updated representation	Required
✓ ETag / If-Match implemented	Required for concurrency control
✓ Proper status codes used	200/201/204/400/401/403/404/405/409/412/422/500
✓ Singleton creation allowed via PUT	201 Created on first creation
✓ OAS validated via Spectral	Governance automation ready

## Developer Summary

- Use **PUT** for full resource replacement or idempotent creation at a known URI.
- Include the entire object; missing fields imply deletion.
- Support **ETag** and **If-Match** for concurrency safety.
- Return 201 Created on first creation, 200 OK on updates.
- Ensure idempotency — repeated identical requests yield identical state.
- Use **PATCH** for partial updates.
- Follow the approved status-code matrix and RFC 7807 error model.
- For **singleton resources**, PUT can create or replace them at a fixed URI; always include ETag and If-Match.

## Conditional Update Decision Tree





---

## API Security Standards

### API Security Standards

### API Security Standard: Identity Derivation, Multi-Tenancy & Zero Trust

## 1. Purpose

This standard defines how APIs must derive user identity and tenant context within a **Strict Zero Trust Architecture (ZTNA)**.

Zero Trust assumes the internal network is hostile, therefore:

- **Identity must always come from a cryptographically verified token**, and
- **Tenant context must always be explicitly provided and validated**, not inferred.

This prevents spoofing, privilege escalation, cross-tenant leakage, and ambiguous authorization behavior across APIs and services.

---



## 2. Core Identity Principle

**Identity MUST come from the verified authentication token.**

Backends must NOT accept identity hints from:

- Request bodies
- Custom or forwarded headers
- Query parameters
- Emails supplied by the caller

The token is the **single source of truth** for actor identity.

Actor identity (who is performing the action) is derived from token claims such as:

- sub (subject identifier — REQUIRED)
- email (optional contact)
- scp / scope
- roles / groups (optional)

Notification recipients (e.g., notification\_email, manager\_email) may appear in the request body, but they **never serve as acting-user identity**.

---

## 3. Allowed vs Forbidden Identity Scenarios

Scenario	Allowed?	Reason
<b>A. "Notify Me"</b> – Acting user wants an email	<b>FORBIDDEN</b>	System must derive acting user's email from token. Caller cannot override identity.
<b>B. "Notify Boss / Another Party"</b>	<b>ALLOWED</b>	Additional recipients are not identity. Body fields allowed when clearly named.
<b>C. "Act on My Behalf (Self)"</b>	<b>FORBIDDEN</b>	Actor must come from token's sub. No identity in request payload.
<b>D. "Admin Acting on Behalf of End User"</b>	<b>SPECIAL CASE</b>	Requires delegated authorization (RFC 8693 act claim). Backend validates both admin & represented user.

Scenario	Allowed?	Reason
E. "Partner Acting on Behalf of Customer"	SPECIAL CASE	Allowed only with delegated access tokens or JWT assertions. Backend validates partner → customer linkage via scopes & delegation rules.
F. "SaaS / Automation App Acting on Behalf of User"	SPECIAL CASE	Requires delegated user-consent tokens. Backend validates app identity, user identity, and granted scopes.

## 4. Gateway Requirements (Defense in Depth)

Gateways MUST:

### 1. Validate

- Perform preliminary signature + expiry checks
- Reject invalid tokens early

### 2. Propagate

Forward the **original** Authorization: Bearer <token> header unchanged.

### 3. Sanitize

Strip all caller-provided identity headers:

- X-User
- X-Email
- X-Authenticated-User
- X-Acting-User

Gateways may add diagnostic headers, but they are **non-authoritative**.

## 5. Backend Requirements (True Zero Trust)

Backends MUST independently validate:

### 1. Token authenticity

Signature, issuer, audience, expiry, and required claims.

### 2. Actor identity

Derived only from the token (sub, optional claims).

### 3. Delegated identity (acting on behalf of)

If token contains delegated identity (e.g., act, RFC 8693):

- Validate both the acting identity and represented identity
- Validate permitted delegated scopes
- Enforce tenant and resource boundaries

### 4. Tenant boundaries

The backend must verify the caller is authorized to act *in the tenant they specify*.

### 5. No identity in payload

Identity hints in request bodies must be ignored and should not be present in compliant APIs.

---

## 6. Multi-Tenant Identity & Authorization

Multi-tenant architectures allow users, partners, and systems to interact with multiple organizations or accounts.

Zero Trust requires **explicit tenant selection** and **verifiable tenant authorization** for every request.

Identity = Who

Tenant = Where / On behalf of whom

Both must be provided independently.

---

# 6.1 Identity vs Tenant: Critical Distinction

## Identity (actor) — provided by token

- sub
- email
- scp
- roles / groups
- partner\_id (optional)

## Tenant (context) — provided by the request

- tenant\_id
- org\_id
- account\_id

**Identity tells us who is calling.**

**Tenant tells us where they are acting.**

They MUST NOT be collapsed into each other.

---

# 6.2 Required Token Claims for Multi-Tenant Context

To enforce tenant-specific authorization, tokens MUST include:

Claim	Purpose
sub	Who the actor is
scp / scope	What actions are allowed
tenant_membership ( <b>custom</b> )	List of tenants the user has access to (recommended)
act ( <b>delegated identity</b> , RFC 8693)	For acting on behalf of another user/tenant
partner_id ( <b>custom</b> )	Identifies partner organization (if applicable)
azp	Authorized party for SaaS/on-behalf-of flows

---

## 6.3 Explicit Tenant Selection (Mandatory)

The API MUST require the caller to explicitly specify the tenant they are acting in, e.g.:

```
/tenants/{tenant_id}/orders
POST /orders?tenant_id=12345
```

### Why must the tenant be provided explicitly?

1. **Users often belong to multiple tenants** (consultants, partners, admins).
2. **Tokens do not represent tenant intent**, only identity.
3. **Inference creates ambiguity** and breaks Zero Trust.
4. **Explicitness prevents cross-tenant privilege escalation.**
5. **Auditing demands clear, deliberate tenant selection.**
6. **Identity  $\neq$  Tenant**, and must not imply tenant context.
7. **Delegated access requires explicit “on behalf of” tenant resolution.**

**Explicit tenant selection is the only way to guarantee correct authorization in multi-tenant APIs.**

---

## 6.4 Tenant Cannot Be Inferred

The tenant MUST NOT be inferred from:

- “default tenant” settings
- email domain
- token issuer or IdP tenant
- resource IDs
- internal lookup
- user profile metadata

### Why inference is banned

- Creates unintended privilege escalation
  - Causes operations on the wrong tenant
  - Breaks federated identity expectations
  - Violates Zero Trust: **Assume nothing. Validate everything.**
  - Inconsistent behavior across APIs
  - Impossible to audit reliably
-

## 6.5 Tenant Membership Validation

Once tenant is explicitly provided, backend MUST verify:

1. Caller is mapped to that tenant
  2. Caller has appropriate scope for that tenant
  3. Delegation or admin privileges (if cross-tenant)
  4. Access is logged for audit
  5. If any check fails → **403 Forbidden**
- 

## 6.6 Cross-Tenant Access

Cross-tenant access is **never permitted by default**.

It is only allowed via **delegated authorization** or **elevated admin scopes**.

### Pattern 1 — Delegated Access (“Acting on behalf of Customer”)

Used for:

- Partners
- Resellers
- SaaS integrations
- Bots / automation systems

Required:

- Token with act **claim** (RFC 8693)
- Delegated scopes granting partner → customer access
- Optional constraints like `partner_id`, `customer_id`

Backend validates:

- Partner identity
  - Customer identity
  - Delegation chain
  - Allowed scopes
  - Tenant boundaries
- 

### Pattern 2 — Admin Scopes (Internal elevated access)

Used by:

- Support teams

- NOC / operations
- Internal engineering tools

Required:

- Token containing admin-level scopes (e.g., `tenant.admin`, `customer.manage`)
- Optional delegated identity (act) if impersonation occurs

Backend validates:

- Admin's identity
- Admin privileges
- Tenant scope
- Operation type matches admin rights

## 6.7 Multi-Tenant Scenarios

Scenario	Allowed?	Reason
<b>MT-1: User acts in tenant they belong to</b>	✓ ALLOWED	Tenant is explicit; membership & scopes validated.
<b>MT-2: User attempts to act in tenant they do NOT belong to</b>	✗ FORBIDDEN	Tenant isolation violation; return 403.
<b>MT-3: Internal Admin acts across tenants</b>	⚠ SPECIAL CASE	Allowed only with admin scopes/delegation.
<b>MT-4: Partner acts on behalf of Customer</b>	⚠ SPECIAL CASE	Must use delegated tokens (RFC 8693) and authorized scopes.
<b>MT-5: SaaS app acts on behalf of end-user</b>	⚠ SPECIAL CASE	Requires delegated user-consent token and validated authorization chain.

## 7. PII & Email Handling

### Actor Email

- Always derived from token
- Never supplied in payload
- If wrong → user updates profile in IdP

### Notification Emails

Allowed only for contacts that are **not the actor**, e.g.:

- manager\_email
  - approver\_email
  - notification\_email
- 

## 8. Governance as Code (Spectral Rule)

```
rules:
 no-acting-user-in-body:
 description: "Acting user identity must be derived from the authentication token, no
 severity: error
 given: "$.paths[*][*][?(@.security)]..requestBody.content.*.schema.properties"
 then:
 field: "@key"
 function: pattern
 functionOptions:
 notMatch: "^(user_email|actor_email|acting_user|customer_email|user_id)$"
```

---

## 9. Migration Plan

Phase	Action
1. Discover	Identify APIs containing acting user identity in payload.
2. Backend Update	Switch identity derivation to token.
3. Contract Update	Remove identity fields; publish new versions.
4. Deprecate	Follow COM/API deprecation lifecycle.
5. Hard Gate	Gateway blocks requests that include forbidden identity fields.

---

## 10. Final Summary

- Identity = Token**
- Tenant = Explicit**
- Delegation = RFC 8693 or Admin Scopes**
- Notification ≠ Identity**
- Headers = Untrusted**
- Zero Trust = Verify at Every Layer**



# API Governance Process (Integrated with SDLC)

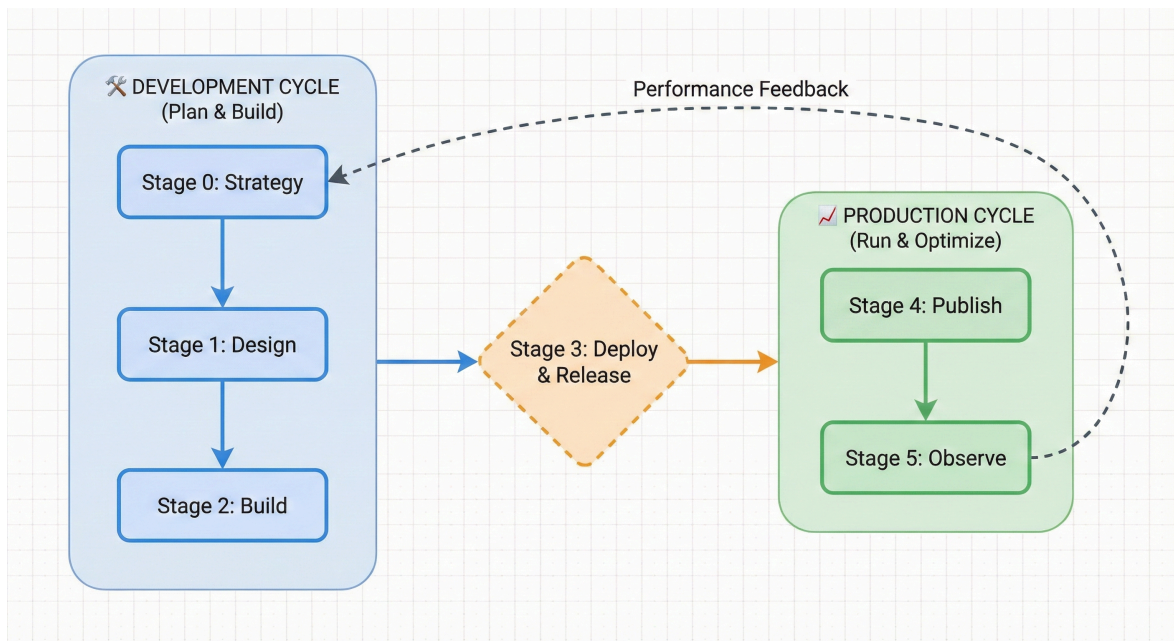
## API Governance Process (Integrated with SDLC)

### Executive Summary

This framework establishes a comprehensive API governance process that integrates with the Software Development Lifecycle (SDLC), incorporating industry-leading practices and forward-thinking approaches for API-as-a-Product management. It emphasizes automation, self-service, observability, and continuous improvement while ensuring security, consistency, and strategic alignment.

### Core Principles

- **Shift-Left Everything:** Security, quality, and compliance checked early
- **Automate First:** Manual governance only where human judgment adds value
- **API-as-a-Product:** APIs are products with lifecycle, customers, and metrics
- **Developer Experience (DX):** Minimize friction, maximize self-service
- **Continuous Governance:** Not gating, but guiding and measuring



Explore the detailed phases of the API Lifecycle:

# Governance Lifecycle Stages

- [Enterprise API Governance & Publishing Architecture](#)
- [API Governance: Operating Model](#)
- [API SDLC Stage 0: Discovery & Strategy](#)
- [API SDLC Stage 1: Planning & Design](#)
- [API SDLC Stage 2: Implementation & Testing](#)
- [API SDLC Stage 3: Deployment & Release](#)
- [API SDLC Stage 4: Publication & Enablement](#)
- [API Governance: Operations & Continuous Improvement](#)
- [API Governance: Reporting & Exception Management](#)
- [API Governance: Repository Onboarding - Bootstrap Structure and Check-In Paths](#)

## Enterprise API Governance & Publishing Architecture

### 1. Executive Summary

This document outlines the target architecture for the API Lifecycle, moving the organization to a **Centralized Registry & Governance Model**.

In this architecture, the **API Catalog** acts as the Single Source of Truth for all API contracts. While ownership remains federated (teams own their domains), the physical specs reside in this structured Monorepo (`api-catalog`). This ensures **global discoverability**, **standard compliance**, and separates the **Interface Lifecycle from the Code Lifecycle**.

To manage this at scale, we introduce an "**Artifact-Based Distribution**" strategy:

- **Upstream (Governance):** Developers define contracts in the **API Catalog**, passing automated linter checks and manual governance gates based on the API's `lifecycleStage`.
- **Handover (Nexus):** Approved specifications and behavioral metrics are packaged into immutable versioned artifacts (Zips).
- **Downstream (Distribution):** Artifacts are automatically distributed to Portals for documentation and Observability Platforms for alerting.

### 2. Core Architectural Principles

- **Centralized Catalog (Inner Source):** All API specifications reside in the `api-catalog` repository. Teams submit Pull Requests to their specific Domain folders. This enforces a standardized directory structure across the enterprise.
- **Smart Governance Gates:** Review requirements are dynamic. Internal APIs pass with automated checks; Public APIs trigger a programmatic requirement for API Architect review based on `api-metadata.yaml`.

- **Immutable Artifacts:** We do not sync files directly via copy-paste. We publish versioned artifacts to Nexus. This ensures that what is verified in Non-Prod is exactly what is deployed to Prod.
- **The Operational Manifest Pattern:** Implementation details (SLAs, Alert Thresholds) are not merged into the public contract. They travel as a separate "Manifest File" (sla-config.yaml) alongside the OAS.

### 3. The Directory Structure (The Standard)

Regardless of the design tool used, artifacts must land in this structure to trigger the pipeline.

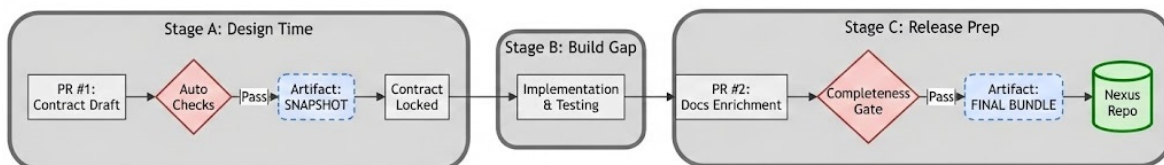
```

api-catalog/
├── domains/ # Root for all API Contracts
│ ├── mcgw/ # Domain Name
│ │ ├── v1/ # Major Version (Breaking Changes = New Folder)
│ │ │ ├── docs/ # Human-Readable Documentation
│ │ │ │ ├── API-CHARTER.md # Architecture & Plan (Stage 1)
│ │ │ │ ├── error-codes.md # Business Error Registry (Stage 2)
│ │ │ │ └── release-notes.md # Release Notes (Stage 2)
│ │ │ ├── openapi.yaml # The Interface Contract (OAS 3.x)
│ │ │ ├── api-metadata.yaml # Identity, Lifecycle, & Ownership
│ │ │ └── sla-config.yaml # Operational Manifest (Alerts/SLAs)
│ │ └── v2/ # Future iterations
├── reusable-fragments/ # Shared OAS Components
│ ├── domains/ # Domain-specific reusables
│ └── shared/ # Enterprise-wide standards
│ ├── headers/ # e.g., TraceContext.yaml
│ └── schemas/ # e.g., LpdError.yaml

```

### 4. Phase 1: The Governance Lifecycle (Upstream)

This phase occurs entirely within the Central Registry. We utilize a "Two-PR Strategy" corresponding to the Lifecycle Stages.



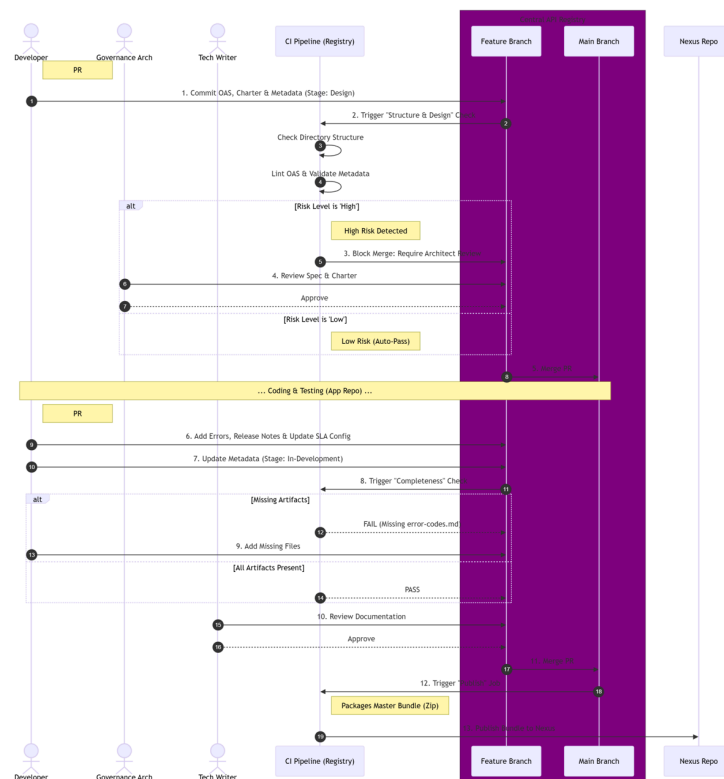
#### Stage A: Design Time (PR #1 - The Contract & Charter)

- **Trigger:** lifecycleStage: Design
- **Draft:** Developer creates a feature branch in the Registry and commits openapi.yaml, api-metadata.yaml, sla-config.yaml (Draft), and API-CHARTER.md.

- **Version Declaration:** The Developer MUST set the `info.version` in `openapi.yaml`. This value determines the artifact version.
- **Automated Checks:**
  - *Structure Check:* Validates files are in `domains/{domain}/v{n}/`.
  - *Linting:* Spectral linting runs on OAS.
  - *Smart Gate:* The CI pipeline parses `api-metadata.yaml`.
    - *Auto-Pass:* If `riskLevel: Low`.
    - *Mandatory Review:* If `riskLevel: High` or `audience: Public`, the pipeline adds the Governance Architect as a required reviewer.
    - **Snapshot Publishing (Parallel Support):** Upon passing checks, the pipeline publishes a **Snapshot Artifact** (e.g., `inventory-1.2.0-SNAPSHOT.zip`) to the Nexus Dev Repository.
- *Note:* This allows the **Test Environment** to consume `v1.2.0-SNAPSHOT` while **Production** continues to consume the stable `v1.1.0` artifact safely.
- **Contract Lock:** Once approved, PR #1 is Merged.

## Stage B: Implementation Gap

- **Coding:** Implementation occurs in the Application Repository. The Developer builds against the locked contract.
- **Hydration:** The application build process extracts the version from the `openapi.yaml` to populate the runtime header, ensuring code and contract match.



## Stage C: Release Preparation (PR #2 - The Bundle)

- **Trigger:** Developer manually updates lifecycleStage to Active.
- **Artifact Enrichment:** The Developer adds the implementation details (error-codes.md, release-notes.md, sla-config.yaml).
- **Completeness Gate:** The pipeline detects the target stage is Active. It strictly enforces that all documentation files are present and that the info.version has not been decreased.
- **Publishing (The Release Event):** Upon merging PR #2, the pipeline performs the Version & Release Routine:
- **Tagging:** It creates a Git Tag matching the version (e.g., inventory-v1.2.0) to mark the history.
- **Packaging:** It bundles all files into the final Service Bundle.
- **Distribution:** It pushes the immutable artifact (e.g., inventory-1.2.0.zip) to the Nexus Release (Prod) Repository.

## 5. Maintenance & Hotfix Workflow (Addressing Parallel Versions)

Since the Registry main branch always tracks the latest development (e.g., v1.2), we utilize **Long-Running Support Branches** to support older versions (e.g., v1.1) running in Production without cross-contamination.

 **Architectural Note: The "Support Branch" Pattern** This workflow follows the industry-standard **Maintenance Branch** strategy (distinct from ephemeral "hotfix" branches).

- **Purpose:** To maintain legacy major versions (e.g., v1.x) in parallel with active development (v2.x).
- **Isolation:** These branches (support/v1.1) **never** accept merges from main. They only accept cherry-picked patches. This guarantees that no new features accidentally leak into the production hotfix.

**Scenario:** Prod is on v1.1.0. Test is working on v1.2.0. We need a critical hotfix for Prod (becoming v1.1.1).

1. **Establish Maintenance Branch:** (One-time setup per minor version)
2. If it doesn't exist yet, we create a long-living support branch from the release tag:
3. `git checkout tags/inventory-v1.1.0 -b support/v1.1`
4. **Create Patch:** The developer creates a short-lived fix branch from that support base:
5. `git checkout support/v1.1 -b patch/v1.1.1`

6. **Apply Fix:** Developer modifies `openapi.yaml` (or documentation), increments the version to `1.1.1`, and commits.
7. **Publish Hotfix:** The PR targets the `support/v1.1` branch (NOT `main`).
8. **Isolation:** Since `main` is never merged into `support/v1.1`, this branch contains *only* `v1.1` code. There is zero risk of leaking `v1.2` features.
9. The pipeline detects a change on a `support/*` branch, runs governance checks, and publishes `inventory-1.1.1.zip`.
10. **Re-Integrate (Mandatory Cherry-Pick):**
11. **The Constraint:** We **CANNOT** merge `support/v1.1` into `main` because it would overwrite new `v1.2` features.
12. **The Action:** If the bug also exists in `v1.2`, the developer **MUST cherry-pick** the specific commit from the patch branch and apply it to `main`.
13. **Outcome:** The bug is squashed in both the current production version (`v1.1.1`) and the future release (`v1.2.0`).

#### Result:

- **Prod:** Updates dependency to consume `inventory-1.1.1.zip`.
- **Test:** Continues using `inventory-1.2.0-SNAPSHOT.zip`.
- **Nexus:** Holds all three versions (`1.1.0`, `1.1.1`, `1.2.0-SNAPSHOT`) simultaneously.

## 6. Phase 2: The Distribution Strategy (Downstream)

Once the artifact is secured in Nexus, the "Publish Once, Pull Anywhere" strategy begins.

The Service Bundle (Artifact Structure)

The Zip file in Nexus contains:

- `openapi.yaml` (The Interface)
- `api-metadata.yaml` (The Identity)
- `error-codes.md` (The Docs)
- `release-notes.md` (The Value)
- `API-CHARTER.md` (The Plan)
- `sla-config.yaml` (The Ops Manifest)

#### Consumer 1: Narnia Pipeline (External Portal)

- **Logic:** Downloads Zip --> Sanitizes (Removes Charter & SLA Config) --> Publishes `openapi.yaml` and `docs/*.md` to the External Developer Portal.

#### Consumer 2: Backstage Pipeline (Internal Portal - Future State)

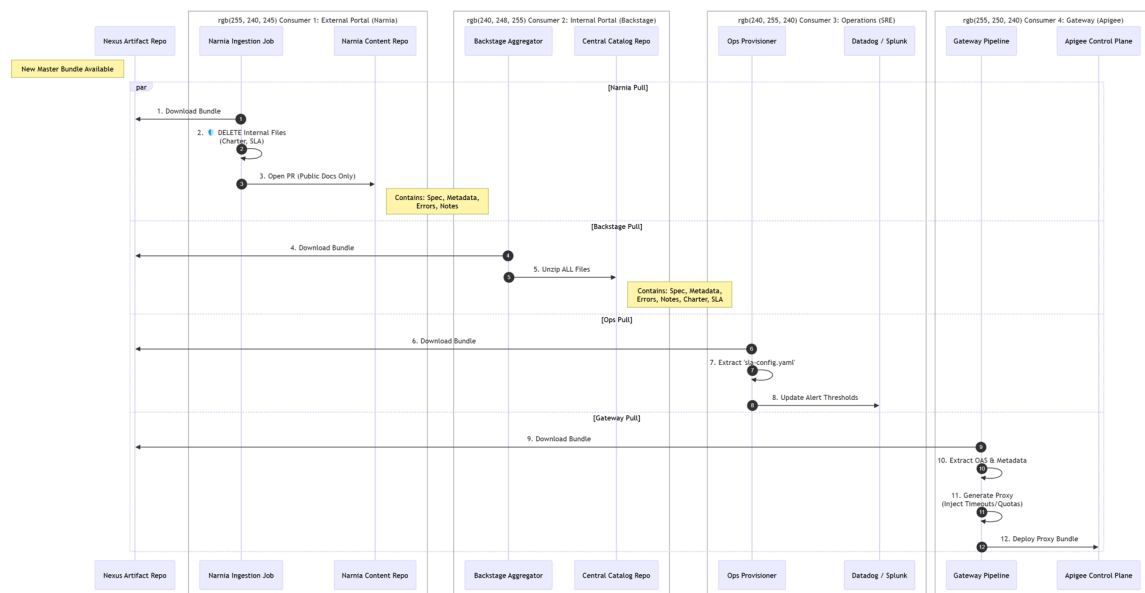
- **Logic:** Downloads Zip --> Extracts ALL files --> Renders full context (including internal architecture docs) to the Internal Catalog.

### Consumer 3: Ops/SRE Pipeline (Observability - Future State)

- **Logic:** Downloads Zip --> Extracts only `sla-config.yaml` --> Applies Alerting Rules to AppDynamics/Prometheus via API.

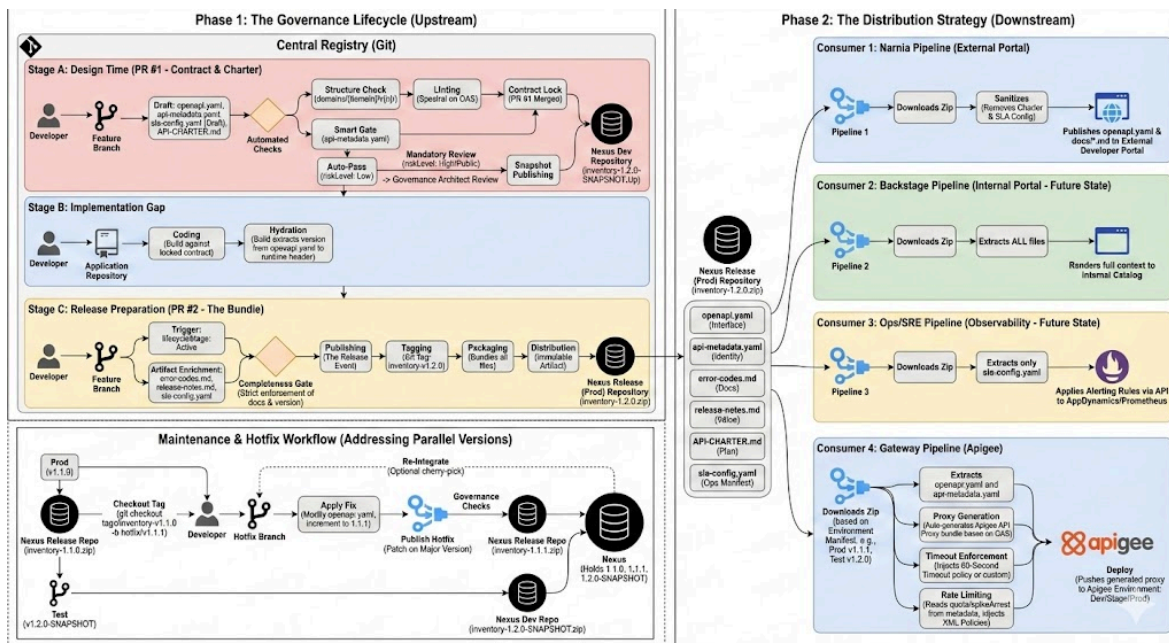
### Consumer 4: Gateway Pipeline (Apigee)

- **Responsibility:** Runtime Configuration & Traffic Management.
- **Logic:**
  - Downloads Zip based on the **Environment Manifest** (e.g., Prod requests v1.1.1, Test requests v1.2.0).
  - Extracts `openapi.yaml` and `api-metadata.yaml`.
  - **Proxy Generation:** Auto-generates the Apigee API Proxy bundle based on the paths and verbs in the OAS.
  - **Timeout Enforcement:** Injects the standard 60-Second Timeout policy (or custom value if an Exception is logged in metadata).
  - **Rate Limiting:** Reads the quota or `spikeArrest` values from `api-metadata.yaml` and injects the corresponding XML Policies into the Proxy PreFlow.
  - **Deploy:** Pushes the generated proxy to the Apigee Environment (Dev/Stage/Prod).



## End-to-End Architecture: Governance & Distribution





## API Governance: Operating Model

### Governance Bodies

#### API Steering Committee

##### (Formerly API Governance Board)

- **Purpose:** Investment approval, strategic alignment, and final authority.
- **Composition:** Chief Architect, Product Leadership (VP Level), Platform Engineering Lead, CISO/Security Lead.
- **Cadence:** Quarterly strategic reviews + Monthly "Investment Gate" (for Stage 0 decisions).
- **Responsibilities:**
  - **Investment Decision (Stage 0):** Issue the final **GO / NO-GO** decision for new API products based on the Business Case.
  - **Ratify Strategy:** Review and approve API principles and standards drafted by the Centralized Team.
  - **Strategic Alignment:** Ensure API investments align with high-level business goals.
  - **Escalation Authority:** Final arbiter for blocking issues or governance exceptions that carry significant risk.

#### Centralized API Governance Team (Core Team) - *The Execution Engine*



## (Formerly API Governance Working Group)

- **Purpose:** Operational execution, drafting standards, tooling, and registry integrity.
- **Composition:** Full-time Governance Leads, API Architects, Developer Experience Engineers.
- **Cadence:** Daily/Weekly operational execution.
- **Responsibilities:**
- **Strategy & Principles (Drafting):** Research, define, and draft API principles and standards for CoE approval.
- **Registry Management:** Own the "Single Source of Truth" and ensure metadata quality.
- **Manual Reviews:** Conduct deep-dive reviews for high-risk, public-facing, or complex APIs that automation cannot cover.
- **Consultation:** Provide "Office Hours" support to Domain teams during early design phases.
- **Governance Automation Platform:** Build, maintain, and evolve the core platform capabilities, including:
  - **Internal API Repository:** A centralized catalog (e.g., Backstage, Confluence) acting as the searchable "Single Source of Truth" for all API assets.
  - **Duplicate Detection Engine:** Automated tools (semantic search, schema comparison) to identify overlapping or redundant functionality during Stage 0.
  - **Policy-as-Code Engine:** Automated policy enforcement (Spectral, custom rules) to block non-compliant specs.
  - **Design-Time Governance:** IDE plugins, CLI tools, and pre-commit hooks to catch issues early.
  - **Governance Dashboard:** Real-time visibility into portfolio health and compliance metrics
  - **AI-Assisted Review:** ML-powered design suggestions and anomaly detection to augment manual reviews.

## Domain API Councils (Future State)

- **Purpose:** Domain-specific governance within overall standards.
- **Composition:** Domain architects, product owners, and engineering leads per domain.
- **Cadence:** Bi-weekly domain sessions.
- **Responsibilities:**
- Ensure domain model consistency (e.g., "What is a 'Customer' in our domain?").
- Cross-team API coordination within the domain.
- Define domain-specific patterns and conventions (within global guardrails).
- Handle local exceptions and internal peer reviews.

## API Community of Practice

**Purpose:** Drive adoption and continuous improvement by connecting API producers and consumers to discuss tooling gaps, developer friction, and practical implementation challenges.

**Composition:** Open to all API developers, QA Engineers, Domain Architects, and representatives from the Centralized Core Team.

**Cadence:** Monthly knowledge-sharing sessions + Async collaboration channels (Slack/Teams).

### Responsibilities:

- **Governance Tuning:** Act as the "Voice of the Developer" to provide feedback to governance
- **Gap Identification:** Identify missing patterns, shared capabilities, or tooling needs that the Centralized Team should prioritize.
- **RFC Review:** Provide "sanity check" feedback on new standards proposed by the Core Team before they are ratified.
- **Knowledge Sharing:** Showcase internal success stories, reusable libraries, and inner-source solutions to reduce duplication across domains.

## API SDLC Stage 0: Discovery & Strategy

### Overview

**"Do we need this API, and does it align with our business strategy?"**

Stage 0 is the Business Validation phase. It occurs before technical design or coding begins. Its primary purpose is to prevent "API Sprawl" by ensuring every new API has a clear business value, a defined consumer, and does not duplicate existing capabilities.

- **Trigger:** Business need identified (e.g., "Partners need real-time inventory access").
- **Primary Output:** API Product Brief (Approved).
- **Outcome:** Decision to **GO** (Move to Stage 1: Design), **NO-GO**, or **DEFER**.

---

### Roles & Responsibilities (RACI)

Role	Responsibility	Focus Area
Product Owner (PO)	Accountable	Owns the "Why" and the "Value." Writes the Brief.

<b>Business Stakeholder</b>	<b>Consulted</b>	Provides budget approval and ROI estimates.
<b>API Steering Committee</b>	<b>Informed</b>	Tracks the portfolio health and enforces standards.
<b>API Architecture Agent (AI)</b>	<b>Consumer</b>	<b>Ingests this brief to auto-generate the Draft OAS, Schemas, and Mock Servers.</b>

## Key Activities

### Business Case Development

The PO articulates the problem statement. They must move beyond "We need an API" to "We need to solve X problem for Y consumer."

- **Action:** Draft the API Product Brief.
- **Key Question:** Who will call this API, and why?

### Portfolio Analysis

**Before any design work starts, the PO must verify this capability doesn't already exist.**

- **Action:** Search the API Registry/Catalog.
- **Key Question:** Does a legacy service or another API already offer this data?
- **Outcome:**
  - *Green Field:* No existing service; proceed to build.
  - *Wrapper:* Legacy SOAP service exists; build a REST Experience Layer on top.
  - *Duplicate:* API exists; project is rejected or redirected to use existing asset.

### Governance & Feasibility

The PO and Solution Architect meet to classify data, identify dependencies, and **validate strict domain boundaries**

- **Action:** Determine Data Classification (Public, Internal, PII).
- **Action:** Identify critical dependencies (e.g., "Depends on SAP migration").
- **Action:** Verify Domain Alignment .
- *Objective:* Ensure this API does not expose capabilities that belong to another domain (preventing "God Services").
- *The Rule:* This Domain must be the **System of Record** for the action being performed.
- *Validation Questions:*

- "Does this capability modify state that is legally owned by another team?" (e.g., Inventory API trying to Create\_Customer).
  - "Is this a convenience function that duplicates logic existing elsewhere?" (e.g., Inventory API calculating Tax instead of calling the Tax Service).
  - *Anti-Pattern Check:*
  - **Inventory Domain** exposing Process\_Payment -->  **REJECT** (Belongs to Billing).
  - **Inventory Domain** exposing Reserve\_Stock -->  **APPROVE**.
- 

## Governance Gates (The Checklist)

The API Steering Committee checks these items before approving the move to the next stage.

-  **Strategic Fit:** The API aligns with current business goals.
  -  **Uniqueness:** Confirmed no duplicate functionality exists in the Registry.
  -  **AI Readiness:** "Nouns" (Entities), "Verbs" (Capabilities), and "Outcomes" sections use the required **Table Format** (no paragraphs).
  -  **Visibility Check:** Internal vs. Public fields are explicitly tagged.
  -  **Domain Ownership:** A clear Domain (e.g., Inventory) is assigned.
  -  **Data Classification:** Data sensitivity is identified (triggers InfoSec engagement if PII/PCI).
  -  **Value Defined:** Success metrics (KPIs) are documented.
- 

## Tooling & Storage

To maintain a "Single Source of Truth," we avoid scattered Word docs or emails.

- **System of Record:** Confluence (API Governance Space).
  - **Artifact:** Native Confluence Page using the **Structured Data Template** (Below).
  - **Tracking:** The page must use the Page Properties Macro to populate the central Governance Dashboard.
  - **Linkage:** If approved, a Jira Epic is created and linked to this Confluence page.
- 

# API PRODUCT BRIEF (Stage 0 — Discovery & Strategy)

System of Record: Confluence

Owner: Product

Purpose: Validate business need, define value, and capture structured business semantics that downstream automation can transform into API design artifacts.

Meta Information

Property	Value (Example)
Status	● Draft
Proposed API Name	Inventory Availability & Reservation API
Business Domain	Inventory
Product Owner	@ABC
Business Stakeholders	Merch Ops, Partner Integration Team
Target Launch	Q3 2025
Funding Source	Program X
Data Sensitivity Classification	Internal
Related Initiatives	Partner Portal Modernization
Version	v0.1

Executive Summary (Narrative)

Example:

Partners and internal systems need real-time access to product availability and the ability to reserve items before checkout. This API will expose accurate stock levels and support short-term reservations to prevent overselling.

The Business Case

The Problem (“Why Now?”)

Example:

- Partners currently rely on daily batch exports for stock levels.
- This leads to overselling, customer complaints, and 15% cart abandonment.
- New strategic partner requires real-time access for integration.

The Solution (“The What”)

Example:

Provide a real-time API that allows partners and internal applications to:

- Check inventory availability.

- Reserve items temporarily.
- Release reservations.

Success Metrics (KPIs)

KPI	Target (Example)
Reduce stock-related support tickets	–20% by Q4
Partner adoption	5 partners by Q2
Real-time inventory response time	< 200ms (P95)

Business Entities (Nouns / Data Models)

Instructions for PO: List the core objects. Use the "Visibility" column to flag Internal-only fields (e.g., backend multipliers, legacy IDs) so the AI excludes them from the public API schema.

Entity Name	Field Name	Data Type	Visibility	Required?	Validation / Constraints
InventoryItem	sku	String	Public	Yes	Max 20 chars, Alphanumeric
InventoryItem	qty_available	Integer	Public	Yes	Min: 0
InventoryItem	cost_multiplier	Float	Internal	No	Backend logic only.
Reservation	reservation_id	String	Public	Yes	UUID (Read Only)
Reservation	expires_at	DateTime	Public	Yes	ISO8601 Standard

Business Capabilities (Verbs / API Resources)

Instructions for PO: Map user actions to system resources. Explicitly define the Parent Resource to ensure the AI generates the correct URL hierarchy (e.g., /warehouses/{id}/inventory vs. /inventory).

Actor	Action	Parent Resource	Target Entity	Input Data	Purpose / Rule
Partner System	Check	Warehouse	InventoryItem	SKU	Returns quantity available.
Partner System	Reserve	N/A (Root)	Reservation	SKU, Qty	Reservation requires

					available quantity > 0.
Internal Ops	<b>Release</b>	Reservation	Reservation	Res. ID	Used when orders are canceled.

### Business Rules & Constraints

Rule / Constraint	Description (Example)
<b>Reservation Timeout</b>	Expires after 15 minutes if not converted to an order.
<b>Uniqueness Rule</b>	SKU must be unique across warehouses.
<b>Minimum Stock Threshold</b>	Reservations not allowed when quantity_available < 1.

### Expected Outcomes (Success & Failure Scenarios)

*Instructions for PO: Define specific error behaviors. The AI uses the HTTP Code column to generate the OAS responses section.*

Scenario	Outcome	User Message (Example)
SKU not found	Request rejected	"Item does not exist."
Insufficient quantity	Reservation denied	"Not enough stock available."
Reservation created	Success	"Reservation confirmed. Expires in 15 min."
Inventory system offline	Soft failure	"Real-time data unavailable; using cached values."

### States & Lifecycle (Optional)

State	Description (Example)
active	Reservation currently holding quantity.
expired	Reservation timed out.
canceled	Released by system or admin.

### Events / Triggers (Optional)

Event	Description	Expected Behavior (Example)
-------	-------------	-----------------------------

<b>Inventory Updated</b>	Stock adjusted in ERP	API reflects new quantity within 1–2 seconds.
<b>Reservation Expired</b>	Timer event	Automatically release quantity.
<b>Order Completed</b>	Checkout system event	Reservation converted to permanent allocation.

## Strategic Alignment

Fit with Company Strategy

Check all that apply:

- ☐ Customer Experience
- ☐ Efficiency & Cost Reduction
- ☐ New Revenue
- ☐ Compliance
- ☐ Digital Transformation

**Explanation:** Supports real-time omnichannel experience and partner integration roadmap.

### Risks of NOT Doing This

- Loss of new partner contract.
- Continued overselling.
- Customer churn.
- Support costs remain high.

## Governance & Feasibility Check

### Portfolio & Reusability Check

Check	Result	Example Notes
Duplicate API exists?	No	Legacy SOAP exists but too slow.
Overlapping services?	Yes	Existing Inventory Feed (batch), but not real-time.
Domain ownership clear?	Yes	Owned by Inventory domain.

### Build vs Buy

Option	Recommendation	Example Reason
<b>Build</b>	✓	Real-time inventory is core IP.
<b>Buy</b>		No suitable vendor.



Reuse		Legacy layer insufficient.
-------	--	----------------------------

## Dependencies & Assumptions

Dependency	Description	Risk / Impact Example
ERP System	Provides stock levels	ERP latency may delay updates.
Partner SSO	Required for access	Integration complexity.
Order Service	Converts reservations	Dependency on Q2 rollout.

## High-Level Estimates (ROM)

Estimate	Value (Example)
T-Shirt Size	M
Team Needs	1 BE Dev, 1 QA, 20% DevOps
Cost Estimate	\$250k
Expected ROI	High partner adoption → +\$2M GMV

## Decision Log

Date	Decision	Approver	Notes (Example)
03/10/2025	GO	@Director API Strategy	Proceed to Stage 1 (Design)

# API SDLC Stage 1: Planning & Design

## Overview

**"How will we build this, what is the contract, and is it safe?"**

Stage 1 consolidates Planning, Design, and Review into a single, continuous workflow.

The Development Team defines the technical charter (Pattern, Security), the API Specification (OpenAPI), and the Operational Profile (SLA Config) in a feature branch. The Pull Request (PR) serves as the single Quality Gate where Automation, Security, and Architecture reviews converge.

- **Trigger:** Stage 0 (Brief) Approved.
- **Input:** PR containing the API Charter, OpenAPI Spec, and SLA Config.

- **Output:** A merged PR containing the approved design artifacts.

## Roles & Responsibilities (RACI)

Role	Responsibility	Focus Area
Development Team	Accountable	Writes Charter, designs Spec, <b>defines SLAs</b> (sla-config.yaml), mocks API, opens PR.
Solution Architect	Responsible	Reviews pattern alignment, domain modeling, and <b>feasibility of SLA targets</b> .
Security Engineer	Responsible	Reviews auth design, threat model, and data handling.
Governance Platform	Automated	Lints specs, validates schema compliance, generates mocks.
Domain Council	Consulted	Ensures alignment with domain vocabulary (DDD).

## Key Activities

### 1. Setup & Intake

The Development Team initiates the work by creating a new branch in the repository.

- **Action:** Create a branch (e.g., design/feature-name).
- **Tooling:** Repo is auto-provisioned with CI/CD pipelines pre-configured.

### 2. Technical Chartering (The Plan)

The team creates API-CHARTER.md.

- **Decisions:** Pattern selection (REST vs Event) and Versioning strategy.
- **Security:** Define Scopes (e.g., read:inventory) and Auth flows (OAuth2).
- **Domain:** Define Bounded Contexts and Aggregates.

### 3. Operational Profiling (The SLA)

The team creates sla-config.yaml.

- **Performance:** Define latency and availability targets.
- **Alerting:** Define threshold rules and notification channels (e.g., PagerDuty vs Slack).
- **Metrics:** Define custom business metrics to be instrumented.

### 4. Contract Design (The Spec)

The team authors the `openapi.yaml` (or AsyncAPI/GraphQL schema).

- **Contract-First:** Write YAML before code.
- **Error Handling:** Design standard error schemas.
- **Domain Modeling:** Align schema objects with Domain Aggregates.

## 5. Submission via Pull Request

When design is complete and local linting passes, the developer opens a PR titled: "Design Review: [API Name]".

### Governance Gates (The PR Review)

The PR triggers a tiered review process.

#### Tier 1: Automated Review (Blocking)

Immediate feedback via CI/CD. Must pass to proceed.

- **Linting:** Spec conforms to Corporate Style Guide (Spectral).
- **Completeness:** Documentation score > 80% (Descriptions/Examples present).
- **Drift Detection:** Checks if schema contradicts the Domain Canonical Model.
- **SLA Validation:** Checks if `sla-config.yaml` exists and conforms to schema.

#### Tier 2: Security Review (Async - Optional)

Triggered if automated checks pass.

-  **Auth Design:** Is the mechanism (OAuth2/OIDC) appropriate for the Data Class?
-  **Granularity:** Are scopes sufficiently granular (Least Privilege)?
-  **Threat Modeling:** Automated STRIDE analysis (or manual for High Risk).
-  **Data Protection:** Encryption and PII handling requirements met.

#### Tier 3: Governance & Architecture Review (Async - Optional)

Triggered if automated checks pass.

-  **Pattern Match:** Is REST/Event/GraphQL the correct choice for this use case?
-  **Domain Alignment:** Correct Bounded Context and vocabulary usage.
-  **Consumer Experience:** Is the design intuitive?
-  **Operational Feasibility:** Are the targets in `sla-config.yaml` realistic given the architecture?

### Review Types (Risk-Based Routing)

Refer to `api-metadata.yaml` (Section: Risk & Routing) for the definition of Risk Levels and Review Routing logic.

# Exception Handling

If a team cannot meet a standard:

- 1. **Request:** Developer adds exception label to PR with business justification in comments.
- 2. **Review:** Governance Lead reviews the request and presents the case to the API Center of Excellence (CoE) for a final decision.
- 3. **Registry:** Exception is logged in the "Exception Registry" for future tech debt cleanup.

## Tooling & Storage

- **System of Record:** Git (Source) + API Registry (Metadata/Catalog).
  - **Artifacts:**
    - specs/openapi.yaml (The Contract)
    - docs/API-CHARTER.md (The Narrative)
    - sla-config.yaml (The Operational Manifest)
    - api-metadata.yaml (The Discovery Metadata)
- 

## API Technical Charter Template

### API Technical Charter

Purpose: Defines the architectural decisions, security model, and domain alignment of the API.

Mandatory For: All API Design Pull Requests.

#### 1. General Metadata

Refer to api-metadata.yaml for comprehensive ownership details.

Property	Value	Description
Target Release	[Q3 2025]	Estimated launch date.

#### 1. Architectural Design

##### 2.1 Pattern Selection

Decision Category	Selection	Justification
Interface Style	[REST / GraphQL / gRPC]	Why was this chosen over others?

<b>Interaction Style</b>	[Sync / Async / Batch]	e.g., Request/Response vs Event-Driven.
<b>Versioning Strategy</b>	[URI Path / Header]	Must align with Corp Standards.

## 2.2 Domain Alignment (DDD)

- **Bounded Context:** [e.g., Inventory Context]
- **Aggregates Exposed:** [e.g., StockItem, Reservation]
- **Drift Check:**
  - ☐ No, strictly adheres to Domain Model.
  - ☐ Yes, proposing new terms: [List terms]
- **Security & Data Privacy**

## 3.1 Authentication & Authorization

- **Auth Mechanism:** [e.g., OAuth2 via OIDC]
- **Identity Provider:** [e.g., Okta / Azure AD]

## 3.2 Permission Scopes (Least Privilege)

Scope Name	Description	User Persona
inventory:read	Allows reading stock levels.	Viewer, Partner
inventory:write	Allows adjusting stock counts.	Admin, System Svc

## 3.3 Data Handling

- **Highest Classification:** [Confidential] (PII / PCI / Internal)
- **PII Involved?** [Yes/No] (List fields if Yes).
- **Encryption (Transit):** [TLS 1.3]
- **Encryption (Rest):** [AES-256]
- **Operational Profile**
- **Performance & Alerting:** Defined in `sla-config.yaml`.
- **Observability Links:** Defined in `api-metadata.yaml`.
- Exceptions & Tech Debt

Are you requesting an exception to standard governance rules?

- ☒ No.
- ☐ Yes.

Exception Category	Justification	Remediation Plan
[e.g., Auth Standard]	[Legacy client cannot support OIDC]	[Migrate client by Q4]

## SLA Configuration (sla-config.yaml)

This file defines the operational "Promise" (SLA) and the "Watchdog" (Alerting) for the API. It is machine-readable and used by the CI/CD pipeline to provision monitoring.

### Example File

```

schemaVersion: "v1.0"
1. Global Defaults (The Floor)
These apply to all operations unless overridden.
defaults:
 availability: 99.9%
 latency:
 p95: 500ms
 p99: 2000ms
 throughput: 1000 # TPS

2. Per-Operation Overrides (The Reality)
operations:
 - operationId: getInventoryStatus # Matches OAS operationId
 latency:
 p95: 50ms # Hot path, must be fast

 - operationId: generateMonthlyReport
 latency:
 p95: 5000ms # Batch job, allowed to be slow

3. Alerting Policy (The Configuration)
Defines when to wake people up.
alerts:
 - name: HighErrorRate
 condition: "error_rate > 5% for 5m"
 severity: P1-Critical
 channel: PagerDuty-OnCall

 - name: LatencyDegradation
 condition: "p95_latency > threshold for 10m"
 severity: P2-Warning
 channel: Slack-#inventory-alerts

4. Business Metrics (The Value)
metrics:
 - name: business.orders.placed
 type: Counter
 description: "Total count of successful orders"

```

## JSON Schema for sla-config.yaml

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "API SLA Configuration",
 "type": "object",
 "properties": {
 "schemaVersion": {
 "type": "string",
 "enum": ["v1.0"]
 },
 "defaults": {
 "type": "object",
 "properties": {
 "availability": { "type": "string", "pattern": "^\\d{2}\\.|\\d{1,3}%$" },
 "latency": {
 "type": "object",
 "properties": {
 "p95": { "type": "string", "pattern": "^\\d+ms$" },
 "p99": { "type": "string", "pattern": "^\\d+ms$" }
 },
 "required": ["p95"]
 },
 "throughput": { "type": "integer", "minimum": 1 }
 },
 "required": ["latency"]
 },
 "operations": {
 "type": "array",
 "items": {
 "type": "object",
 "properties": {
 "operationId": { "type": "string" },
 "latency": {
 "type": "object",
 "properties": {
 "p95": { "type": "string", "pattern": "^\\d+ms$" }
 }
 }
 },
 "required": ["operationId"]
 }
 },
 "alerts": {
 "type": "array",
 "items": {
 "type": "object",
 "properties": {
 "name": { "type": "string" },
 "condition": { "type": "string" },
 "severity": { "type": "string", "enum": ["P1-Critical", "P2-Warning", "P3-Info"] },
 "channel": { "type": "string" }
 },
 "required": ["name", "condition", "severity", "channel"]
 }
 }
 }
}
```

```

 }
 },
 "metrics": {
 "type": "array",
 "items": {
 "type": "object",
 "properties": {
 "name": { "type": "string" },
 "type": { "type": "string", "enum": ["Counter", "Gauge", "Histogram"] },
 "description": { "type": "string" }
 },
 "required": ["name", "type"]
 }
 },
 "required": ["schemaVersion", "defaults", "alerts"]
}

```

## API SDLC Stage 2: Implementation & Testing

### Overview

**"Does the code match the contract, is the performance documented, and are the operations defined?"**

Stage 2 focuses on building the service, validating it against the locked API Contract from Stage 1, and enriching the artifact bundle with implementation-specific details. Governance in this stage shifts from "Design Review" to "Pipeline Validation."

- **Trigger:** Stage 1 PR Merged (Contract Locked).
- **Input:** Approved OpenAPI Specification + NFRs from `API-CHARTER.md`.
- **Output:** A deployable, verified artifact bundle containing Code, Docs, and Operational Manifests.

### Roles & Responsibilities (RACI)

Role	Responsibility	Focus Area
Development Team	Accountable	Writes code, fixes test failures, <b>implements instrumentation (AppD/ELK), and configures Alerting Rules/Dashboards.</b>
QA / SDET	Responsible	Owns the test suite execution, generates the Performance Report, and <b>verifies (via Pipeline or Manual Inspection) that Alerts trigger during Load Testing.</b>



<b>Platform Engineering</b>	<b>Consulted</b>	Provides CI/CD runners and runners for contract testing.
<b>Security Engineer</b>	<b>Consulted</b>	Reviews SAST/DAST findings if vulnerabilities block the gate.

## Key Activities

### 1. Contract-First Implementation

Developers use the approved `openapi.yaml` to drive development.

- **Server Stubs:** Generate boilerplate code from the spec (e.g., via OpenAPI Generator) to ensure routing matches the contract exactly.
- **SDK Generation:** Generate client SDKs for downstream consumers to use in integration tests.
- **Operational Endpoints (Mandatory):** Every API must expose standard observability endpoints to support Kubernetes Liveness and Readiness probes.

### 2. Documentation & SLA Enrichment

Since specific operational details are only known after coding and testing, the Developer must manually enrich the repository before the Release PR.

- **Error Documentation:** Developer commits `error-codes.md`, mapping specific business error codes (e.g., ERR-4002) to human-readable remediation steps.
- **Release Notes:** Developer commits `release-notes.md` detailing the value delivered in this version.
- **Operational Manifest (`sla-config.yaml`):** Upon receiving the load test report from QA, the Developer creates/updates the `sla-config.yaml` file. This acts as a formal **Attestation of Performance**, defining the specific latency thresholds and alert rules that will be provisioned in the monitoring platform.

### 3. Testing Layers

The implementation must pass multiple validation layers:

- **Unit Testing:** Validates business logic in isolation.
- **Contract Testing:** Validates that the provider (API) strictly adheres to the OpenAPI schema (Dredd/Schemathesis).
- **Performance Testing:** Validates that the API meets the NFRs defined in the Charter. Crucially, the metrics verified here are what the developer must submit in the `sla-config.yaml`.
- **Security Testing:** Scans code dependencies (SCA) and endpoints (DAST) for vulnerabilities.

- **Business Acceptance Testing (BAT):** Validates that the API fulfills the end-to-end business requirements and user journeys (e.g., "Create Order" --> "Process Payment" --> "Generate Invoice"). This confirms the API solves the actual business problem defined by the Product Owner.

#### 4. Observability & Alerting Strategy

Before deployment, the service must be instrumented to emit data, and the monitoring platform must be configured to visualize that data and alert on breaches.

**Responsibility: The Development Team** is responsible for both the code instrumentation and the alert configuration.

- **Instrumentation (The Data):**
- **Tracing:** Distributed tracing (OpenTelemetry/AppD Agents) with context propagation.
- **Metrics:** Standard "RED" metrics (Rate, Errors, Duration) and custom Business Metrics.
- **Logs:** Structured JSON logging (ELK) with correlation IDs.
- **Configuration (The Consumption):**
- **Alerting Rules:** The Developer must configure alerts in AppDynamics or ELK that align strictly with the `sla-config.yaml`.
  - *Requirement:* If the Manifest says "P95 < 500ms", a corresponding Health Rule must exist in AppD to trigger if P95 > 500ms.
  - **Service Dashboard:** A standardized dashboard must be created showing the API's "Golden Signals" (Latency, Traffic, Errors, Saturation).

#### Governance Gates

In Stage 2, "Governance" is synonymous with "The Build Pipeline" or "QA Verification".

1. **Automated Checks (Blocking)**
2. **Contract Fidelity:** Does the implementation response match the OpenAPI schema?
3. **Completeness Gate (Artifact Check):** The pipeline checks for the existence of required files.
4. *Fail if missing:* `error-codes.md`, `release-notes.md`, `sla-config.yaml`.
5. **Security Gate:** Zero Critical/High vulnerabilities in dependencies or code.
6. **Quality Gate:** Code coverage > 80% and no "Code Smells".
7. **Operational Readiness Gate (Blocking)**
8. **Alert Configuration Check:**

9. **Option A (Automated):** The pipeline validates that an Alert Policy file exists (e.g., `appd-rules.json`) and that its thresholds match the values in `sla-config.yaml`.
10. **Option B (Manual QA):** QA visually verifies that the active Health Rules in the monitoring tool (e.g., AppDynamics) match the SLA thresholds and attaches a screenshot/link to the Performance Report.
11. **Dashboard Check:**
12. **Option A (Automated):** The pipeline validates that a `dashboardUrl` link is present in the `api-metadata.yaml`.
13. **Option B (Manual QA):** QA verifies that the Dashboard exists, is accessible, and was populating data correctly during the load test.

## The Feedback Loop

Often, developers discover during coding that the Spec (Stage 1) is flawed or needs adjustment.

- **The Rule:** The Code must never drift from the Spec.
- **Scenario:** Developer realizes `orderId` needs to be a string (UUID), not an integer.
- **Process:**
- **Stop Implementation:** Do not just change the code.
- **Update Spec:** Create a hotfix branch for the Spec (`fix/order-id-type`).
- **Mini-Review:** Submit a PR for the Spec change.
  - *If Breaking:* Requires full Stage 1 Architect review.
  - *If Non-Breaking:* Auto-approved or fast-tracked.
  - **Resume:** Once the Spec PR is merged, regenerate stubs/tests and continue implementation.

## Artifacts & Storage

- **Code Repository:** Contains source code, tests, Dockerfiles, **and Alerting Configuration**.
- **Nexus (Master Bundle):** The final output contains the "Service Bundle":
  - `openapi.yaml` (Public Contract)
  - `api-metadata.yaml` (Taxonomy)
  - `error-codes.md` (Docs)
  - `release-notes.md` (Docs)
  - `API-CHARTER.md` (Internal Architecture)

- sla-config.yaml (Internal Ops Manifest)

## Template: error-codes.md

Purpose:

While the OpenAPI Specification (OAS) defines the structure of error responses (the schema), it is impractical to document every specific business logic error variant inside the YAML itself. This file serves as the definitive registry of all possible Error Codes and Messages returned by the API, organized by Endpoint and Verb.

### Governance Check:

- Do the codes listed here follow the <domain>.<category>.<reason> string format?
- Are 401, 403, and 500 errors documented in the Global section?
- Do endpoint-specific errors map to the correct HTTP status?

```
Error Code Registry
API Name: [Inventory Service API]
Version: v1.0

Global Errors
These errors apply to all endpoints in the API and adhere to the **LPDP-Mini v1.0** stan

| HTTP Status | Error Code (String) | Standard Message | Remediation / Cause |
| :--- | :--- | :--- | :--- |
| `400` | `request.invalid_json` | "Request body is not valid JSON." | Client sent malfo
| `401` | `auth.missing` | "Authorization header is required." | Supply a valid Bearer t
| `403` | `auth.forbidden` | "Caller not permitted to perform this operation." | Token i
| `429` | `traffic.limit_exceeded` | "Rate limit exceeded." | Slow down requests. Check
| `500` | `system.unknown` | "Unexpected system condition." | Internal failure. Retry on
| `503` | `dependency.failed` | "Backend dependency failed." | A downstream service (e.g

Endpoint-Specific Errors

`POST /inventory/reservations`
Summary: Create a new stock reservation.

| HTTP Status | Error Code (String) | Message Pattern | Trigger Condition |
| :--- | :--- | :--- | :--- |
| `404` | `resource.not_found` | "No StockItem matched the provided SKU." | The `sku` pr
| `409` | `resource.conflict` | "StockItem is already reserved." | Race condition; anoth
| `422` | `validation.min_qty` | "[quantity] must be at least 1." | Client tried to rese
| `422` | `validation.max_qty` | "[quantity] cannot exceed 50." | Bulk reservation limit

`GET /network/vrfs/{id}`
Summary: Retrieve VRF details.

| HTTP Status | Error Code (String) | Message Pattern | Trigger Condition |
```

```
| :--- | :--- | :--- | :--- |
| `404` | `resource.not_found` | "No VRF matched the provided name and customer." | ID i
| `409` | `resource.ambiguous` | "More than one VRF matches the provided criteria." | Da
| `503` | `dependency.timeout` | "VRF service did not respond." | Timeout from the legac
```

## Template: release-notes.md

Purpose:

To communicate the value delivered and technical evolution of the API to consumers. This file acts as the "News Feed" for the service.

### Governance Check:

- Are releases dated and versioned?
- Are "Breaking Changes" clearly highlighted?
- Is the "Value" clear (not just "fixed bug", but "fixed bug that caused X")?

```
Release Notes
Service: [Inventory Service API]

[v1.2.0] - 2024-03-15
🚀 New Features
* **Bulk Reservations:** Added `POST /inventory/reservations/bulk` to allow reserving up
 * *Value:* Reduces latency for partners processing large orders by 40%.
* **Enhanced Error Details:** Adoption of **LPDP-Mini v1.0**. All errors now return cons

🐛 Bug Fixes
* **Race Condition:** Fixed `resource.conflict` errors occurring when two users cart the
* **Validation:** Fixed an issue where `validation.cidr_invalid` was incorrectly trigger

⚠️ Deprecations
* **Field:** `sku_legacy_id` is now deprecated in favor of `global_sku`.
 * *Timeline:* Field will be removed in v2.0 (Q4 2025).

[v1.1.0] - 2024-01-10
🚀 New Features
* Initial release of the **Global Inventory Lookup**.
* Added support for `resource.ambiguous` error code when searching by partial name match

🔒 Security
* Enforced stricter `auth.forbidden` rules. Users with `viewer` role can no longer acces
```

## API SDLC Stage 3: Deployment & Release

## Overview

Stage 3 ensures that the validated **Logical Contract** (OAS + metadata) is safely translated into the **Physical Runtime** (Gateway configuration, policies, routing, rate limits, and security).

The focus is: **Is the infrastructure aligned, is the gateway synced, and is the rollout safe?**

**Trigger:** All Stage 2 gates passed; Artifact Bundle is production-ready.

**Input:** Verified Artifact Bundle

**Output:** Live API serving traffic with operational baselines active.

---

## Roles & Responsibilities (RACI)

Role	RACI	Focus
DevOps / SRE	Accountable	Deployment pipeline, rollout safety, rollback automation
Gateway Team	Responsible	Gateway ingestion of OAS, route/plugin/policy generation
Development Team	Consulted	Service-level verification during and after rollout
Security Engineer	Consulted	Runtime security posture (MTLS, OAuth, WAF)
API Governance WG	Informed	Catalog updates, release traceability

---

## Key Activities

### Pre-Deployment (Infrastructure & Config)

Environment is prepared before any customer traffic is shifted.

- **Infrastructure Provisioning:** IaC applies updates to autoscaling groups, LBs, health checks.
- **Gateway Sync:** Pipeline converts OAS + metadata into runtime configuration:
- **Routes** → created from paths
- **Policies** → rate limits, authn/authz, transformations
- **Security** → TLS, JWT validation, mTLS posture
- **DNS & Certificates:** DNS records staged; certificates validated/rotated.

- **Approvals:**
  - Lower environments → automated
  - Production → CAB approval
- 

## Deployment Execution (Rollout Strategy)

Uses modern safe-deployment techniques to minimize customer impact.

- **Blue-Green:** Stand up Green alongside Blue; switchover after validation.
- **Canary:** Gradually shift traffic (1% → 10% → 50% → 100%) to detect issues early.
- **Traffic Routing:** Gateway directs requests to the new version via weighted routing.
- **Rollback:** Automatic rollback if:
  - Error rate increases > 1%
  - P95 latency deviates beyond SLO
  - Health probes fail consecutively

Rollback execution is automated and SRE-owned.

---

## Post-Deployment Validation

Ensures runtime behavior matches Stage 2 baselines.

- **Synthetic Checks:** Directed probes verify uptime & correct responses.
  - **SLO Verification:** Compare latency, throughput, and error rates with baselines from `sla-config.yaml`.
  - **Security Posture:** WAF, OAuth/JWT verification, mTLS, and denied-paths are validated.
  - **Logging & Observability:** Confirm structured logs, traces, metrics are flowing end-to-end.
- 

## Environment Progression (“Path to Live”)

### Development

- Auto-deploy on merge
- Purpose: rapid developer feedback

### Test / QA

- Auto-deploy + Automated Test Suite
- Purpose: functional correctness

## Staging

- Auto-deploy + Approval Gate
- Purpose: performance, security, and production-like validation

## Production

- CAB approval + Canary rollout
  - Purpose: safe release, zero downtime
- 

# Governance Checkpoints

## Automated Checks (Pipeline Controls)

- **Spec-to-Gateway Sync:** Confirms gateway config matches OAS (no drift).
  - **Policy Compliance:** Validates required policies (e.g., rate limiting, CORS, auth).
  - **Certificate Validity:** Ensures certificates are valid and not nearing expiry.
  - **Health Checks:** /health and /ready return 200.
  - **Smoke Tests:** Minimal happy-path scenarios executed automatically.
- 

## Manual / Process Checks (Release Gates)

- **Runbook Verification:** Operational runbook exists and is accessible.
- **Disaster Recovery:** DR plan documented and linked in service metadata.
- **Monitoring Sign-off:** Dashboards receiving real-time telemetry (no flatlines).
- **Security Policy Confirmation:** Authentication/authorization enforced; no unprotected endpoints.

# API SDLC Stage 4: Publication & Enablement

## Objective

*"Is the API discoverable, usable, and supported?"*

Stage 4 converts the API from a runtime technical asset into a fully discoverable Product. Governance in this stage focuses on the "Consumer Experience" (DX) to ensure that the API is not just functional, but also easy to adopt.

- **Trigger:** Stage 3 (Deployment) complete; API is live in Production and the Sandbox Environment is active.
- **Input:** Live Endpoints, `api-metadata.yaml`, and Validated SDK Artifacts (from Stage 2).
- **Output:** A published entry in the Developer Portal, active "Try It" capability linked to Sandbox, and accessible Client Libraries.



## Roles & Responsibilities (RACI)

Role	RACI	Focus Area
Dev Portal Team	Accountable	<b>Platform Experience.</b> Orchestrates the entire publication lifecycle: ingestion, SDK promotion, catalog registration, and key provisioning workflows.
Development Team	Responsible	<b>Content &amp; Runtime Owner.</b> Responsible for the accuracy of documentation/guides and the <b>reliability/uptime of the Sandbox environment.</b>
API Gov WG	Consulted	Reviews categorization and tagging for catalog consistency.
Marketing / Comms	Consulted	Coordinate announcements for major/public releases.

## Key Activities

### 1. Automated Platform Ingestion & Asset Promotion

The Developer Portal Platform acts as the orchestrator. It scans the repository metadata and performs the following actions automatically:

- **Catalog Registration:**
  - *Service Identity:* Populates Ownership, Domain, and Support Contacts from `api-metadata.yaml`.
  - *API Reference:* Renders the interactive `openapi.yaml`.
  - *Observability:* Displays real-time health status. Publication is not complete until the service status is discoverable from the Portal entry.
  - *Search Indexing:* Indexes the API by capabilities, domain, and audience.
- **Asset Promotion (Client SDKs):**
  - Pushed to Public Registries (npm, PyPI) and Public GitHub (Source Mirror) after passing a mandatory Secret Scan.
- **Sandbox Registration (Access, not Provisioning):**
  - The Portal binds the documentation's "Try It" button to the **External Sandbox URL** (already deployed in Stage 3).
  - It verifies the Sandbox is reachable before publishing the page.
- **Monetization & Entitlements (If Applicable)**

For APIs participating in billing or internal chargeback models, the business rules must be visible to the consumer.

- **Plan Visibility:** Display available tiers (e.g., "Free," "Gold," "Enterprise") with their respective quotas.
- **Pricing Display:** Ensure cost-per-call or monthly subscription fees are clearly listed.
- **Onboarding, Support & Enablement (Portal Experience)**

The Portal Platform ingests and renders the following assets to drive self-service adoption:

- **Guided Onboarding:** Renders the page with approved artifacts as a rich "Getting Started" guide.
- **Support Documentation:** Renders `error-codes.md` and `release-notes.md`.
- **Interactive Code Samples:** Automatically generates or renders validated snippets (curl, Python, JS) embedded within the documentation.
- **Key Provisioning:** Enables the "Get Access" workflow based on `api-metadata.yaml`.
- **Sandbox Access:** Users can immediately generate a Client ID/Secret for the **Sandbox Environment** (Self-Service).
- **Prod Access:** If `approval: manual` is set, the Portal triggers the approval workflow notification to the Product Owner. If `approval: auto`, keys are issued immediately.

### Self-Service Consumer Experience (The Goal)

Governance success is measured by the ability of a consumer to complete this loop without emailing the API team:

1. **Browse:** Find "Inventory API" via search tags.
2. **View:** Read the `openapi.yaml` to understand the data model.
3. **Evaluate:** Review Pricing Plans & Quotas to select the right tier.
4. **Try:** Use the Sandbox credentials to run a curl command using the Code Samples.
5. **Access:** Generate a Production API Key via the Provisioning Workflow or download the SDK.
6. **Monitor (Consumer Dashboard):** The consumer logs into the portal to see their Personal Usage Metrics:
  7. *My Request Volume* (Am I growing?)
  8. *My Quota Usage* (Am I about to be rate-limited?)
  9. **Support:** View the Error Codes page when something breaks.

### Governance Checkpoints

### Automated Checks (The Pipeline)

- **Documentation Build:** Did the Static Site Generator (SSG) build the docs without errors?
- **Link Validation:** Are there any 404s in the description links?
- **Example Code Validation:** The pipeline extracts code blocks from the docs and runs them against the **Sandbox Environment** to ensure they actually work.
- **SDK Availability:** Verifies that the SDK packages are successfully reachable in the target Registry (Nexus/Public).
- **Security Scan (Public Only):** Verifies no secrets are present in artifacts heading to public GitHub/Registries.

### Manual/Process Checks (The Gates)

- ☒ **"First 5 Minutes" Test:** A random dev from another team tries to make a successful API call using only the documentation.
- ☒ **Sandbox Veracity:** Does the Sandbox behave like Production? (Verified by Development Team).
- ☒ **Consumer Analytics Verification:** Verify that the "My Usage" dashboard correctly displays data when test calls are made.
- ☒ **Pricing/Chargeback:** If this is a billable API, is the cost model clearly visible and accurate?
- ☒ **Versioning Integrity:** Ensure the published version matches `api-metadata.yaml` and does not overwrite or hide prior supported major versions.

### Artifacts & Outcomes

- **Developer Portal Entry:** The "Storefront" for the API.
- **Published SDKs:** Packages in target registries (Internal or Public).
- **Consumer Dashboard:** Enabled views for specific API Keys/Users.
- **Announcement:** Email/Slack blast to the consumerAudience defined in metadata.

## API Governance: Operations & Continuous Improvement

### Objective

#### "Is the API healthy, valuable, and compliant?"

This process defines the "Day 2" activities that occur continuously after an API is live. Governance here shifts from "Gatekeeping" to **"Auditing & Reporting"** (monitoring runtime health, standards compliance, and business value).

- **Trigger:** API is active in Production.
- **Input:** Live Traffic, Support Tickets, Cost Reports, and Vulnerability Scans.
- **Output:** Optimization Backlog, Compliance Reports, and Portfolio Health Scores.

### 1. Roles & Responsibilities (RACI)

Activity Domain	Accountable (Owner)	Responsible (Doer)	Executive Oversight
<b>Monitoring &amp; Observability</b>	<b>SRE / Platform Lead</b>	<b>Dev Team</b> (Instrumentation) <b>SRE</b> (Infrastructure) <b>Product Owner</b> (Business & Cost Signals)	<b>Steering Committee</b> (Receives Availability Reports)
<b>Consumer Feedback Loop</b>	<b>Product Management</b>	<b>Product Owner</b> (Analysis) <b>Customer Success</b> (Outreach)	<b>Steering Committee</b> (Receives Adoption Reports)
<b>Continuous Governance</b>	<b>Product Management</b>	<b>API Gov WG</b> (Audits) <b>Security Team</b> (Scanning) <b>Dev Team</b> (Remediation)	<b>Steering Committee</b> (Receives Risk/Compliance Reports)

## 2. Best Practices & Activities

### A. Monitoring & Observability (The Watchtower)

Measure technical and business health in real-time to prevent degradation.

- **Technical Signals (SLI/SLO):** SRE would
- **Practice:** Track "Burn Rate" against the availability target (e.g., 99.9%) defined in `s1a-config.yaml`.
- **Action:** Alerts must trigger *before* the error budget is exhausted, allowing SREs to intervene proactively.
- **Distributed Tracing:** Dev Team would
- **Practice:** Implement full trace propagation (W3C Trace Context) to analyze end-to-end latency across microservices.
- **Action:** Identify and refactor "Hot Paths" that cause user latency > P95 targets.
- **Business & Cost Signals:** Product owner would
- **Practice:** Calculate "Cost per Transaction" (Infra Cost / Request Volume) to track unit economics.
- **Action:** Report feature adoption rates to Product Management to identify high-value vs. low-value assets.

### B. Consumer Feedback Loop (The Voice)

Actively listen to data and consumers to drive the product roadmap.

- **Usage Analytics & Zombie Detection:** Gateway/SRE would

- **Practice:** Identify endpoints with zero or near-zero usage over 90 days ("Zombie APIs").
- **Action:** Flag these for Deprecation to reduce attack surface and maintenance cost.
- **Friction Analysis:** Product Owner would
- **Practice:** Correlate Support Tickets with high rates of 400 Bad Request or 401 Unauthorized errors.
- **Action:** Update error-codes.md or the Developer Portal guides to address common user errors.
- **Advisory Board:**
- **Practice:** Hold quarterly feedback sessions with key strategic consumers.
- **Action:** Direct input into the API Roadmap for major version enhancements.

### C. Continuous Governance (The Guard)

Automated checks ensure the live service does not degrade or drift from standards over time.

- **Security Posture Management:**
  - **Practice:** Continuous scanning of running containers for new CVEs and configuration drifts.
  - **Action:** Enforce remediation SLAs (e.g., Criticals fixed within 48 hours).
  - **Certificate Management:**
  - **Practice:** Automated alerting 30 days prior to TLS certificate expiration.
  - **API Health Scoring:**
  - **Practice:** Assign a composite score (0-100) to every API based on: *Uptime + Documentation Completeness + Security Score + Standards Compliance*.
  - **Action:** Low-scoring APIs (< 70) are flagged for mandatory "Service Improvement Plans."
- 

## 3. Executive Reporting (Steering Committee)

The **API Governance Steering Committee** receives the **Monthly Executive Report** to drive enterprise strategy adjustments:

1. **Portfolio Health:** Overall Availability and Performance trends across the enterprise.
2. **Risk Profile:** Number of APIs with unresolved High/Critical vulnerabilities or expired certificates.

3. **Compliance Rate:** Percentage of APIs fully adhering to Design and Documentation standards.
4. **Strategic Value:** Top 10 most used APIs and bottom 10 "Zombie" candidates for retirement.

## API Governance: Reporting & Exception Management

### Purpose

This document defines how API governance findings are evaluated, reported, waived, approved, tracked, and surfaced within the centralized API Governance repository.

The model is designed to:

- Translate low-level linting output into product-level risk and planning signals
- Enable developer progress without hiding technical debt
- Make governance auditable, repeatable, and automation-first
- Communicate consumer trust and risk via a computed governance state

Governance is treated as a product capability, not a compliance checklist.

---

### Key Concept: Two Reports, Different Intent

The pipeline produces two distinct reports, each with a Summary and a Detail section.

Report	Purpose	Audience
Summary & Findings Report	Full compliance view (before exceptions)	PMs, Delivery Leads, Architects, Engineers
Tech Debt Report	Accepted technical debt under an approved exception	PMs, Governance, Leadership

Both reports use the same category model and severity model to remain comparable.

---

### Severity Model

The governance system uses three severity levels, consistently across both reports:

#### **ERROR — Blocking**

Critical risk (security exposure, breaking change, invalid contract).

Target state: must be resolved before GA or public release, unless explicitly accepted via an approved exception.

### **WARN — Significant Debt**

Non-blocking deviations that materially increase long-term operational cost, support burden, or client confusion if left unresolved.

### **HINT — Best Practice / Polish**

Non-blocking quality and consistency improvements.

Any INFO findings are normalized to HINT for reporting consistency.

---

## **Report 1: Summary & Findings Report**

### **Purpose**

This report represents the ground truth of the API contract against governance rules before exceptions are applied.

It answers:

- What issues exist?
  - Where is the risk?
  - What would block release if no waivers were granted?
- 

### **Report 1 – Summary Section (PM & Delivery Lead Focused)**

The Summary provides a flattened, category-level view of all findings.

Canonical Summary Format

```
[
 {
 "category": "URI & Resource Design",
 "ERROR": 3,
 "WARN": 2,
 "HINT": 1,
 "TOTAL": 6
 },
 {
 "category": "HTTP Semantics & Protocol Usage",
 "ERROR": 2,
 "WARN": 3,
 "TOTAL": 5
 }
]
```

This format is the canonical PM-facing contract and is used for:

- Planning

- Dashboards
  - Jira automation
  - Trend analysis
- 

## How PMs & Delivery Leads Use Report 1

- **Sizing** – TOTAL approximates remediation scope
  - **Planning** – Create one Epic per category with ERROR/WARN > 0
  - **Prioritization** – Resolve ERRORS first
  - **Readiness** – Determine whether a full exception is required
- 

## Report 1 – Detailed Findings (Engineering Focused)

Each entry represents a single governance violation.

Field	Description
Rule	Unique rule identifier
Category	Governance category
Severity	ERROR, WARN, HINT
Message	Human-readable explanation
Location	JSONPath / JSON Pointer
Source	File (e.g., openapi.yaml)

---

## API Governance Report – Category Model (PM-Facing)

The following categories are stable, mutually exclusive, and serve as the unit of planning and prioritization.

Category	What It Covers	Why It Matters to PMs	Mapped Rules / Standards
URI & Resource Design	URI naming, resource naming, collection vs item paths, search semantics, taxonomy alignment	Cross-API consistency and evolvability	resources, API URI Standard Proposal
HTTP Semantics & Protocol Usage	HTTP verbs, status codes, idempotency, headers, pagination	Predictable client behavior and tooling compatibility	get-method, post, put, patch, delete, idempotency, headers, pagination
Versioning & Compatibility	Versioning rules, breaking-change detection, deprecation policy enforcement	Prevents breaking clients and uncontrolled migrations	versioning, breaking-change rules, deprecation checks



Category	What It Covers	Why It Matters to PMs	Mapped Rules / Standards
Data Contract (Payload, Schema, Errors & Types)	JSON payloads, Problem Details, enums, required fields, data formats	Client stability and interoperability	json-payload, problem-details, data-types-formats, date-time
Security & Compliance	OWASP checks, auth misuse, sensitive data exposure	Regulatory and brand risk reduction	owasp
OAS Specification Quality	Structural correctness of the OpenAPI document	CI/CD and tooling reliability	oas-best-practice-rules.yaml, OAS Best Practices

## Key Governance Constraints

- Each Spectral rule maps to exactly one category
- Categories are mutually exclusive
- Categories are stable over time
- Categories are reporting and prioritization units, not implementation details

## Exception & Waiver Model

### Core Principle

Developers declare exceptions.

The pipeline enforces approval, visibility, and governance state.

Exceptions allow progress without erasing visibility of risk.

### Exception Types

#### Full Exception (Governance Opt-Out)

Applies to the entire API.

- Governance checks still run and report truth
- Gating is bypassed
- Used only for exceptional, time-bound cases
- Accepted debt remains fully visible in the Tech Debt Report

Scoped (partial) exceptions are not supported due to the stateless nature of PR validation and re-linting.

# Developer Input: governance-exceptions.yaml

Exceptions are declared via a structured file stored alongside the OAS.

---

## Canonical File Header (Required)

```
schemaVersion: v1
apiExposure: public | internal
apiOrigin: greenfield | existing
governanceProfile: full | basic
```

---

## Full Exception Example (Public or Internal – Entire API)

```
schemaVersion: v1
apiExposure: public
apiOrigin: existing
governanceProfile: full

fullException:
 scope: entire-api
 reason: Transitional partner API; replacement contract will be published post-Q4
 ticket: JIRA-99999
 owner: partner-platform-team
 expiresOn: 2026-01-31
 signOff:
 type: steering-committee
 confluenceUrl: https://confluence.company.com/x/XYZ789
```

---

## Full Exception (Internal API – Product Owner Sign-Off)

```
schemaVersion: v1
apiExposure: internal
apiOrigin: existing
governanceProfile: full

fullException:
 scope: entire-api
 reason: Legacy platform API pending decommission; no remediation planned
 ticket: JIRA-88888
 owner: legacy-platform-team
 expiresOn: 2026-01-31
 signOff:
 type: product-owner
 name: Jane Doe
 role: Product Owner
 email: jane.doe@company.com
 approvalDate: 2025-09-12
```

---

## Pipeline-Enforced Rules (Authoritative)

- schemaVersion is mandatory
  - apiOrigin (greenfield | existing) is mandatory
  - apiExposure (public | internal) is mandatory
  - governanceProfile values: basic, full only
  - Developers cannot modify governanceLevel manually
  - Full exceptions must use fullException
  - reason, ticket, owner, expiresOn, and signOff are mandatory
  - Expired exceptions fail the pipeline
- 

## Approval Rules

- Public APIs → Product Owner and Steering Committee
  - Internal APIs → Product Owner only
- 

## Governance Level Computation (Pipeline-Owned)

governanceLevel is a terminal state.

If the pipeline fails, no governanceLevel is written.

Allowed Values

- Fully Governed
- Lift and Shift
- Exception

Partially Governed is not emitted because scoped (partial) exceptions are not supported.

---

## Decision Tables (Authoritative)

### Public APIs — Greenfield

Profile	Blocking	Exception	Approval	Result	governanceLevel
full	none	none	—	pass	Fully Governed
full	any	full	PO + Steering	pass	Exception
basic	—	—	—	fail	not emitted

---

## Public APIs — Existing

Profile	Blocking	Exception	Approval	Result	governanceLevel
full	none	none	—	pass	Fully Governed
full	any	full	PO + Steering	pass	Exception
basic	WARN-only	none	—	pass	Lift and Shift

## Internal APIs — Greenfield & Existing

Profile	Blocking	Exception	Approval	Result	governanceLevel
full	none	none	—	pass	Fully Governed
full	any	full	PO	pass	Exception
basic	WARN-only	none	—	pass	Lift and Shift

## Report 2: Tech Debt Report

### Purpose

The Tech Debt Report captures only what the team chose not to fix under an approved exception.

It answers:

- What debt exists?
- Why was it accepted?
- Who approved it?
- When does it expire?

This report is generated by the pipeline.  
Developers never author it.

### Location

/docs/tech-debt.md

## Report 2 – Summary Section (Accepted Debt Only)

```
[
 { "category": "HTTP Semantics & Protocol Usage", "ERROR": 1, "TOTAL": 1 },
```

```
{ "category": "URI & Resource Design", "WARN": 2, "TOTAL": 2 }
]
```

---

## Report 2 – Tech Debt Context (Authoritative)

This section mirrors the approved full exception exactly and is written once per report:

```
techDebtContext:
 scope: entire-api
 reason: Transitional partner API; replacement contract will be published post-Q4
 ticket: JIRA-99999
 owner: partner-platform-team
 expiresOn: 2026-01-31
```

No per-finding repetition of ticket/owner/expiresOn is allowed in the detail sections.

---

## Report 2 – Detailed Findings (By Severity)

### Accepted Errors (ERROR)

Category	Rule	Location / Scope	Message

All accepted ERROR findings are governed by the single Jira ticket and expiry date defined in Tech Debt Context.

---

### Accepted Warnings (WARN)

Category	Rule	Location / Scope	Message

---

### Accepted Hints (HINT)

Category	Rule	Location / Scope	Message

---

## Auditing

Once a terminal state has been reached, the pipeline creates an audit record in `api-metadata.yaml` capturing:

- the date the spec was reviewed

- the names of reviewers who approved the spec
- or, in the case of exception, the date the exception was granted and names of reviewers

## api-metadata.yaml — Review Audit Trail (Terminal Record)

```
reviewAuditTrail:
 reviewId: GOV-2026-00192
 reviewDate: 2026-01-05T14:22:31Z
 reviewStatus: Approved | Exception
 reviewers:
 - name: Jane Doe
 role: Product Owner
 - name: API Steering Committee
 securityReviewId: SEC-REV-10455
```

## How This Fits into the PR Workflow

This governance model is enforced entirely through the pull request workflow, ensuring that governance decisions are transparent, auditable, and difficult to bypass.

### End-to-End Flow

- Developer opens PR with OpenAPI
- Pipeline runs Spectral
- All applicable governance rules are executed based on governanceProfile
- Findings are classified by category and severity
- Pipeline generates Report 1 (Summary & Findings Report)
- Represents the ground truth
- No exceptions applied
- Used for planning, prioritization, and readiness assessment
- Developer responds to findings
- Fixes issues where possible
- Adds governance-exceptions.yaml only for items that cannot be fixed immediately
- Submits a full exception when required
- Governance and required authority approve via CODEOWNERS
- Internal APIs: Product Owner sign-off required
- Public APIs: Product Owner request and Steering Committee Confluence sign-off required

- Pipeline re-runs
  - Validates exception structure and approvals
  - Rejects expired or invalid exceptions
  - Pipeline generates Tech Debt Report
  - Output location: /docs/tech-debt.md
  - Includes accepted technical debt under the approved exception
  - Uses Tech Debt Context + Errors/WARN/HINT detail sections
  - Pipeline computes governanceLevel
  - Based on governanceProfile, linting results, approved exception scope, API exposure, API origin
  - governanceLevel is written only if the pipeline succeeds
  - PR passes only if all gating conditions are satisfied
  - Invalid profile selection
  - Missing approvals
  - Unapproved or expired exceptions
  - Manual edits to governanceLevel
  - Any of the above causes the pipeline to fail and no governanceLevel is emitted.
- 

## Outcome

This workflow ensures that:

- Governance decisions are explicit, reviewed, and traceable
- Risk is never hidden, even when progress is unblocked
- Public exposure is protected through additional oversight
- Governance state is earned, not declared

The result is a process that is auditable, repeatable, automation-first, and difficult to game.

---

## Final Notes

This reporting and governance model:

- Treats API Governance as a product capability
- Enables PM-driven planning
- Preserves transparency of risk

- Allows controlled flexibility via explicit waivers (full exception only)
- Ensures accountability:
- Steering Committee for public APIs
- Product Owner for internal APIs

Once adopted, it becomes the shared operational language between Product, Engineering, Architecture, and Governance.

## API Governance: Repository Onboarding - Bootstrap Structure and Check-In Paths

### Purpose

This document defines the governance workflow for onboarding and maintaining OpenAPI Specifications (OAS) in the central governance repository. It establishes a phased approach that supports current authoring tools (e.g., SwaggerHub) while enabling the target state where teams author OpenAPI natively in the governance repo and submit PRs directly.

A key design principle is separation of concerns:

- **Bootstrap:** create the canonical directory structure for an API version.
  - **Check-In:** validate and commit OAS content into that structure, creating a PR and running linting.
- 

## Current Reality and Target State

### Current Reality (Tool-Centric Authoring)

- Teams author OAS in tools like SwaggerHub.
- Teams submit their OAS via a governance intake issue.
- Automation checks content into the governance repo and opens a PR.

### Transitional Reality (Central Repo as Governance Gate)

- Teams already have OAS developed (often single-file).
- They want to “run it through governance” by checking into the central repo.
- They may not yet have modular fragments, but they still need linting and standards validation.

### Target State (Repo-Native Authoring)



- Teams clone the governance repo locally.
  - They create/update `openapi.yaml` and optional fragments.
  - They create a PR directly and governance CI validates it.
- 

## Two Separate Automation Capabilities

### 1) Bootstrap Capability (Directory Structure Creation)

#### Goal

Create the canonical folder structure for an API version in the governance repo even before any OAS content exists (or before content is compliant).

#### When to use

- New API onboarding
- New major version onboarding (v2)
- Setting up a repo workspace for a team transitioning to repo-native authoring

#### Output

A PR that introduces:

```
domain/<domainName>/<majorVersion>/
 openapi.yaml # placeholder or minimal skeleton
 paths/
 components/schemas/
 components/parameters/
 components/headers/
 components/responses/
 components/requestBodies/
 components/securitySchemes/
 components/examples/
 README.md # optional scaffolding guidance
```

#### Behavior

- Always succeeds unless the target path already exists.
- Does not run full semantic linting (only "bootstrap validity" checks).

#### Why separate bootstrap

- Teams can begin collaborating on structure immediately.
- Creates a stable landing zone regardless of whether the spec is ready.
- Enables the target state: "clone → edit → PR".

## Directory and File Purposes

### `openapi.yaml`

#### **Purpose:**

Serves as the **single OpenAPI entrypoint** for the API version.

- MAY contain the full OpenAPI specification as a single file, or
  - MAY act as a lightweight root document that references fragments under `paths/` and `components/**` using `$ref`
  - Is the file used by governance automation, bundling, linting, publishing, and gateway onboarding
- 

### `paths/`

#### **Purpose:**

Holds **Path Item Object fragments**, typically grouped by resource or business capability.

- Used when teams modularize operations instead of defining all paths inline in `openapi.yaml`
  - Files in this directory SHOULD define Path Item Objects (e.g., `/services`, `/orders`)
  - Enables clean separation of large APIs into manageable units
- 

### `components/schemas/`

#### **Purpose:**

Contains **reusable data model definitions**.

- Includes object schemas, enums, and primitive fragments (e.g., `ServiceId`, `CustomerType`)
  - Schemas defined here are referenced from request bodies, responses, and parameters
  - Must not contain path definitions or operation logic
- 

### `components/parameters/`

#### **Purpose:**

Contains **reusable parameter definitions**.

- Used for query parameters, path parameters, and **request headers** (`in: header`)
  - Common examples include pagination parameters, filtering parameters, and correlation IDs
  - Promotes consistency across operations and APIs
- 

### `components/headers/`

**Purpose:**

Contains **reusable response header definitions**.

- Used only for headers returned by the API
  - Must not be used for request headers
  - Referenced from response definitions under components/responses or inline responses
- 

**components/responses/****Purpose:**

Contains **reusable response objects**.

- Includes standard success responses and shared error responses (e.g., 400, 404, 500)
  - Often references schemas and response headers defined elsewhere
  - Helps enforce consistent error handling and response structure
- 

**components/requestBodies/****Purpose:**

Contains **reusable request body definitions**.

- Used for complex or commonly reused request payloads (e.g., search requests, bulk operations)
  - Referenced by operations in paths/
  - Keeps large request schemas out of path definitions
- 

**components/securitySchemes/****Purpose:**

Defines **authentication and authorization mechanisms**.

- Includes OAuth2, mTLS, API key, or other security schemes
  - Referenced by operations or applied globally in `openapi.yaml`
  - Centralizes security configuration for consistent enforcement
- 

**components/examples/****Purpose:**

Stores **reusable example payloads**.

- Used to provide consistent examples across schemas, requests, and responses
  - Improves API documentation quality without cluttering core definitions
  - Optional but recommended for developer experience
-

## 2) Check-In Capability (Spec Intake + Validation + PR)

### Goal

Take OAS content (single-file or bundle), place it into the canonical structure, and run linting/validation as PR checks.

### When to use

- SwaggerHub-developed specs being onboarded
- Existing specs being centralized for linting
- Incremental migration to fragments

### Inputs

- ZIP upload (preferred) or single-file
- Domain + major version

### Behavior

Automation:

1. Extracts into a temporary workspace
2. Detects whether content is:
3. single-file OAS, or
4. fragmented bundle
5. Overlays content into:  
    `domain/<domainName>/<majorVersion>/`
6. Creates a PR
7. CI runs:
8. bundling (single-file output for validation/publishing)
9. Spectral governance rules
10. additional checks (\$ref containment, structure correctness if fragments present)

### Failure handling

- If extraction shows broken structure (e.g., \$ref points outside version folder), automation fails early and updates the issue with actionable errors.
  - If structure is acceptable but content fails linting, PR is still created and CI reports failures (preferred for collaboration).
-

# Submission Modes Supported During Transition

## Mode A: “Already Built Spec” → Central Repo Check-In (Now)

- Developers export:
- one file (`openapi.yaml`) OR
- a bundle (with prescribed directory structure)
- Upload ZIP in issue form
- Automation check-in creates PR and runs linting

## Mode B: Repo-Native Authoring (Target)

This mode supports teams that want to author OpenAPI specifications **directly in the central governance repository** using local tooling (e.g., VS Code), while still preserving a controlled onboarding and branching model.

### Workflow

1. **Developer submits a Bootstrap Issue**
2. Purpose: request creation of the canonical directory structure **and a dedicated working branch** for the API or major version.
3. No OpenAPI content is required at this stage.
4. **Automation creates structure and branch**
5. Creates:  
  
domain/<domainName>/<majorVersion>/ paths/ components/... \* Creates a named branch for the team (e.g., `api/<domain>/<vMajor>/authoring`). \* Opens (or optionally defers) a bootstrap PR, depending on governance policy.
3. **Developer clones the governance repository**
6. Checks out the pre-created branch.
7. Uses local tooling (VS Code, CLI, linters) to author:
  - `openapi.yaml`, and/or
  - reusable fragments under the standard directory structure.
  - **Developer commits OpenAPI content to the branch**
8. Files are added incrementally.
9. Modularization is optional; single-file specs are supported.
10. **Developer opens a Pull Request**

11. The PR contains only **content changes**, since structure and branch already exist.
12. No additional issue form is required for this step.
13. **Governance CI validates the PR**
14. Runs bundling and linting checks.
15. Enforces \$ref containment and directory correctness when fragments are present.
16. Generates governance reports.

### Key Characteristics

- The issue form is still required **once** to:
  - create the directory structure, and
  - establish the working branch.
  - After bootstrap, the workflow becomes **repo-native and PR-driven**.
  - This approach ensures:
  - consistent repository layout,
  - controlled branch creation,
  - a smooth transition away from tool-centric submission models.
- 

## Recommended Phased Rollout

### Phase 1: Bootstrap + Check-In via Issue Form (Immediate)

- Issue form supports:
- "Bootstrap only" (create structure PR)
- "Check-in spec" (upload zip → PR)
- Governance repo becomes system of record.

### Phase 2: Allow Direct PRs for Teams Ready for Repo-Native Authoring

- Teams can bypass issue form.
- They follow directory standard and open PR.
- CI performs same validations.

### Phase 3: Deprecate "paste OAS in form"

- Only ZIP bundle or direct PR authoring remains.
  - SwaggerHub remains optional as an editor, but output must be checked into repo.
-

# Developer Guidance: How to Work in the Repo

## Minimum viable approach (single-file)

- Edit only:  
domain/<domain>/<vMajor>/openapi.yaml

## Modular approach (fragments)

- Add fragments only as needed:
- paths/\*.yaml
- components/schemas/\*.yaml
- components/parameters/\*.yaml
- components/headers/\*.yaml (response headers)
- etc.

## Rules for modularization

- \$ref must remain within:  
domain/<domain>/<vMajor>/
  - Fragment type must match directory (schemas under schemas, etc.)
  - openapi.yaml remains the entrypoint
- 

# Automation: Deterministic Rules

## Bootstrap

- Creates directories
- Writes minimal openapi.yaml placeholder (or empty skeleton)
- Opens PR
- No semantic linting required

## Check-In

- Accept ZIP
- Extract into temp
- Determine:
- single OAS file → map to openapi.yaml
- bundle → overlay into canonical structure
- Create PR

- CI bundles and lints
- 

## Summary

- Separate **Bootstrap** (structure creation) from **Check-In** (content validation and PR).
- Keep SwaggerHub-friendly intake today via ZIP.
- Provide a clean path to repo-native authoring tomorrow.
- Use identical CI validation for issue-driven PRs and developer-created PRs.